

온라인 부록 A

# 파이썬 입문 및 본문 코드 실습

본문 1-5부의 파이썬 코드를 따라가기 위한 실습 부록

통통한 데이터 분석

서울대학교 과학데이터혁신연구소 · 통계학과

## A.0 파이썬 입문

이 부록은 본문의 1부부터 5부까지에 등장하는 파이썬(Python) 코드를, 프로그래밍이 처음인 독자도 따라갈 수 있도록 안내하기 위해 마련되었다. 본 부록 한 편으로 파이썬 전체를 익히기는 어렵지만, 본문에서 등장하는 데이터 분석 코드를 보고 "무슨 뜻인지 짐작할 수 있는 수준"에는 도달할 수 있도록 핵심만 주렸다.

실행용 파일 온라인부록\_파이썬.ipynb을 책의 루트 폴더에 두었다. 루트에서 실행하면 모든 실습 데이터는 데이터/파일명.csv 경로로 불러온다. 부록의 코드 예제는 Google Colab(코랩)에서 검증하였다. 대부분의 기본 예제는 바로 실행할 수 있고, 일부 선택 패키지는 해당 절의 설치 안내에 따라 추가 설치가 필요하다.

### A.0.1 본문 1-5 부와의 연계

각 절의 끝에는 ► **본문 미리보기** 박스가 있다. 그 박스에서는 "여기서 배운 내용이 본문 어디에서 어떻게 쓰이는지"를 한두 줄로 안내한다. 본문을 읽다가 이해되지 않는 코드가 나오면, 해당 박스를 역으로 찾아 부록의 해당 절로 돌아오면 된다.

#### ► 본문 미리보기 — 이 부록 ↔ 본문

- 1장(시각화): pandas, matplotlib, seaborn 절을 미리 읽으면 도움이 된다.
- 2부(모델링 1): pandas와 statsmodels 절을 미리 읽으면 도움이 된다.
- 3부(모델링 2): pandas와 scikit-learn 절을 미리 읽으면 도움이 된다.
- 4장(추론): NumPy의 난수, pandas의 groupby, scipy.stats 절이 핵심이다.
- 5장(패턴 탐색): pandas, scikit-learn 절을 가볍게 훑으면 충분하다.

### A.0.2 파이썬 환경 설정

파이썬(Python)은 1990년대 초 네덜란드의 Guido van Rossum이 처음 개발한 범용 프로그래밍 언어로, 데이터 분석·인공지능·웹 개발 등 다양한 분야에서 가장 널리 쓰이는 언어이다. 파이썬에는 다음과 같은 특징이 있다.

- 문법이 간결하고 읽기 쉽다 — 초보자에게 친숙하다.
- 무료 오픈소스이다 — 누구나 자유롭게 설치하고, 라이브러리를 추가할 수 있다.
- 윈도우, 맥, 리눅스 등 모든 운영체제에서 동일하게 사용할 수 있다.

- NumPy, pandas, matplotlib, scikit-learn 등 데이터 분석을 위한 풍부한 패키지(라이브러리)를 갖추고 있다.
- 오류 메시지가 친절하고, Stack Overflow 와 같은 커뮤니티가 매우 활발하다.

## 파이썬을 실행하는 세 가지 환경

이 책의 모든 코드는 다음 세 환경 중 어느 것에서도 실행할 수 있다. 처음 시작하는 독자에게는 ❶ Google Colab을 가장 권장한다. 설치가 필요 없고, 본인의 노트북 사양과 무관하게 동일한 결과를 얻을 수 있기 때문이다.

- ❶ **Google Colab** — 구글이 제공하는 무료 클라우드 노트북. 별도 설치가 필요 없고, 구글 계정만 있으면 바로 사용할 수 있다.
- ❷ **Anaconda + Jupyter Notebook** — 파이썬과 데이터 분석에 필요한 거의 모든 패키지를 한꺼번에 설치할 수 있는 배포판. 컴퓨터에 직접 설치한다.
- ❸ **Python 공식 + IDE(VS Code 등)** — 가장 가볍게 설치하는 방법이지만, 패키지 관리에 약간의 학습이 필요하다.

## Google Colab 으로 시작하기 (권장)

Colab은 별도 설치 없이 웹 브라우저에서 곧장 파이썬을 실행할 수 있는 환경이다. 구글 드라이브와 연동되어 있어 작성한 노트북이 자동으로 저장된다.

### 준비물

- 구글 계정 (Gmail 계정이면 된다)
- 크롬·엣지·사파리 등 표준 웹 브라우저

### 처음 사용하는 절차

- ❶ 브라우저에서 <https://colab.research.google.com> 에 접속한다.
- ❷ 구글 계정으로 로그인한다.
- ❸ 화면 우측 하단의 [+ 새 노트] 또는 [파일 → 새 노트]를 클릭한다.
- ❹ 빈 노트북이 열리면, 첫 셀에 아래와 같이 입력하고 **Shift + Enter** 를 누른다.

```
print("Hello, Colab!")
1 + 2
```

아래와 같은 출력이 셀 바로 아래에 표시되면 정상적으로 실행된 것이다.

```
Hello, Colab!
3
```

**Tip**

Colab의 파일 이름은 화면 좌측 상단을 클릭해 변경할 수 있다.

셀 좌측의 ▶ 버튼이나 Shift + Enter 단축키로 실행한다.

왼쪽 메뉴의 폴더 아이콘을 누르면 Colab 가상 디스크에 파일을 업로드할 수 있다.

**자주 쓰는 단축키 (Colab)**

- **Shift + Enter** — 현재 셀 실행 후 다음 셀로 이동
- **Ctrl + Enter** — 현재 셀만 실행
- **Alt + Enter** — 실행 후 아래에 새 셀 생성
- **Esc → A** — 위에 새 셀 추가
- **Esc → B** — 아래에 새 셀 추가
- **Esc → D D** — 현재 셀 삭제 (D 두 번)
- **Ctrl + M M** — 셀을 마크다운(설명용)으로 변경
- **Ctrl + M Y** — 셀을 코드로 변경

**내 컴퓨터에 설치하기 (Anaconda)**

장기적으로 데이터 분석을 본격적으로 다루려면, 본인의 컴퓨터에 파이썬 환경을 직접 두는 편이 편리하다. 가장 표준적인 방법이 Anaconda 배포판을 설치하는 것이다.

**Anaconda 설치**

- ❶ Anaconda 사이트(<https://www.anaconda.com>)에 접속해 [Free Download]를 클릭한다.
- ❷ 이메일을 입력하고, 본인의 운영체제(Windows / macOS / Linux)에 맞는 [Distribution Installer]를 선택해 다운로드한다.
- ❸ 다운로드한 설치파일을 실행한다 (수 분 소요).

**가상 환경 만들기**

프로젝트마다 사용하는 패키지 버전이 달라질 수 있다. 이런 경우를 대비해 "가상 환경(virtual environment)"을 따로 만들어 두면 안전하다.

- ❶ Anaconda Navigator 를 실행한 뒤, 좌측의 [Environments] 탭에서 [Create]를 누른다.
- ❷ 이름(예: alpha-stat)과 파이썬 버전을 선택하고 [Create]를 누른다.
- ❸ 새로 만든 환경의 ▶ 버튼 → [Open Terminal]을 누르고, 다음을 차례로 입력한다.

```
pip install ipykernel
```

```
python -m ipykernel install --user --name alpha-stat --display-name "alpha-stat"
```

이렇게 하면 새로 만든 환경이 Jupyter Notebook의 "커널" 목록에 나타난다.

## Jupyter Notebook 실행

Anaconda Navigator의 [Home] 탭에서 "Jupyter Notebook" 항목의 [Launch] 버튼을 누르면, 웹 브라우저가 열리며 작업 가능한 디렉터리가 표시된다. [New → Notebook]을 누르면 새 노트북이 생성된다. 커널 선택 창에서 방금 만든 alpha-stat을 선택한다.

### Jupyter Notebook이란?

코드와 실행 결과, 그래프, 글(마크다운)을 한 문서에 함께 둘 수 있는 대화형 개발 환경이다. .ipynb 라는 확장자를 갖는 파일로 저장되며, Colab에서도 동일한 형식을 사용한다.

## 패키지 설치하기

이 책에서 다루는 코드는 다음 패키지를 사용한다. 이름과 역할을 가볍게 훑어 두자.

- **numpy** — 배열 기반 수치 계산
- **pandas** — 데이터프레임(표)을 다루는 라이브러리
- **matplotlib, seaborn** — 시각화
- **scipy** — 과학·통계 계산 (확률 분포, 가설검정 등)
- **statsmodels** — 회귀, ANOVA 등 통계 모델링
- **scikit-learn** — 분류, 군집화 등 머신러닝

Colab에서는 위 패키지가 모두 미리 설치되어 있어 별도의 설치가 거의 필요 없다. 만약 빠진 패키지가 있다면 코드 셀에서 다음과 같이 입력해 설치한다.

```
!pip install pygam
```

Anaconda 환경에서는 터미널을 열고 다음 중 하나를 입력하면 된다.

```
pip install pygam
# 또는
conda install -c conda-forge pygam
```

### 주의

Colab에서는 코드 셀에서도 느낌표 ! 를 앞에 붙여 셀 명령을 실행할 수 있다.

패키지를 새로 설치한 직후에는 [런타임 → 런타임 다시 시작]을 한 번 눌러주면 안전하다.

패키지를 import 했을 때 `ModuleNotFoundError` 가 나면 거의 대부분 설치가 안 된 것이다.

## 패키지 불러와 쓰기

설치된 패키지는 `import` 명령으로 불러와야 사용할 수 있다. 데이터 분석에서는 다음과 같은 별칭(약칭) 사용이 사실상의 표준이다. 이 책의 모든 코드는 이 별칭을 그대로 따른다.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

앞으로 이 부록과 본문에 나오는 `np.`, `pd.`, `plt.`, `sns.`, `stats.` 등은 모두 위 `import` 문이 먼저 실행됐다고 가정한다.

### ▶ 본문 미리보기 — 본문 1-5부 모두

위의 `import` 문은 사실상 본문 모든 챕터의 첫 코드 셀에 등장한다.

본문 코드를 따라 실행할 때는, 그 챕터의 첫 셀에 있는 `import` 문을 가장 먼저 실행해야 한다.

## A.0.3 첫 코드: 출력, 주석, 변수

### 출력 — `print`

파이썬에서 화면에 무언가를 출력할 때는 `print()` 함수를 사용한다. 함수의 괄호 안에 출력하려는 값이나 문자열을 넣어 준다.

```
print("Hello, world!")
print(3.14)
print(1 + 2)
```

```
Hello, world!
3.14
3
```

문자열은 따옴표로 둘러싼다. 작은따옴표(' ')와 큰따옴표(" ") 모두 동일하게 동작하지만, 시작과 끝의 종류는 같아야 한다.

### 주석 — `#`

코드의 일부를 설명하려면 # 기호 뒤에 글을 적는다. # 뒤의 내용은 실행되지 않는다. 주석은 코드의 의도를 미래의 자신과 동료에게 전달하는 가장 기본적인 도구이다.

```
# 이 줄은 실행되지 않는다.
print("Hello")    # 줄 가운데에 써도 된다.
```

## 변수 — 값에 이름을 붙이기

변수는 값에 붙이는 이름이다. 등호(=)를 써서 이름과 값을 연결한다.

```
x = 3.14
y = "Hello"
print(x)
print(y)
```

```
3.14
Hello
```

변수 이름을 정할 때는 다음 규칙을 따른다.

- 영문자·숫자·밑줄(\_)을 조합해 만든다. 공백은 쓸 수 없다.
- 숫자로 시작할 수 없다. (예: 1st 는 안 됨, x1 은 됨)
- 대소문자를 구분한다. (x 와 X 는 다른 변수)
- 파이썬 예약어(`if`, `else`, `for`, `while`, `in`, `return` 등)는 변수명으로 쓸 수 없다.

### 이름 짓기 관습

관행적으로 소문자와 밑줄을 조합한 `snake_case` 를 쓴다. 예: `total_count`, `mean_score`.

한 글자 이름(`x`, `y`) 보다는 의미가 드러나는 이름이 좋다. 본문에서도 `score`, `weight` 등 의미 있는 이름이 자주 쓰인다.

## A.0.4 기본 자료형

파이썬의 단일 값(스칼라) 자료형은 다음 다섯 가지로 정리된다.

- **숫자형(Numeric)** — 정수(`int`)와 실수(`float`)
- **논리형(Boolean)** — `True`, `False`
- **문자열(str)** — 따옴표로 둘러싼 글자
- **날짜형(datetime)** — 날짜와 시간을 표현
- **None** — "값이 정의되지 않음"을 뜻하는 특수 값

어떤 변수가 어떤 자료형인지 확인하려면 `type()` 함수를 쓴다.

```
print(type(3))
print(type(3.14))
print(type("Hello"))
print(type(True))
print(type(None))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
```

## 숫자형

정수와 실수는 다음과 같은 산술 연산자를 공유한다.

- **+** 더하기, **-** 빼기, **\*** 곱하기, **/** 나누기
- **//** 몫(정수 나눗셈), **%** 나머지, **\*\*** 거듭제곱
- **==** 같다, **!=** 다르다, **<** **<=** **>** **>=** 대소 비교

```
a, b = 7, 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)      # 나눗셈은 항상 실수
print(a // b)     # 몫
print(a % b)      # 나머지
print(a ** b)     # 7의 3제곱
```

```
10
4
21
2.3333333333333335
2
1
343
```

실수에서 자릿수를 정리할 때는 **round()** 함수를 쓴다.

```
print(round(3.1415, 2))
print(round(2.675, 2))    # 부동소수점 저장 방식 때문에 2.67이 나오기도 한다
```

```
3.14
2.67
```

## 논리형

**True** 와 **False** 두 값만 갖는 자료형이다. 비교 연산이나 조건 판단의 결과로 자주 등장한다.

```
print(3 > 2)
print(3 == 4)

# 논리 연산자: and, or, not
print(True and False)
print(True or False)
print(not True)
```

```
True
False
False
True
False
```

논리형은 정수의 부분 집합처럼 동작해서, 연산할 때 **True**는 1, **False**는 0처럼 다뤄진다.

```
print(True + True + False)    # 1 + 1 + 0
print(sum([True, False, True, True])) # 조건이 참인 개수
```

```
2
3
```

## 문자열

문자열은 따옴표로 둘러싸서 만든다. 인덱싱·슬라이싱·연결·반복 등 다양한 조작을 지원한다.

```
s = "abcdef"
print(s[0])      # 첫 글자: a
print(s[0:3])    # 0,1,2 번째: abc
print(s[-1])     # 마지막 글자: f
print(s[-3:])    # 끝에서 세 글자: def

# 연결과 반복
print("Hello, " + "world")
print("=" * 20)
```

```
a
abc
f
def
Hello, world
=====
```

문자열의 길이는 **len()**, 포함 여부는 **in**으로 검사한다.

```
print(len("Hello"))
print("a" in "abc")
print("z" not in "abc")
```

```
5
```

```
True
True
```

숫자 같은 다른 자료형을 문자열로 합치려면 `str()` 로 먼저 변환하거나, f-문자열을 쓴다.

```
x = 3.14
print("x = " + str(x))
print(f"x = {x}, x*2 = {x*2}")
```

```
x = 3.14
x = 3.14, x*2 = 6.28
```

### f-문자열은 분석 보고서의 든든한 친구

본문 4장의 추론 결과 출력에서 `f"{p_value:.4f}"` 같은 표현이 자주 등장한다.  
`:.4f`는 "소수점 아래 4자리"라는 의미다. `:.2%`는 백분율 두 자리이다.

## None

`None`은 "값이 비어 있다", "아직 정해지지 않았다"를 표현하는 특수 값이다. 함수가 결과를 반환하지 않을 때의 기본값이기도 하다.

```
x = None
print(x is None)      # 권장: '== None' 보다 'is None'
print(x is not None)

True
False
```

pandas에서는 결측값을 표현하기 위해 `None` 또는 NumPy의 `NaN`(숫자가 아님)을 쓰는데, 4장 추론 코드에서는 `dropna()` 같은 결측값 처리 함수가 자주 등장한다.

## 날짜와 시간

파이썬에는 `datetime`이라는 내장 모듈이 있어 날짜와 시간을 다룰 수 있다.

```
from datetime import datetime, timedelta

dt = datetime(2026, 4, 26, 13, 0, 0)  # 2026년 4월 26일 13시
print(dt)
print(dt.year, dt.month, dt.day)

# 차이 계산
dt2 = datetime(2026, 5, 1, 9, 0, 0)
delta = dt2 - dt
print(delta)
print(delta.days, "일", delta.seconds // 3600, "시간")
```

```
2026-04-26 13:00:00
2026 4 26
4 days, 20:00:00
4 일 20 시간
```

## A.0.5 자료 구조

앞 절의 자료형들이 "값 한 개"를 담는 그릇이라면, 자료 구조는 "여러 값을 묶는" 그릇이다. 본문 1-5부에서 자주 만나는 네 가지를 다룬다.

### 리스트(list)

리스트는 여러 값을 순서대로 담는, 가장 기본적이며 자주 쓰이는 자료 구조이다. 대괄호 `[ ]`로 만든다.

```
scores = [80, 90, 75, 100, 65]
print(scores)
print(len(scores))      # 길이
print(scores[0])        # 첫 번째 (인덱스는 0 부터)
print(scores[-1])       # 마지막
print(scores[1:4])       # 슬라이싱: 1,2,3 번째

[80, 90, 75, 100, 65]
5
80
65
[90, 75, 100]
```

리스트는 "수정 가능(mutable)" 하다. 값을 추가·삭제·변경할 수 있다.

```
scores.append(88)        # 끝에 추가
print(scores)

scores[0] = 95           # 값 변경
print(scores)

scores.remove(75)        # 값으로 삭제
print(scores)

scores.sort()            # 정렬
print(scores)

[80, 90, 75, 100, 65, 88]
[95, 90, 75, 100, 65, 88]
[95, 90, 100, 65, 88]
[65, 88, 90, 95, 100]
```

### 리스트로 평균·합·최대값

본문 1장에서는 자료의 합·평균을 처음 설명할 때 리스트와 `sum`, `len`, `max`, `min` 같은 내장 함수를 함께 쓴다.

예: `scores = [80,90,75,100,65]; sum(scores) / len(scores)`

### 튜플(tuple)

튜플은 리스트와 비슷하지만 "수정 불가(immutable)" 한 자료 구조다. 소괄호 ( ) 또는 쉼표로 만든다. 함수가 여러 값을 반환할 때 자주 사용된다.

```
pt = (3, 4)
x, y = pt          # 분해 대입
print(x, y)

# 두 값 바꿔치기
a, b = 1, 2
a, b = b, a
print(a, b)
```

```
3 4
2 1
```

### 딕셔너리(dict)

딕셔너리는 키(key)와 값(value)을 짝지어 저장하는 자료 구조이다. 중괄호 { } 와 콜론(:)을 쓴다.

```
person = {"name": "Hong", "age": 30, "city": "Seoul"}
print(person["name"])          # 키로 값 꺼내기
person["age"] = 31             # 값 수정
person["job"] = "Researcher"   # 새 항목 추가
print(person)
print("name" in person)        # 키 존재 여부

Hong
{'name': 'Hong', 'age': 31, 'city': 'Seoul', 'job': 'Researcher'}
True
```

### pandas와의 연결

pandas 의 DataFrame 은 "열 이름 → 값들의 리스트" 형태의 딕셔너리에서 직접 만들 수 있다.

예: `pd.DataFrame({"키": [170,165,180], "몸무게": [60,55,75]})`

### 집합(set)

집합은 중복을 허용하지 않는, 순서가 없는 묶음이다. 중괄호 `{ }` 또는 `set()` 로 만든다. 합집합·교집합·차집합 연산을 직관적으로 할 수 있다.

```
A = {1, 2, 3, 3}
B = {2, 3, 4}
print(A)          # 중복은 자동 제거: {1, 2, 3}
print(A | B)       # 합집합
print(A & B)       # 교집합
print(A - B)       # 차집합
```

```
{1, 2, 3}
{1, 2, 3, 4}
{2, 3}
{1}
```

## A.0.6 제어 흐름과 함수

코드는 위에서 아래로 한 줄씩 실행되지만, 조건에 따라 분기하거나 같은 일을 반복하거나, 자주 쓰는 작업을 함수로 묶을 수 있다.

### 조건문 — if / elif / else

파이썬은 들여쓰기로 "이 코드가 어디에 속해 있는지"를 표현한다. 본문에서는 들여쓰기를 보통 공백 4칸으로 한다.

```
x = 75
if x >= 90:
    grade = "A"
elif x >= 80:
    grade = "B"
elif x >= 70:
    grade = "C"
else:
    grade = "D"
print(grade)
```

C

#### 들여쓰기 규칙

들여쓰기 칸 수는 일관돼야 한다. 같은 블록의 첫 줄과 둘째 줄이 다르게 들여 써져 있으면 `IndentationError` 가 발생한다.

Jupyter/Colab 은 자동으로 4칸 들여쓰기를 맞춰 주므로 큰 걱정은 없다.

### 반복문 — for, while

리스트나 다른 순차형 자료를 차례로 훑어볼 때는 **for** 문을 쓴다. 반복 횟수를 직접 정하려면 **range()** 와 함께 쓴다.

```
scores = [80, 90, 75, 100, 65]
for s in scores:
    print(s)

print("---")

for i in range(3):          # 0, 1, 2
    print("반복", i)
```

```
80
90
75
100
65
---
반복 0
반복 1
반복 2
```

"인덱스와 값을 함께" 보고 싶다면 **enumerate()** 가 깔끔하다.

```
names = ["Alice", "Bob", "Charlie"]
for i, name in enumerate(names):
    print(i, name)
```

```
0 Alice
1 Bob
2 Charlie
```

## 리스트 컴프리헨션

"리스트의 각 원소에 어떤 연산을 적용해 새 리스트를 만들기"는 다음과 같이 한 줄로 쓸 수 있다.

```
scores = [80, 90, 75, 100, 65]
doubled = [s * 2 for s in scores]
high = [s for s in scores if s >= 85]
print(doubled)
print(high)
```

```
[160, 180, 150, 200, 130]
[90, 100]
```

### ▶ 본문 미리보기 — 4장 추론 / 부트스트랩

4장에서 부트스트랩 표본의 평균을 1,000번 계산할 때, `[평균계산 for _ in range(1000)]` 형태의 리스트 컴프리헨션이 자주 등장한다.

## while 문

정해진 횟수가 아니라 "조건이 참인 동안" 반복하려면 **while** 문을 쓴다.

```
n = 1
while n < 100:
    n = n * 2
print(n)
```

128

## 함수 정의 — def

자주 쓰는 작업은 함수로 묶어 두면 같은 코드를 다시 쓸 필요가 없다.

```
def average(xs):
    return sum(xs) / len(xs)

def grade(x, cutoffs=(90, 80, 70)):
    if x >= cutoffs[0]:
        return "A"
    elif x >= cutoffs[1]:
        return "B"
    elif x >= cutoffs[2]:
        return "C"
    else:
        return "D"

print(average([80, 90, 75, 100, 65]))
print(grade(85))
```

82.0

B

함수는 다음 요소로 이뤄진다.

- **def** 키워드 — 함수를 정의한다는 표시
- 함수 이름과 매개변수 목록 — **average(xs)**의 **xs** 가 매개변수
- 함수 본문 — 들여쓴 코드 블록
- **return** — 결과 값을 돌려준다. 생략하면 **None** 이 반환된다.

## 기본값과 키워드 인자

매개변수에는 기본값을 정해 둘 수 있다. 호출할 때는 이름을 지정해 전달할 수도 있다.

```
def greet(name, greeting="안녕"):
    return f"{greeting}, {name}!"
```

```
print(greet("민수"))
print(greet("Alice", greeting="Hello"))
```

```
안녕, 민수!
Hello, Alice!
```

## 람다(lambda) — 한 줄짜리 익명 함수

간단한 함수는 `lambda`로 한 줄에 정의할 수 있다. pandas의 `apply`, sort의 `key` 인자 등에 쓰인다.

```
square = lambda x: x ** 2
print(square(5))

# 정렬 기준으로 사용
words = ["banana", "fig", "apple"]
print(sorted(words, key=lambda w: len(w)))
```

```
25
['fig', 'apple', 'banana']
```

### ▶ 본문 미리보기 — 1장 시각화 / 4장 추론

본문에서는 데이터의 행을 어떤 함수로 변환할 때 `df["col"].apply(lambda x: ...)` 패턴을 자주 사용한다.

예: `df["finish_min"].apply(lambda t: t // 60) # 분 → 시간`

## A.0.7 NumPy — 수치 계산의 기본

`NumPy`("넘파이")는 파이썬에서 수치 계산을 위한 표준 라이브러리이다. 핵심은 `ndarray`라 불리는 다차원 배열이며, 리스트보다 훨씬 빠르고, 통계·선형대수·난수 생성 같은 풍부한 함수를 제공한다.

관례적으로 `import numpy as np`라고 줄여 쓴다.

### 배열 만들기

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
print(a)
print(type(a))
print(a.dtype)          # 원소의 자료형
print(a.shape)          # 모양 (5,) = 1 차원, 길이 5

# 2 차원 배열 (행렬)
M = np.array([[1, 2, 3], [4, 5, 6]])
print(M)
```

```
print(M.shape)
```

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
int64
(5,)
[[1 2 3]
 [4 5 6]]
(2, 3)
```

자주 쓰는 "기성 배열" 생성 함수도 알아두면 편리하다.

```
print(np.zeros(5))          # 0 으로 채워진 길이 5 배열
print(np.ones((2, 3)))      # 1 로 채워진 2x3 배열
print(np.arange(0, 10, 2))  # 0 부터 10 전까지 2 씩
print(np.linspace(0, 1, 5)) # 0~1 구간에서 등간격 점 5 개 생성
```

```
[0. 0. 0. 0. 0.]
[[1. 1. 1.]
 [1. 1. 1.]]
[0 2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
```

## 인덱싱·슬라이싱·불리언 마스크

```
a = np.array([10, 20, 30, 40, 50])
print(a[0])      # 첫 원소
print(a[-1])     # 마지막
print(a[1:4])    # 인덱스 1~3
```

```
# 조건으로 추출 (불리언 마스크)
print(a > 25)
print(a[a > 25])
```

```
10
50
[20 30 40]
[False False True True True]
[30 40 50]
```

## 벡터 연산과 브로드캐스팅

리스트와 가장 큰 차이는, 배열끼리의 연산이 "원소별로" 자동으로 적용된다는 점이다. 같은 일을 리스트로 하려면 for 문이 필요하다.

```
x = np.array([1, 2, 3, 4])
y = np.array([10, 20, 30, 40])

print(x + y)
```

```
print(x * y)
print(x ** 2)
print(np.sqrt(x))      # 원소별 제곱근

# 스칼라와의 연산은 모든 원소에 적용 (broadcasting)
print(x + 100)
print((x - x.mean()) / x.std())  # 표준화
```

```
[11 22 33 44]
[10 40 90 160]
[1  4  9 16]
[1.  1.41421356 1.73205081 2.]
[101 102 103 104]
[-1.34164079 -0.4472136  0.4472136  1.34164079]
```

## 통계 함수

```
a = np.array([3, 1, 4, 1, 5, 9, 2, 6, 5])

print(a.sum())
print(a.mean())
print(a.std())          # 표본표준편차가 아닌 모표준편차(분모 n)
print(a.std(ddof=1))    # 분모 n-1: 표본표준편차
print(a.min(), a.max())
print(np.median(a))
print(np.percentile(a, [25, 50, 75]))
```

```
36
4.0
2.494438257849294
2.6457513110645907
1 9
4.0
[2. 4. 5.]
```

## 난수 — np.random

난수는 시뮬레이션·부트스트랩·재현 가능한 데이터 생성에서 핵심이다. 결과를 매번 동일하게 재현하려면 "시드(seed)"를 고정한다.

```
rng = np.random.default_rng(seed=42)

print(rng.random(3))          # 0~1 균등분포
print(rng.normal(loc=0, scale=1, size=5))  # 표준정규에서 5 개
print(rng.integers(low=1, high=7, size=10)) # 주사위 10 번
print(rng.choice([10, 20, 30], size=4, replace=True))
```

```
[0.77395605 0.43887844 0.85859792]
[0.94056472 -1.95103519 -1.30217951 0.1278404 -0.31624259]
```

```
[4 1 6 3 4 3 2 6 5 4]
[20 30 20 20]
```

### ▶ 본문 미리보기 — 4장 추론 / 부트스트랩 시뮬레이션

본문 4장의 부트스트랩 코드는 `np.random.choice(data, size=len(data), replace=True)` 를 1000회 반복해 평균의 분포를 만든다.

5장에서도 군집 시뮬레이션 데이터를 만들 때 `np.random.normal` 이 자주 등장한다.

## A.0.8 pandas — 데이터프레임 다루기

**pandas**("판다스")는 표 형태의 자료(엑셀의 시트와 같은 모양)를 다루는 표준 라이브러리이다. 1.4.5장에서 다루는 거의 모든 자료가 **pandas**의 **DataFrame**으로 표현된다.

관례적으로 `import pandas as pd`로 줄여 쓴다.

### Series 와 DataFrame

pandas의 두 핵심 자료형은 다음과 같다.

- **Series** — 이름표(인덱스)가 붙은 1차원 배열. 한 "열"을 표현한다.
- **DataFrame** — 열 이름이 붙은 2차원 표. "여러 Series가 모인 것"으로 봐도 좋다.

```
import pandas as pd

s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s)
print(s["b"])

df = pd.DataFrame({
    "이름": ["민수", "수민", "지영", "지수"],
    "키":   [175, 168, 162, 180],
    "몸무게": [68, 55, 50, 78],
})
print(df)
```

```
a    10
b    20
c    30
dtype: int64
20
   이름  키  몸무게
0  민수  175    68
1  수민  168    55
2  지영  162    50
3  지수  180    78
```

## CSV 파일 읽기

실제 자료는 보통 CSV(.csv) 또는 Excel(.xlsx) 파일로 주어진다. pandas 는 한 줄로 이를 DataFrame 으로 불러온다.

```
# 인터넷의 CSV
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
df = pd.read_csv(url)

# 또는 내 컴퓨터의 파일
# df = pd.read_csv("데이터/iris.csv")
# 한글 헤더가 깨지면 인코딩을 지정한다 (utf-8 또는 utf-8-sig)
# df = pd.read_csv("데이터/movie_data.csv", encoding="utf-8")

print(df.head())          # 처음 5 행
print(df.shape)           # (행 수, 열 수)
print(df.columns.tolist()) # 열 이름들
print(df.dtypes)          # 열별 자료형
print(df.describe())      # 수치 열 요약 통계
```

### Colab에서 파일 올리기

왼쪽 폴더 아이콘 → 업로드 버튼으로 직접 파일을 올릴 수 있다.

구글 드라이브와 연결하려면 다음을 한 번 실행하면 된다:

```
from google.colab import drive; drive.mount('/content/drive')
```

## 행과 열 선택하기

```
# 한 열만 (Series)
print(df["sepal_length"].head())

# 여러 열 (DataFrame)
print(df[["sepal_length", "species"]].head())

# 조건으로 행 선택 (불리언 마스크)
setosa = df[df["species"] == "setosa"]
print(setosa.shape)

# 조건 결합: & (and), | (or). 각 조건을 괄호로 묶는다.
big = df[(df["sepal_length"] > 6.0) & (df["species"] == "virginica")]
print(big.shape)
```

### .loc 와 .iloc 의 차이

**.loc** — 라벨(이름)으로 접근. 예: df.loc[3, "species"]

`.iloc` — 위치(0부터 시작하는 정수)로 접근. 예: `df.iloc[3, 4]`

본문에서는 의미가 분명한 `.loc` 를 더 자주 사용한다.

## 새 열 만들기

```
df["sepal_area"] = df["sepal_length"] * df["sepal_width"]
print(df[["sepal_length", "sepal_width", "sepal_area"]].head())

# 조건에 따라 라벨 붙이기
df["크기"] = (df["sepal_length"] > 6).map({True: "큼", False: "작음"})
print(df["크기"].value_counts())
```

## 결측값 처리

pandas 는 결측값을 **NaN**(Not a Number)으로 표현한다. NaN 의 존재 여부를 확인하고, 채우거나 제거할 수 있다.

```
import numpy as np

s = pd.Series([1.0, 2.0, np.nan, 4.0])
print(s.isna())          # NaN 인지
print(s.dropna())        # NaN 행 제거
print(s.fillna(0))        # NaN 을 0 으로 대체
print(s.mean())          # NaN 은 자동으로 무시
```

## 그룹별 요약 — groupby

"성별로", "국가별로", "연령대별로" 평균을 구하는 식의 "분리-적용-결합" 작업이 pandas 의 진가가 드러나는 부분이다.

```
g = df.groupby("species")["sepal_length"].agg(["mean", "std", "count"])
print(g)

# 여러 열을 한꺼번에 요약
g2 = df.groupby("species").agg({
    "sepal_length": "mean",
    "sepal_width": "mean",
    "petal_length": "median",
})
print(g2)
```

|            | mean  | std      | count |
|------------|-------|----------|-------|
| species    |       |          |       |
| setosa     | 5.006 | 0.352490 | 50    |
| versicolor | 5.936 | 0.516171 | 50    |
| virginica  | 6.588 | 0.635880 | 50    |

### ▶ 본문 미리보기 — 1장 시각화 / 4장 추론

1장에서는 배급사·연도별 평균 관객 수를 계산하는 코드에 `groupby()`가 등장한다.  
4장에서는 보스턴 마라톤 자료를 연령·그룹·성별로 묶어 평균 완주 시간을 비교할 때 `groupby(...).agg(...)`가 사용된다.

## 두 표를 합치기 — `merge`, `concat`

열 이름 중 하나를 기준으로 두 `DataFrame`을 "가로로" 이어 붙이려면 `merge`, 같은 모양을 "세로로" 쌓으려면 `concat`을 쓴다.

```
left = pd.DataFrame({"id": [1, 2, 3], "score": [80, 90, 70]})
right = pd.DataFrame({"id": [1, 2, 4], "name": ["민수", "수민", "지영"]})

# 둘 다 가진 id 만 (inner)
print(pd.merge(left, right, on="id", how="inner"))
print("---")
# 왼쪽 기준으로 모두 (left)
print(pd.merge(left, right, on="id", how="left"))
print("---")
# 세로로 쌓기
print(pd.concat([left, left], ignore_index=True))
```

## 교차표(`crosstab`)와 피벗

범주형 자료의 교차표(분할표, 교차분석표)는 `pd.crosstab`으로 만든다.

```
tab = pd.crosstab(df["species"], df["크기"])
print(tab)

# 비율로 보기
print(pd.crosstab(df["species"], df["크기"], normalize="index").round(2))
```

### ▶ 본문 미리보기 — 1장 시각화 / 4장 추론(카이제곱)

1장에서는 두 범주형 변수의 관계를 살피기 위한 분할표 코드가 등장한다.  
4장에서는 같은 분할표를 `chi2_contingency`에 넘겨 카이제곱 검정을 수행한다.

## A.0.9 matplotlib 과 seaborn — 시각화

그래프 그리기에는 두 라이브러리가 자주 함께 쓰인다. `matplotlib`("맷플롯립")은 "기본 그림 도구"이고, `seaborn`("시본")은 그 위에서 통계 그래프를 더 쉽게 그리도록 만들어졌다. `seaborn`으로 그린 그림도 결국 `matplotlib` 객체이므로, 두 라이브러리는 함께 쓸 수 있다.

## 첫 번째 그림

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2 * np.pi, 100)
y = np.sin(x)

plt.figure(figsize=(6, 3))
plt.plot(x, y, color="tab:blue", linewidth=2, label="sin(x)")
plt.title("Sin curve")
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

### 주피터·코랩에서의 그림

Jupyter / Colab에서는 plt.show() 를 빼도 셀 끝에서 자동으로 그림이 표시된다.

한글 폰트가 깨질 때는: plt.rc('font', family='Malgun Gothic'); plt.rc('axes', unicode\_minus=False)

## 자주 쓰는 그림들

### 산점도 — plt.scatter

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
df = pd.read_csv(url)

plt.figure(figsize=(6, 4))
for sp, sub in df.groupby("species"):
    plt.scatter(sub["sepal_length"], sub["sepal_width"], label=sp, alpha=0.7)
plt.xlabel("sepal_length")
plt.ylabel("sepal_width")
plt.legend()
plt.show()
```

### 히스토그램 — plt.hist

```
plt.figure(figsize=(6, 3))
plt.hist(df["sepal_length"], bins=20, edgecolor="white")
plt.xlabel("sepal_length")
plt.ylabel("count")
plt.show()
```

### 상자그림 — seaborn

```
import seaborn as sns

plt.figure(figsize=(6, 3.5))
sns.boxplot(data=df, x="species", y="sepal_length")
plt.show()
```

## 히트맵 — 상관행렬

```
num_cols = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
corr = df[num_cols].corr()

plt.figure(figsize=(4.5, 4))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
plt.show()
```

## 한 그림에 여러 패널 — subplots

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))

axes[0].hist(df["sepal_length"], bins=20)
axes[0].set_title("sepal_length")

sns.boxplot(data=df, x="species", y="petal_length", ax=axes[1])
axes[1].set_title("petal_length by species")

plt.tight_layout()
plt.show()
```

이 패턴(하나의 그림판 위에 여러 칸을 잡고, 각 칸에 그림을 채우기)은 본문 1장의 다양한 시각화에  
서 반복적으로 등장한다.

### ▶ 본문 미리보기 — 1장 시각화 — 거의 전체

1장은 plt와 sns를 가장 적극적으로 쓰는 장이다.

막대그래프(plt.bar / sns.barplot), 산점도(plt.scatter / sns.scatterplot), 히트맵  
(sns.heatmap), 박스플롯(sns.boxplot), 파이차트(plt.pie) 등이 모두 등장한다.

### ▶ 본문 미리보기 — 4장 추론

4장에서는 표본 분포와 부트스트랩 분포를 sns.histplot 또는 plt.hist로 그리고, plt.axvline  
로 신뢰구간 경계선을 함께 그린다.

## A.0.10 통계 모델링 라이브러리 맛보기

본문 2부부터는 회귀, 가설검정, 분류, 군집화 같은 통계·머신러닝 코드가 등장한다. 여기서는 본문에 가장 자주 등장하는 세 라이브러리를 한 짝씩 "맛보기"로 소개한다. 각 함수의 자세한 의미는 본문 해당 장에서 다룬다.

## scipy.stats — 확률 분포와 가설검정

4장 추론에서 가장 핵심이 되는 라이브러리이다. 다음 두 가지 일을 한다.

- **확률 분포의 함수값 계산:** 정규분포·t 분포·이항분포 등의 pdf, cdf, ppf, rvs
- **가설검정:** t-검정, 카이제곱 검정, ANOVA 등

### 정규분포 다루기

```
from scipy import stats

# 표준정규에서 확률밀도, 누적확률, 분위수
print(stats.norm.pdf(0))           # f(0)
print(stats.norm.cdf(1.96))        # P(Z <= 1.96)
print(stats.norm.ppf(0.975))       # 97.5% 분위수 ≈ 1.96

# 정규분포에서 표본 100 개 뽑기
np.random.seed(0)
sample = stats.norm.rvs(loc=170, scale=10, size=100)
print(sample[:5].round(2))

0.3989422804014327
0.9750021048517795
1.959963984540054
[187.64 174.    179.79 192.41 188.68]
```

### 일표본 t-검정

```
# 평균이 170 인지 검정
t, p = stats.ttest_1samp(sample, popmean=170)
print(f"t = {t:.3f}, p-value = {p:.4f}")

t = 0.590, p-value = 0.5562
```

### 이표본 t-검정

```
x = stats.norm.rvs(loc=10, scale=2, size=30, random_state=1)
y = stats.norm.rvs(loc=12, scale=2, size=30, random_state=2)
t, p = stats.ttest_ind(x, y, equal_var=False)
print(f"t = {t:.3f}, p-value = {p:.4f}")

t = -2.832, p-value = 0.0064
```

## 카이제곱 독립성 검정

```
# 분할표 (예: 2x3)
table = np.array([[30, 20, 10],
                  [25, 35, 20]])
chi2, p, dof, expected = stats.chi2_contingency(table)
print(f"chi2 = {chi2:.3f}, p = {p:.4f}, 자유도 = {dof}")
```

### ▶ 본문 미리보기 — 4장 추론 — 거의 전 절

위의 `stats.ttest_1samp` / `ttest_ind` / `chi2_contingency`가 4장의 가설검정 코드 대부분의 핵심을 이룬다.

신뢰구간을 부트스트랩으로 만들 때는 `np.percentile(boot_means, [2.5, 97.5])` 같은 방식이 자주 쓰인다.

## statsmodels — 회귀와 ANOVA

`statsmodels`("스태츠모델즈")는 R 사용자에게 친숙한 "식(formula)" 표기로 회귀 모형을 적합하는 라이브러리이다. 보고서 형식의 결과 요약を 깔끔하게 출력한다.

### 선형 회귀

```
import statsmodels.formula.api as smf

# 가짜 데이터
rng = np.random.default_rng(0)
n = 100
x = rng.normal(size=n)
y = 2 + 3 * x + rng.normal(scale=1.0, size=n)
df = pd.DataFrame({"x": x, "y": y})

# y = b0 + b1*x
model = smf.ols("y ~ x", data=df).fit()
print(model.summary().tables[1])
print("R^2 =", model.rsquared.round(3))
```

두 개 이상의 설명변수를 쓸 때는 `+`로 잇고, 범주형 변수는 `C(...)`로 감싸 더미 변수를 자동 생성하게 한다.

```
# 다중회귀 + 범주형
df["g"] = np.where(df["x"] > 0, "A", "B")
m = smf.ols("y ~ x + C(g)", data=df).fit()
print(m.params)
```

### 로지스틱 회귀

```
# 0/1 반응변수
df["y_bin"] = (df["y"] > df["y"].median()).astype(int)
log_m = smf.logit("y_bin ~ x", data=df).fit(dis=False)
print(log_m.summary().tables[1])
```

## 일원분산분석(ANOVA)

```
from statsmodels.stats.anova import anova_lm

iris = pd.read_csv(
    "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv")
m = smf.ols("sepal_length ~ C(species)", data=iris).fit()
print(anova_lm(m, typ=2))
```

### ▶ 본문 미리보기 — 2부 모델링1 / 4장 추론

2부에서는 야구 투수의 연봉 자료에 `smf.ols("log_salary ~ G + ERA + C(closer)", data=df).fit()` 같은 형태로 회귀를 적합한다.

4장에서는 보스턴 마라톤 자료에 `ols + anova_lm`으로 분산분석을 수행한다.

## scikit-learn — 분류와 군집화

`scikit-learn`("사이킷런", import 시에는 `sklearn`)의 대부분의 모형은 `.fit()`으로 학습하고 `.predict()`로 예측하는 공통 인터페이스를 사용한다.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

iris = pd.read_csv(
    "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv")
X = iris.drop(columns=["species"])
y = iris["species"]

# 1) 학습용·검증용 나누기
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

# 2) 표준화
scaler = StandardScaler()
X_tr_s = scaler.fit_transform(X_tr)
X_te_s = scaler.transform(X_te)

# 3) 모형 적합 (로지스틱 회귀 분류기)
clf = LogisticRegression(max_iter=1000)
```

```
clf.fit(X_tr_s, y_tr)
```

# 4) 예측·평가

```
y_hat = clf.predict(X_te_s)
print("정확도:", accuracy_score(y_te, y_hat))
print(confusion_matrix(y_te, y_hat))
```

```
정확도: 0.9111111111111111
[[15  0  0]
 [ 0 14  1]
 [ 0  3 12]]
```

### scikit-learn의 "4단계 패턴"

- 1) 데이터 분할 (train\_test\_split)
- 2) 전처리 객체에 .fit\_transform → 학습 자료에는 fit\_transform, 검증 자료에는 transform
- 3) 모형 객체 생성 후 .fit(X, y)
- 4) .predict(X) 로 예측, 평가 함수로 비교

본문 3부의 어떤 분류기(KNN, 나무모형, RandomForest 등)도 이 패턴을 따른다.

## 기타 자주 쓰는 모형

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB

# 모두 같은 인터페이스
for name, model in [
    ("KNN", KNeighborsClassifier(n_neighbors=5)),
    ("Tree", DecisionTreeClassifier(random_state=0)),
    ("RF", RandomForestClassifier(n_estimators=100, random_state=0)),
    ("NB", GaussianNB()),
]:
    model.fit(X_tr_s, y_tr)
    print(f"{name:>5}: {accuracy_score(y_te, model.predict(X_te_s)):.3f}")
```

### ▶ 본문 미리보기 — 3부 모델링2

3부에서는 위 패턴으로 KNN, 의사결정나무, 랜덤 포레스트, XGBoost 등을 비교한다.

데이터셋은 hn19.csv (국민건강영양조사 일부)로, 당뇨 여부를 분류한다.

### ▶ 본문 미리보기 — 5장 패턴 탐색

5장의 군집 분석은 sklearn.cluster 의 KMeans, AgglomerativeClustering 과

`scipy.cluster.hierarchy`의 `linkage / dendrogram`을 사용한다.  
주성분분석은 `sklearn.decomposition`의 `PCA`가 표준이다.

## A.0.11 자주 만나는 오류, 그리고 AI에게 물어보기

프로그래밍을 처음 시작하는 사람의 가장 큰 어려움은 "오류 메시지" 그 자체다. 다행히 파이썬의 오류 메시지는 비교적 친절한 편이고, 자주 만나는 종류도 정해져 있다. 이 절에서는 본문의 코드를 따라 하다가 마주치기 쉬운 오류를 정리한다.

### 자주 만나는 오류 5 가지

#### ModuleNotFoundError — 패키지가 없다

```
ModuleNotFoundError: No module named 'pygam'
```

원인 — 해당 패키지가 설치되어 있지 않다.

해결 — 코드 셀에서 `!pip install pygam`을 한 번 실행한다. 설치 후 한 번은 [런타임 다시 시작]을 누르는 것이 안전하다.

#### NameError — 정의되지 않은 이름

```
NameError: name 'df' is not defined
```

원인 — 변수를 만든 셀을 아직 실행하지 않았거나, 노트북을 새로 열어 이전에 정의했던 변수가 사라진 경우가 가장 많다.

해결 — [런타임 → 모두 다시 실행]으로 위 셀부터 차례로 다시 실행한다.

#### TypeError — 자료형이 맞지 않는다

```
TypeError: can only concatenate str (not "int") to str
```

원인 — 문자열과 숫자를 + 로 이으려 했다.

해결 — `"x = " + str(x)` 와 같이 변환하거나, `f"x = {x}"`처럼 f-문자열을 쓴다.

#### KeyError — 그런 키가 없다

```
KeyError: '몸무게'
```

원인 — DataFrame의 열 이름을 잘못 입력했다 (오타·공백·전각/반각 차이가 흔한 원인이다).

해결 — `df.columns.tolist()` 로 실제 열 이름을 출력해 비교한다. 공백이 섞여 있다면 `df.columns`

`= df.columns.str.strip()` 로 정리한다.

## IndentationError — 들여쓰기가 어긋났다

```
IndentationError: unexpected indent
```

원인 — 같은 블록 안에서 들여쓰기 칸 수가 다르다.

해결 — 들여쓰기를 모두 공백 4칸으로 일치시킨다. Jupyter / Colab은 보통 자동으로 맞춰 주지만, 외부에서 코드를 붙여 넣은 경우 종종 어긋날 수 있다.

## 오류 메시지 읽는 법

파이썬 오류 메시지는 "역순"으로 읽는 것이 효과적이다. 가장 아래에 있는 두 줄을 먼저 본다.

```
...
File "/tmp/example.py", line 4, in <module>
    print(df["몸무게"])
KeyError: '몸무게'
```

- **마지막 줄** — 오류의 종류와 직접적 원인 (`KeyError: '몸무게'`)
- **그 위 줄** — 오류가 난 코드의 위치 (4 번째 줄, `print(df["몸무게"])`)

이 둘만 명확히 읽어도 대부분의 문제는 빠르게 잡힌다.

## AI 에게 물어보기

ChatGPT, Claude, Gemini 같은 대화형 AI는 코드 에디터의 "옆자리에 앉은 동료"처럼 활용할 수 있다. 단, AI를 효과적으로 쓰려면 다음을 기억하면 좋다.

### 좋은 질문 = 충분한 맥락

초보자가 가장 흔히 저지르는 실수는 "왜 안 되죠?"만 묻는 것이다. 다음 세 가지를 함께 주면 답의 정확도가 크게 올라간다.

- **내가 무엇을 하려고 했는지** (목표)
- **내가 실행한 코드** (가능하면 통째로)
- **받은 오류 메시지 또는 출력 결과** (전체 그대로)

다음은 잘 쓴 질문의 예시이다.

### 질문 예시

[목표] iris 데이터에서 species 별 평균 sepal\_length를 그래프로 그리려고 합니다.

[코드] 다음을 실행했습니다.

```
import pandas as pd
df = pd.read_csv("iris.csv")
df.groupby("species")["sepal_length"].mean().plot.bar()
```

[오류] FileNotFoundError: [Errno 2] No such file or directory: 'iris.csv'

왜 이 오류가 나며, Colab에서는 어떻게 해결하면 좋을까요?

## AI 를 활용하는 다섯 가지 패턴

- **코드 설명 부탁** — "이 코드가 무엇을 하는지 한 줄씩 설명해 줘."
- **오류 진단** — "다음 오류 메시지의 원인과 해결법을 알려 줘. (오류 첨부)"
- **코드 작성** — "DataFrame df 에서 income 이 상위 10%인 행만 골라 평균 BMI 를 구하는 코드를 써 줘."
- **개선 제안** — "같은 결과를 더 짧고 읽기 쉽게 다시 써 줄 수 있어?"
- **개념 설명** — "카이제곱 검정과 t-검정은 언제 어떻게 다르게 쓰는 거야? 짧게 설명해 줘."

## AI 의 답을 검증하는 습관

AI는 종종 그럴듯하게 들리지만 실제로는 틀린 답을 줄 수 있다. 다음 습관을 들이면 좋다.

- 받은 코드는 반드시 직접 실행해 본다 — 출력이 기대와 같은지 확인한다.
- **라이브러리 함수 이름이 의심되면 공식 문서를 한 번 더 본다** (예: pandas, scikit-learn 공식 문서).
- 결과의 단위·부호·자릿수가 상식적인지 의심해 본다. 회귀계수가 너무 크거나, P-값이 음수라면 어딘가 잘못된 것이다.

AI는 빠른 길잡이일 뿐, 최종 판단의 책임은 결국 사용자에게 있다는 점을 기억하면 충분하다.

## 더 공부하고 싶다면

- **공식 문서** — 가장 정확한 정보의 출처. 본문에서 사용된 라이브러리는 모두 잘 정리된 공식 문서를 가지고 있다.
  - NumPy: <https://numpy.org/doc/>
  - pandas: <https://pandas.pydata.org/docs/>
  - matplotlib: <https://matplotlib.org/stable/>
  - seaborn: <https://seaborn.pydata.org/>
  - scipy: <https://docs.scipy.org/doc/scipy/>
  - statsmodels: <https://www.statsmodels.org/>

- scikit-learn: <https://scikit-learn.org/stable/>
- **커뮤니티** — Stack Overflow, Real Python, Kaggle Notebooks. 같은 오류로 고민하는 사람은 늘 있다.
- **동영상 강의** — "통통한 데이터 분석 공개강좌" 영상이 본 책의 출발점이며, 본문의 코드와 직접 짝을 이룬다.

이 부록의 분량으로 다루지 못한 주제는 매우 많지만, 본문 1-5부의 코드를 따라가는 데 필요한 디딤돌은 모두 있어 두었다. 새 명령어를 만나면 두려워하지 말고 위에 적은 "AI에게 묻고, 공식 문서로 확인하고, 직접 실행해 보는" 세 단계를 반복하면 된다. 그것만 익숙해지면, 이 책 이후의 어떤 데이터 분석도 두렵지 않을 것이다.

## A.1 1 부 실습

이 장에서는 본문 1부에서 다룬 자료 정리와 시각화의 개념을 영화 데이터, 서울 아파트 거래 데이터, 서울시 문화행사 데이터를 이용해 파이썬으로 직접 진행해 본다.

### A.1.1 환경 설정과 데이터 로딩

본격적인 분석에 앞서 시각화에 필요한 라이브러리(모듈)를 불러온다. 한글이 포함된 그래프를 그릴 때는 운영체제에 설치된 한글 폰트도 함께 설정하는 것이 좋다.

#### [파이썬 코드 1-1]

```
# 라이브러리 로딩
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib as mpl
import seaborn as sns
import matplotlib.font_manager as fm

# 마이너스 기호가 깨지지 않도록 설정
mpl.rcParams["axes.unicode_minus"] = False

# 운영체제별 한글 폰트 설정
available = {f.name for f in fm.fontManager.ttflist}
for font in ["Malgun Gothic", "Apple SD Gothic Neo", "NanumGothic"]:
    if font in available:
        plt.rcParams["font.family"] = font
        break
```

`pandas`의 `read_csv()`를 이용하여 분석에 사용할 세 가지 데이터를 불러온다. 데이터를 불러올 때는 파일의 경로가 정확해야 함에 주의한다.

#### [파이썬 코드 1-2]

```
# 데이터 로딩
영화데이터 = pd.read_csv("데이터/movie_data.csv")
부동산데이터 = pd.read_csv("데이터/seoul_apartment_transactions.csv")
서울시문화행사정보 = pd.read_csv("데이터/seoul_cultural_events.csv")

# 데이터 앞부분 확인
영화데이터.head()
```

영화 데이터에는 영화명, 개봉년도, 관객수, 매출액, 스크린수, 대표국적, 배급사 등의 변수가 포함되

어 있다. 부동산 데이터에는 자치구명, 건물용도, 물건금액(만원), 건물면적(m<sup>2</sup>), 건축년도 등이, 문화 행사 데이터에는 공연/행사명, 자치구, 분류 등이 포함되어 있다. 부동산 데이터에서는 분석에 적합하도록 '아파트' 거래만 추리고 일부 변수의 이름을 줄여 사용한다.

### [파이썬 코드 1-3]

```
# 아파트 거래 자료만 추출하고 변수명 정리
아파트데이터 = (
    부동산데이터
    .loc[lambda df: (df["건물용도"] == "아파트") & (df["건축년도"].notna())]
    .rename(columns={"물건금액(만원)": "가격", "건물면적(m2)": "건물면적"})
)
```

## A.1.2 자료 변환: Wide ↔ Long

본문 2장에서 다룬 wide-long 변환을 직접 수행해 본다. 먼저 치료제 A, B를 받은 결과가 wide 형식으로 정리된 작은 자료를 만들어 보자.

### [파이썬 코드 1-4]

```
# 치료제 데이터 (wide 형식)
trt_wide = pd.DataFrame({
    "id": [1, 2, 3],
    "a": [23, 17, 9],
    "b": [16, None, 11],
})
trt_wide
```

`melt()`를 이용하면 a, b 두 열을 하나의 'treatment' 열로 모아 long 형식으로 바꿀 수 있다.

### [파이썬 코드 1-5]

```
# wide → long
trt_long = trt_wide.melt(
    id_vars="id",           # 그대로 유지할 열
    value_vars=["a", "b"],  # 하나로 모을 열
    var_name="treatment",   # 열 이름이 담길 새 변수
    value_name="response",  # 값이 담길 새 변수
)
trt_long
```

반대로 long → wide 변환은 `pivot_table()`을 사용한다.

### [파이썬 코드 1-6]

```
# long → wide
```

```
trt_wide_recovered = (
    trt_long
    .pivot_table(index="id", columns="treatment", values="response")
    .reset_index()
)
trt_wide_recovered
```

본문에서 살펴본 결핵 데이터(country-year-cases-population)에도 같은 변환을 적용해 본다.

### [파이썬 코드 1-7]

```
# 결핵 데이터 (wide 형식)
tb_wide = pd.DataFrame({
    "country": ["Afghanistan", "Afghanistan", "Brazil", "Brazil",
               "China", "China"],
    "year": [1999, 2000, 1999, 2000, 1999, 2000],
    "cases": [745, 2666, 37737, 80488, 212258, 213766],
    "population": [19987071, 20595360, 172006362, 174504898,
                  1272915272, 1280428583],
})

# wide → long
tb_long = tb_wide.melt(
    id_vars=["country", "year"],
    value_vars=["cases", "population"],
    var_name="type",
    value_name="count",
)

# long → wide (다시 원래 형태로)
tb_wide_recovered = (
    tb_long
    .pivot_table(index=["country", "year"], columns="type", values="count")
    .reset_index()
)
```

cases와 population이 한 열에 섞여 있는 long 형식은 두 변수의 관계를 다루기 어렵지만, 형태를 자유롭게 오갈 수 있으면 자료를 분석 목적에 맞게 재구성할 수 있다.

## A.1.3 데이터 변환의 5 가지 문법: 배급사 RTP 분석

본문에서 다룬 "최근 인기가 크게 떨어진 배급사는 어디인가?"라는 질문을, 다섯 가지 문법(mutate → select → filter → group\_by + summarize → arrange)을 차례로 적용해 답해 본다.

### 1) 새로운 변수 만들기(Mutate): 대표배급사 만들기

배급사 정보에는 한 영화에 여러 배급사가 함께 적혀 있는 경우가 있다. 첫 번째로 표기된 배급사를

대표배급사로 정의한다.

#### [파이썬 코드 1-8]

```
df = 영화데이터.copy()
df["대표배급사"] = (
    df["배급사"]
    .astype("string")
    .str.split(",")
    .str[0]
    .str.strip()
)
```

## 2) 열 선택하기(Select): 필요한 변수만 남기기

이후 분석에서는 대표배급사, 개봉년도, 관객수, 영화명만 사용한다.

#### [파이썬 코드 1-9]

```
base = df.loc[:, ["대표배급사", "개봉년도", "관객수", "영화명"]]
```

## 3) 행 추출하기(Filter): 결측값 제거하기

주요 변수에 결측값이 있는 행은 제거한다.

#### [파이썬 코드 1-10]

```
base = base.dropna(subset=["대표배급사", "개봉년도", "관객수"])
```

## 4) 요약하기(Summarize): 배급사-연도별 평균 관객 수 계산하기

대표배급사와 개봉년도를 기준으로 그룹화한 뒤, 그룹별 평균 관객수와 영화 수를 구한다.

#### [파이썬 코드 1-11]

```
stats = (
    base
    .groupby(["대표배급사", "개봉년도"], as_index=False)
    .agg(
        평균관객수=("관객수", "mean"),
        영화수=("영화명", "count"),
    )
)
```

## 5) 새로운 변수 만들기(Mutate): RTP 계산하기

각 배급사의 전성기 평균 관객수(peak)를 구하고, 연도별 평균 관객수가 그 전성기에 비해 몇 퍼센트

인지를 나타내는 RTP(Ratio to Peak)를 계산한다.

#### [파이썬 코드 1-12]

```
peak = (
    stats
    .groupby("대표배급사", as_index=False)
    .agg(peak_평균관객수=("평균관객수", "max"))
)

stats = stats.merge(peak, on="대표배급사", how="left")
stats["RTP"] = stats["평균관객수"] / stats["peak_평균관객수"]
```

### 6) 행 추출하기(Filter) + 정렬하기(Arrange): 최근 연도 기준 RTP가 낮은 배급사

가장 최근의 개봉년도 자료만 남기고, RTP가 낮은 순으로 정렬해 상위 일부만 살펴본다.

#### [파이썬 코드 1-13]

```
recent_year = int(base["개봉년도"].max())

recent = (
    stats
    .query("개봉년도 == @recent_year")
    .sort_values("RTP")
    .loc[:, ["대표배급사", "개봉년도", "영화수",
             "평균관객수", "peak_평균관객수", "RTP"]]
    .head(10)
)
recent
```

이제 RTP가 가장 낮은 7개 배급사를 막대그래프로 시각화한다.

#### [파이썬 코드 1-14]

```
recent_top7 = recent.head(7)

plt.figure(figsize=(8, 4))
sns.barplot(data=recent_top7, x="대표배급사", y="RTP", color="skyblue")
plt.xticks(rotation=45, ha="right")
plt.xlabel("대표배급사")
plt.ylabel("RTP")
plt.title("RTP가 가장 낮은 7개 배급사")
plt.tight_layout()
plt.show()
```

이 결과는 본문 [그림 1-15]에 해당한다. 다섯 가지 문법이 분리된 단계가 아니라 자연스럽게 이어지는 하나의 분석 흐름임을 확인할 수 있다.

## A.1.4 자료 결합

자치구 단위로 요약된 문화행사 자료와 아파트 거래 자료를 결합해, 두 자료의 자치구 목록이 어떻게 겹치고 어디서 갈라지는지 살펴본다.

### [파이썬 코드 1-15]

```
# 자치구별 문화행사 수 요약
문화_요약 = (
    서울시문화행사정보
    .groupby("자치구", dropna=False)
    .size()
    .reset_index(name="행사수")
)

# 자치구별 아파트 거래 요약
부동산_요약 = (
    아파트데이터
    .query("건축년도 != 0")
    .groupby("자치구명", dropna=False)
    .agg(
        거래건수=("가격", "count"),
        물건금액_평균=("가격", "mean"),
        물건금액_표준편차=("가격", "std"),
        건물면적_평균=("건물면적", "mean"),
        건물면적_표준편차=("건물면적", "std"),
    )
    .reset_index()
)
```

세 가지 결합 방식을 비교해 본다.

### [파이썬 코드 1-16]

```
# inner join: 두 표 모두에 존재하는 자치구만
문화_부동산_inner = pd.merge(
    문화_요약, 부동산_요약,
    how="inner", left_on="자치구", right_on="자치구명",
)
print(f"inner join 행 수: {len(문화_부동산_inner)}")

# left join: 문화행사 표 기준, 아파트 정보가 없으면 결측값
문화_부동산_left = pd.merge(
    문화_요약, 부동산_요약,
    how="left", left_on="자치구", right_on="자치구명",
)
```

```
print(f"left join 행 수: {len(문화_부동산_left)}")

# full join: 양쪽 표의 모든 자치구 포함
문화_부동산_full = pd.merge(
    문화_요약, 부동산_요약,
    how="outer", left_on="자치구", right_on="자치구명",
)
print(f"full join 행 수: {len(문화_부동산_full)}")
```

같은 두 표를 합치더라도 어떤 join을 선택하느냐에 따라 행 수와 결측값의 분포가 달라진다. 어떤 방식을 쓸지는 "빠진 데이터를 어떻게 다룰 것인가"에 대한 데이터 분석가의 판단에 달려 있다.

## A.1.5 범주형 자료의 분포

영화 데이터의 **대표국적** 변수를 이용해 범주형 변수의 분포를 시각화한다. 먼저 `value_counts()`로 도수분포표를 만든다.

### [파이썬 코드 1-17]

```
대표국적_집계 = (
    영화데이터["대표국적"]
    .value_counts(dropna=True)
    .rename("영화수")
    .reset_index()
    .rename(columns={"index": "대표국적"})
)
print(대표국적_집계.head(10))
```

### 가로 막대그래프

국적 이름이 길 수 있으므로 가로 막대그래프로 그린다. y축에 국적, x축에 영화 수를 두면 이름이 잘리지 않고 자연스럽게 읽힌다.

### [파이썬 코드 1-18]

```
plt.figure(figsize=(8, 5))
sns.barplot(data=대표국적_집계, y="대표국적", x="영화수",
            color="mediumturquoise")
plt.xlabel("영화 수")
plt.ylabel(None)
plt.title("대표국적별 영화 수")
plt.tight_layout()
plt.show()
```

## 파이차트

상위 8개 국적 내 구성 비율을 파이차트로 나타낸다. 범주가 많아지면 조각이 잘게 나뉘어 읽기 어려워지므로 일부만 추리는 것이 좋다.

#### [파이썬 코드 1-19]

```
top8 = (
    영화데이터["대표국적"]
    .value_counts()
    .rename("영화수")
    .reset_index()
    .rename(columns={"index": "대표국적"})
    .head(8)
)
top8["비율"] = top8["영화수"] / top8["영화수"].sum()

plt.figure(figsize=(8, 6))
plt.pie(top8["비율"], labels=None,
        autopct="%1.1f%%", startangle=90, counterclock=True,
        colors=plt.cm.Dark2.colors)
plt.legend(top8["대표국적"], title="대표국적",
           loc="center left", bbox_to_anchor=(1, 0.5))
plt.title("상위 8개 대표국적 내 구성 비율", fontsize=14)
plt.tight_layout()
plt.show()
```

## A.1.6 연속형 자료의 분포

아파트 거래 데이터의 **가격** 변수를 이용해 연속형 변수의 분포를 시각화한다. 가격은 값의 범위가 매우 넓어 한쪽에 몰려 보이므로, 로그를 취해 분포를 보기 좋게 만든다.

### 히스토그램

#### [파이썬 코드 1-20]

```
price = 아파트데이터["가격"].astype(float)
x = np.log(price[price > 0].dropna())

plt.figure(figsize=(8, 5))
sns.histplot(x=x, bins=60, color="lightblue", edgecolor="white")
plt.xlabel("log(가격)")
plt.ylabel("건수")
plt.title("아파트 가격 (log) 분포")
plt.tight_layout()
plt.show()
```

## 커널 밀도 추정(KDE)과 대역폭 비교

대역폭(bandwidth)이 달라질 때 KDE 곡선이 어떻게 변하는지 비교하기 위해, 0.05, 0.1, 0.2 세 값으로 동시에 KDE를 계산해 하나의 그림에 그린다.

#### [파이썬 코드 1-21]

```
from scipy.stats import gaussian_kde

# KDE 를 그릴 x 축 그리드 생성
x_grid = np.linspace(x.min() - 0.2, x.max() + 0.2, 1000)

# 대역폭별 KDE 계산
bw_list = [0.05, 0.1, 0.2]
density_list = []
for bw in bw_list:
    kde = gaussian_kde(x, bw_method=bw / np.std(x, ddof=1))
    y = kde(x_grid)
    density_list.append(pd.DataFrame({
        "x": x_grid, "y": y,
        "bandwidth": f"bandwidth = {bw}",
    }))
density_df = pd.concat(density_list, ignore_index=True)
```

이제 히스토그램과 세 가지 KDE 곡선을 함께 그린다. 히스토그램의 y축을 밀도(density) 기준으로 맞춰야 KDE 곡선과 같은 축에서 비교할 수 있다.

#### [파이썬 코드 1-22]

```
plt.figure(figsize=(10, 6))
sns.histplot(x=x, stat="density", bins=60,
             color="lightblue", edgecolor="white", alpha=0.8)
sns.lineplot(
    data=density_df, x="x", y="y", hue="bandwidth",
    palette={"bandwidth = 0.05": "red",
             "bandwidth = 0.1": "blue",
             "bandwidth = 0.2": "green"},
    linewidth=1.5,
)
plt.xlim(x.min() - 0.2, x.max() + 0.2)
plt.title("아파트 가격 (log) 분포: 히스토그램과 커널 밀도 추정")
plt.xlabel("log(가격)")
plt.ylabel("밀도")
sns.despine()
plt.tight_layout()
plt.show()
```

대역폭이 작을수록 곡선이 데이터의 미세한 변동에 민감하게 반응하고, 클수록 부드럽게 다듬어지는 것을 확인할 수 있다.

## A.1.7 두 범주형 변수의 관계

영화 데이터에서 **대표국적**과 **5년내개봉작** 두 변수의 관계를 살펴본다.

### 분할표

`pd.crosstab()`으로 두 변수의 빈도 분할표를 만든다. 행과 열의 합계도 함께 표시하면 전체 구조를 파악하기 쉽다.

#### [파이썬 코드 1-23]

```
분할표 = pd.crosstab(영화데이터["대표국적"], 영화데이터["5년내개봉작"])

# 한계합 추가
분할표_한계포함 = 분할표.copy()
분할표_한계포함.loc["합계"] = 분할표_한계포함.sum()
분할표_한계포함["합계"] = 분할표_한계포함.sum(axis=1)
분할표_한계포함
```

### 모자이크 플롯

각 범주 조합의 빈도를 직사각형의 면적으로 표현한다. 범주가 많으면 보기 어려워지므로 상위 4개국적만 골라 그린다.

#### [파이썬 코드 1-24]

```
from statsmodels.graphics.mosaicplot import mosaic
from matplotlib.patches import Patch

# 상위 4개 국적만 선택
영화데이터["개봉 5년 이내"] = 영화데이터["5년내개봉작"]
df_mosaic = 영화데이터[["대표국적", "개봉 5년 이내"].dropna()]
top4 = df_mosaic["대표국적"].value_counts().nlargest(4).index
df_mosaic = df_mosaic[df_mosaic["대표국적"].isin(top4)]
df_mosaic["대표국적"] = df_mosaic["대표국적"].astype(str)
df_mosaic["개봉 5년 이내"] = df_mosaic["개봉 5년 이내"].astype(str)

# 색 지정
color_dict = {"Y": "#46C7C7", "N": "#F1948A"}
def fill_color(key):
    return {"color": color_dict.get(key[1], "white")}

plt.figure(figsize=(10, 6))
mosaic(df_mosaic, ["대표국적", "개봉 5년 이내"],
        properties=fill_color, labelizer=lambda key: "", gap=0.02)
plt.title("상위 4개 대표국적과 5년내개봉작 여부", fontsize=14)
```

```
plt.xticks(rotation=45, ha="right")
plt.xlabel("대표국적")
plt.ylabel("개봉 5 년이내", labelpad=40)
legend_patches = [Patch(color=color_dict["N"], label="N"),
                  Patch(color=color_dict["Y"], label="Y")]
plt.legend(handles=legend_patches, title="5 년내개봉작", loc="best")
plt.tight_layout()
plt.show()
```

## 히트맵

같은 분할표를 색의 농도로 표현한 것이 히트맵이다. 빈도가 높을수록 셀의 색이 진해진다.

### [파이썬 코드 1-25]

```
plt.figure(figsize=(6, 4))
sns.heatmap(
    분할표, cmap="Greys", linewidths=0.5, linecolor="white",
    annot=True, fmt="d", cbar_kws={"label": "빈도수"},
)
plt.title("대표국적 x 5 년내개봉작 교차 빈도 (히트맵)")
plt.xlabel("5 년내개봉작")
plt.ylabel("대표국적")
plt.tight_layout()
plt.show()
```

## A.1.8 두 연속형 변수의 관계

영화 데이터에서 **스크린수**와 **매출액**의 관계를 산점도로 살펴본다.

## 기본 산점도

### [파이썬 코드 1-26]

```
plt.figure(figsize=(7, 5))
sns.scatterplot(data=영화데이터, x="스크린수", y="매출액", alpha=0.6)
plt.title("스크린수와 매출액의 관계")
plt.xlabel("스크린수")
plt.ylabel("매출액")
plt.tight_layout()
plt.show()
```

대부분의 점이 왼쪽 아래에 뭉쳐 있고 일부 점만 위로 크게 튀어나와 있어, 두 변수의 관계를 직선으로 설명하기 어렵다는 것을 확인할 수 있다.

## 산점도 + 주변 분포 + 추세선

`sns.jointplot()`을 사용하면 산점도, 직선 추세선, 두 변수의 한계분포(주변 히스토그램)를 한번에 그릴 수 있다.

#### [파이썬 코드 1-27]

```
data = 영화데이터[["스크린수", "매출액"]].dropna()

g = sns.jointplot(
    data=data, x="스크린수", y="매출액", kind="reg", height=6,
    scatter_kws={"alpha": 0.4, "s": 20},
    line_kws={"color": "red"},
    marginal_kws={"bins": 30, "alpha": 0.6},
)
g.set_axis_labels("스크린수", "매출액")
g.fig.suptitle("스크린수와 매출액: 산점도와 주변 분포, 직선 추세선", y=1.02)
plt.show()
```

## A.1.9 연속형 자료와 범주형 자료의 관계

영화 데이터의 **대표국적**(범주형)과 **스크린수**(연속형)의 관계를 상자그림으로 본다. 영화 수가 많은 상위 6개 국적만 비교한다.

#### [파이썬 코드 1-28]

```
df = 영화데이터[
    영화데이터["스크린수"].notna() & 영화데이터["대표국적"].notna()
].copy()
top6 = df["대표국적"].value_counts().head(6).index
df = df[df["대표국적"].isin(top6)]

plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x="대표국적", y="스크린수",
            color="lightblue", fliersize=3, linewidth=1)
plt.title("대표국적별 스크린수 분포")
plt.xlabel("대표국적")
plt.ylabel("스크린 수")
plt.xticks(rotation=45, ha="right")
sns.despine()
plt.tight_layout()
plt.show()
```

상자그림은 범주의 수가 많아져도 중앙값, 사분위수, 이상치 유무를 한 화면에서 비교할 수 있다는 장점이 있다.

## A.1.10 자료 변환하여 시각화하기

스크린수와 매출액의 관계는 매출액에 로그를 취하면 훨씬 잘 보인다. 두 가지 방식으로 같은 결과를 얻을 수 있다. 변수 자체를 로그 변환하는 방식과, 변수는 그대로 두고 축의 스케일만 로그로 바꾸는 방식이다.

## 1) 변수를 직접 로그 변환

[파이썬 코드 1-29]

```
df = 영화데이터.dropna(subset=["스크린수", "매출액"]).copy()
df = df[(df["스크린수"] > 0) & (df["매출액"] > 0)]
df["log_매출액"] = np.log(df["매출액"])

plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x="스크린수", y="log_매출액", alpha=0.6, s=20)
plt.xlabel("스크린수")
plt.ylabel("log(매출액)")
plt.title("로그 변환된 매출액 vs 스크린수")
plt.tight_layout()
plt.show()
```

## 2) 축 스케일만 로그로 변환

[파이썬 코드 1-30]

```
plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x="스크린수", y="매출액", alpha=0.6, s=20)
plt.yscale("log")
plt.xlabel("스크린수")
plt.ylabel("매출액 (로그 스케일)")
plt.title("로그 축 적용: 매출액 vs 스크린수")
plt.tight_layout()
plt.show()
```

두 그래프는 점들의 위치는 같지만 y축 라벨이 다르다. 전자는  $\log(\text{매출액})$  값이, 후자는 원래 매출액 값이 로그 간격으로 표시된다.

## A.1.11 조건부 비교: 패킷

여러 조건을 동시에 비교하기 위해 `sns.relplot()`의 `row/col` 인자를 이용해 패킷을 만든다. 5년내 개봉작 여부(행)와 대표국적 상위 4개(열)를 조건으로 삼아, 각 패널 안에서 스크린수와 관객수의 관계를 본다.

[파이썬 코드 1-31]

```

df = 영화데이터[
    (영화데이터["스크린수"] > 0) &
    (영화데이터["관객수"] > 0) &
    영화데이터["대표국적"].notna() &
    영화데이터["5 년내개봉작"].notna()
].copy()

# 패킷이 너무 많으면 해석이 어려우므로 상위 4 개 국적만
top4 = df["대표국적"].value_counts().head(4).index
df = df[df["대표국적"].isin(top4)]
df["대표국적"] = df["대표국적"].astype("category")
df["5 년내개봉작"] = df["5 년내개봉작"].astype("category")

g = sns.relplot(
    data=df, x="스크린수", y="관객수", kind="scatter",
    row="5 년내개봉작", col="대표국적",
    alpha=0.6, height=3.5, aspect=1,
)
for ax in g.axes.flat:
    ax.set_yscale("log")

g.set_axis_labels("스크린수", "관객수 (로그 스케일)")
g.fig.subplots_adjust(top=0.9)
g.fig.suptitle("5 년내개봉작 여부와 대표국적에 따른 스크린수 vs 관객수",
               fontsize=14)
plt.show()

```

행 방향으로는 개봉 시기에 따른 차이가, 열 방향으로는 국적에 따른 차이가 동일한 좌표축 위에서 비교된다. 패킷을 사용하면 각 패널 안의 비교와 패널 사이의 비교가 공간적으로 분리되기 때문에, 색이나 기호만으로 구분하는 경우보다 인지 부담이 줄어든다.

## A.2 2 부 실습

이 장에서는 우리나라 프로야구 불펜 투수 데이터를 이용하여 본문의 2부에서 다룬 선형 회귀분석과 비모수 회귀분석을 파이썬을 이용하여 직접 진행해 본다.

### A.2.1 탐색적 자료 분석

본격적인 분석에 앞서 데이터의 전반적인 특징과 변수 간의 관계를 파악하는 탐색적 자료 분석(Exploratory Data Analysis, EDA)을 진행한다. 먼저 필요한 라이브러리(모듈)를 불러온다.

#### [파이썬 코드 2-1]

```
# 라이브러리 로딩
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

pandas 모듈의 `read_csv()` 를 이용하여 데이터 파일 `reliever_data.csv`를 불러온다. `head()` 를 이용해 데이터의 앞부분을 확인할 수 있으며, `n = 5`와 같이 표시할 행의 수를 명시할 수도 있다. 단, 데이터를 불러올 때는 파일의 경로가 정확해야 함에 주의한다.

#### [파이썬 코드 2-2]

```
# 데이터 로딩
reliever_data_file = '데이터/reliever_data.csv' # 필요시 데이터 경로 설정
baseball = pd.read_csv(reliever_data_file, encoding = 'UTF-8')
# 데이터 앞부분 확인 (5 행까지)
baseball.head(n = 5)
```

`baseball.head()` 를 실행하면 `log_salary` 열의 값들이 8 ~ 12 사이의 숫자로 나타나는 것을 볼 수 있다. 이는 실제 연봉(단위: 만 원)에 자연로그를 취한 값으로, 앞서 우리가 논의했던 분석의 편의와 정규성을 확보하기 위한 변환이 이미 완료된 상태임을 의미한다. 분석에 사용할 주요 변수들을 소개하면 다음과 같다.

#### 프로야구 데이터 주요 변수 소개

**G:** 지난 시즌 출전 경기 수

**ERA:** 지난 시즌 평균자책점

**closer:** 지난 시즌 마무리 투수 여부 (1이면 마무리 투수)

**log\_salary**: 다음 시즌 계약 연봉의 로그 변환값 ( $\log_e$  이용)

참고사항: 2020~2024시즌 성적과 2021~2025 계약 연봉 사용

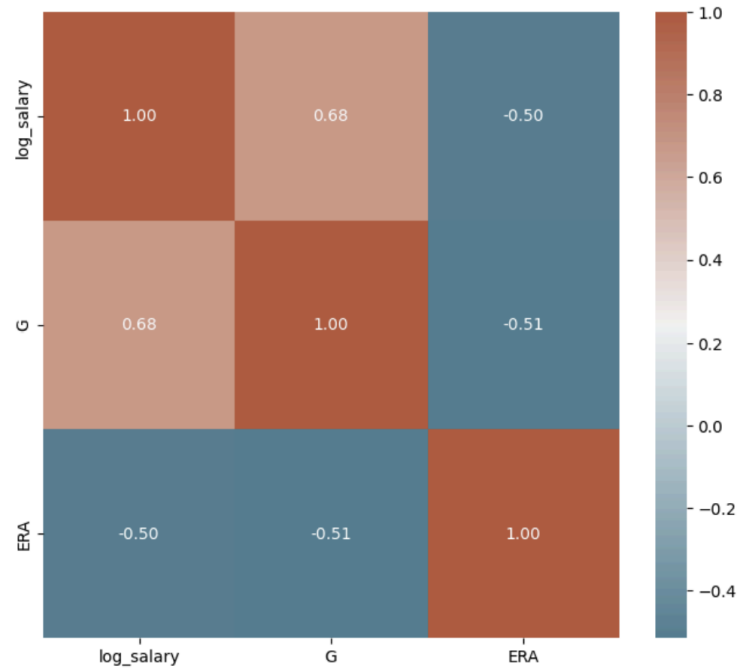
여기에서 지난 시즌 출전 경기 수 **G**, 지난 시즌 평균자책점 **ERA**, 마무리 투수 여부 **closer**를 설명변수로, 로그 변환된 다음 시즌 연봉 **log\_salary**를 반응변수로 이용한다.

본격적으로 회귀모형을 적합하기 전에 변수들이 서로 어떤 관계를 맺고 있는지 이해하는 과정은 매우 중요하다. 그중에서도 **상관계수(correlation coefficient)**를 확인하는 작업은 각 지표가 서로 얼마나 강한 선형 연관성을 가졌는지 파악하는 가장 기본적인 방법이다. 예를 들어, 출전 경기 수 **G**가 많을수록 연봉 **log\_salary**가 높아지는 경향이 있다면 두 변수는 양(+)의 상관관계를 가진다. 반대로 성적이 나쁠수록(평균자책점 **ERA**가 높을수록) 연봉이 낮아진다면 음(-)의 상관관계를 가질 것이다. 이러한 탐색을 통해 예측에 유용한 변수들을 선별하고, 서로 선형적으로 강하게 연관된 변수들을 미리 확인하여 데이터 전반의 구조를 이해할 수 있다. 따라서 상관계수를 확인하는 과정은 회귀모형을 해석력 있고 안정적으로 구성하기 위한 첫 단계라 할 수 있다.

이제 **corr()**을 이용해 상관계수 행렬(correlation matrix)을 만들고, **seaborn** 라이브러리의 **heatmap()**으로 이를 시각화해 보자.

### [파이썬 코드 2-3]

```
# 상관계수 행렬 계산
selected_cols = ['log_salary', 'G', 'ERA']
cor_mat = baseball[selected_cols].corr()
# 상관계수 행렬 시각화
plt.figure()
sns.heatmap(cor_mat, annot=True, fmt=".2f", cmap=sns.diverging_palette(220, 20,
    as_cmap=True))
plt.show()
```

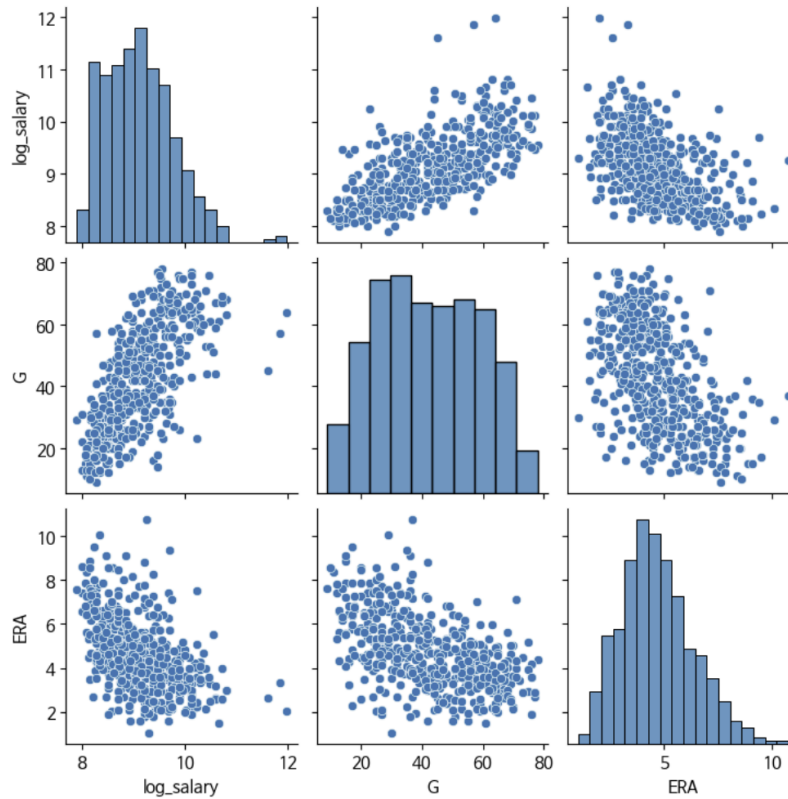


[그림 A-2-1] G, ERA, log\_salary에 대한 상관계수 행렬

[그림 A-2-1]의 히트맵을 통해 G와 log\_salary 사이에는 뚜렷한 양의 상관관계(0.68)가, ERA와 log\_salary 사이에는 음의 상관관계(-0.51)가 존재함을 수치로 확인할 수 있다. 하지만 상관계수는 오직 '**직선적인 관계**'만을 요약할 뿐이다. 데이터 속에 숨겨진 구체적인 곡선의 흐름이나 개별 관측치의 분포를 파악하기 위해서는 반드시 산점도를 그려보아야 한다. seaborn 라이브러리의 `pairplot()`을 이용하면 변수 간의 관계를 한눈에 보여주는 산점도 행렬을 얻을 수 있다.

#### [파이썬 코드 2-4]

```
# log_salary, G, ERA 변수들의 산점도 행렬 그리기
sns.pairplot(baseball, vars=['log_salary', 'G', 'ERA'], diag_kind="kde")
plt.show()
```



[그림 A-2-2] log\_salary, G, ERA에 대한 산점도

상관계수 행렬로 확인했던 변수 간의 강도를 [그림 A-2-2]의 산점도를 통해 시각적으로 재확인할 수 있다. 특히 경기 수(G)가 늘어남에 따라 로그 연봉이 전반적으로 우상향하는 흐름이 관찰되는데, 이 흐름이 얼마나 곧은 직선에 가까운지, 혹은 특정 구간에서 굴곡이 생기는지를 살피는 것이 이후 선형 회귀와 비모수 회귀 중 무엇을 강조할지 결정하는 중요한 단서가 된다.

## A.2.2 선형 회귀분석 실습

이제 선형 회귀분석을 파이썬을 이용해 진행해 본다.

### 설명변수가 출전 경기 수와 평균자책점인 경우 다중 선형 회귀분석 및 진단

출전 경기 수 G와 평균자책점 ERA를 설명변수로, 로그 변환한 다음 시즌 연봉 log\_salary를 반응 변수로 사용하여 선형 회귀모형을 적합해 보자. 파이썬의 statsmodels 라이브러리는 통계학적으로 매우 상세한 결과표를 제공한다.

#### [파이썬 코드 2-5]

```
# 선형회귀분석을 위한 모듈 로드
import statsmodels.formula.api as smf
```

```
# 다중선형회귀 모형의 형태 선언 (로그 연봉 ~ 경기수 + 평균 자책점)
formula_G_ERA = "log_salary ~ G + ERA"

# 다중선형회귀모형 적합
fit_G_ERA = smf.ols(formula=formula_G_ERA, data=baseball).fit()

# 다중선형회귀모형 적합 결과 확인
print(fit_G_ERA.summary())
```

```

                        OLS Regression Results
=====
Dep. Variable:          log_salary      R-squared:                0.493
Model:                  OLS             Adj. R-squared:           0.491
Method:                 Least Squares    F-statistic:              232.6
Date:                  Thu, 16 Oct 2025  Prob (F-statistic):      2.83e-71
Time:                  12:00:00          Log-Likelihood:           -319.68
No. Observations:      481              AIC:                     645.4
Df Residuals:          478              BIC:                     657.9
Df Model:              2
Covariance Type:       nonrobust
=====
                        coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept             8.5401         0.119     72.029     0.000         8.307         8.773
G                     0.0228         0.001     15.210     0.000         0.020         0.026
ERA                   -0.0793         0.015     -5.275     0.000        -0.109        -0.050
=====
Omnibus:              101.962    Durbin-Watson:           2.139
Prob(Omnibus):         0.000    Jarque-Bera (JB):        227.284
Skew:                  1.105    Prob(JB):                4.42e-50
Kurtosis:              5.540    Cond. No.                253.
=====

```

위의 출력 결과는 크게 세 부분으로 나누어 읽을 수 있다.

- **모형의 전체 성능 (상단):** 오른쪽 상단의 **결정계수(R-squared)**를 보자. 값이 0.493 이므로, 우리가 선택한 두 변수가 불펜 투수 연봉 변동의 약 49.3%를 설명하고 있음을 알 수 있다.
- **개별 변수의 영향력 (중간):** 각 변수의 **회귀계수(coef)**와 **P-값(P>|t|)**을 확인한다. G와 ERA 모두 log\_salary와 통계적으로 유의한 관련이 나타났다. 특히 G의 계수가 0.0228 인 것은 출전 경기 수가 1 회 늘어날 때마다 로그 연봉이 약 0.0228 만큼(실제 액수로는 약 2.3%) 상승한다는 뜻이다.
- **모형 가정 확인 (하단):** 하단의 **Omnibus** 나 **Jarque-Bera** 는 잔차가 정규분포를 따르는지 검정하는 지표다. 이 값들의 P-값이 매우 작다면, 잔차가 정규성을 벗어났을 가능성이 크므로 모형 진단 그래프를 통해 추가 확인이 필요하다.

그 외 출력 결과에 포함된 다양한 통계량의 구체적인 의미는 다음 표와 같다.

| 구분    | 항목                             | 의미 및 해석                                                                            |
|-------|--------------------------------|------------------------------------------------------------------------------------|
| 모형 정보 | Adj. R-squared                 | 수정된 결정계수. 설명변수가 늘어날수록 결정계수가 무의미하게 높아지는 단점을 보완한 값이다.                                |
|       | F-statistic/Prob (F-statistic) | 회귀모형 전체가 통계적으로 유의미한지 판정하는 F-검정통계량과 P-값이다.                                          |
|       | AIC / BIC                      | 모형의 복잡도와 적합도를 동시에 고려한 지표다. 여러 모형을 비교할 때 값이 작을수록 좋은 모델이다.                           |
|       | Log-Likelihood                 | 최대가능도 추정법에서의 로그 가능도 값이다. 확률적으로 모델이 데이터를 얼마나 잘 설명하는지 나타낸다.                          |
| 회귀 계수 | std err / t                    | 회귀계수의 표준오차와 t-통계량이다. 다른 조건이 같을 때 $ t $ 가 클수록 해당 회귀계수가 0과 다르다는 통계적 증거가 강하다고 볼 수 있다. |
|       | [0.025 0.975]                  | 회귀계수의 95% 신뢰구간이다. 이 범위 안에 0이 포함되지 않아야 통계적으로 유의미하다고 평가한다.                           |
| 진단 지표 | Durbin-Watson                  | 잔차 사이의 상관성(독립성)을 체크한다. 2에 가까울수록 독립적이며, 0이나 4에 가까우면 자기상관이 의심된다.                     |
|       | Cond. No.                      | 설계행렬의 수치적 조건을 나타내                                                                  |

|  |          |                                                                        |
|--|----------|------------------------------------------------------------------------|
|  |          | 는 지표다. 값이 크면 설명변수 간 강한 선형 관계나 변수 스케일 차이 등으로 수치적 불안정성이 있을 가능성을 점검해야 한다. |
|  | Skew     | 잔차 분포의 비대칭성을 나타내며 0에 가까울수록 좌우 대칭이다.                                    |
|  | Kurtosis | 잔차 분포의 꼬리가 얼마나 두꺼운지를 나타내며 기준값인 3보다 크면 정규분포보다 꼬리가 두꺼워 이상치가 존재할 가능성이 높다. |

회귀모형이 통계적으로 타당하려면, 예측하고 남은 오차인 '잔차'가 특정 패턴 없이 무작위적이어야 한다. 먼저 스튜던트화 잔차를 계산하여 잔차도를 그려보자.

#### [파이썬 코드 2-6]

```
# 선형모형을 이용한 예측값 계산
fitted_vals = fit_G_ERA.fittedvalues

# 스튜던트화 잔차 계산
influence = fit_G_ERA.get_influence()
residuals = influence.resid_studentized_internal
```

#### [파이썬 코드 2-7]

```
# 잔차도 그리기
plt.figure()
sns.residplot(x=fitted_vals, y=residuals)
plt.axhline(0, color='gray', linestyle='--')
plt.show()
```

잔차도에서 점들이 0을 중심으로 특별한 모양(갈때기나 곡선 등) 없이 고르게 퍼져 있다면 선형성 가정이 타당하다고 본다. 만약 특정 방향으로 쏠린다면 모델 구조를 개선해야 한다는 신호다.

마지막으로 잔차가 정규분포를 따르는지 확인하기 위해 정규 Q-Q 플롯을 그려본다.

#### [파이썬 코드 2-8]

```
from statsmodels.graphics.gofplots import qqplot
# 정규분포 Q-Q 플롯 그리기
qqplot(residuals, line='45', fit = True, alpha = 0.5)
plt.show()
```

정규 Q-Q 플롯에서 점들이 45도 직선 위에 예쁘게 놓여 있을수록 잔차가 정규분포를 잘 따른다고 판단한다. 만약 양 끝단이 직선에서 크게 벗어난다면 이상치가 존재하거나 데이터의 분포가 왜곡되었을 가능성이 크다. 이러한 진단 결과는 우리가 앞서 배운 비모수 회귀와 같은 더 유연한 모델이 왜 필요한지를 알려주는 강력한 근거가 된다.

## 설명변수가 출전 경기 수와 마무리 투수 여부인 경우 다중 선형 회귀 분석

현실의 데이터에는 숫자뿐만 아니라 '성별', '지역', 혹은 투수의 역할처럼 범주(Category)로 나뉘어진 정보가 많다. 이번에는 마무리 투수 여부(closer)라는 범주형 변수가 연봉에 어떤 영향을 미치는지 알아본다.

파이썬에서 closer 변수는 0(일반 불펜)과 1(마무리 투수)로 기록되어 있다. 파이썬에서 이를 단순한 숫자로 인식할 수 있으므로, 명시적으로 "이것은 그룹을 나타내는 범주형 변수다"라고 알려주는 과정이 필요하다. 다음은 closer가 범주형인지 확인하는 코드이다.

### [파이썬 코드 2-9]

```
# closer 변수의 데이터형 확인
baseball["closer"].dtype
```

만약 closer가 범주형이 아니라면 다음과 같이 데이터형을 범주형으로 바꿔서 사용한다.

### [파이썬 코드 2-10]

```
# closer 변수를 범주형으로 변환
baseball["closer"] = baseball["closer"].astype("category")
# 변환 후 다시 확인
baseball["closer"].dtype
```

이제 출전 경기 수 G와 마무리 투수 여부 closer를 설명변수로 로그 변환한 다음 시즌 연봉 log\_salary를 반응변수로 사용한 선형 회귀모형을 적합해 본다.

### [파이썬 코드 2-11]

```
# 교호작용을 고려하지 않은 모형
factor_fit1 = smf.ols("log_salary ~ G + closer", data = baseball).fit()
print(factor_fit1.summary())
```

결과표에서 `closer[T.1]`의 계수를 확인해 보자. 반응변수가 로그 연봉이므로, 이 값은 출전 경기 수가 같더라도 마무리 투수 보직을 맡는 것만으로 연봉이 몇 퍼센트(%)나 '점프'하는지를 나타내는 보직 프리미엄이다. 2.2절에서도 언급했듯이 이 계수가 0.579이므로 마무리 투수는 불펜 투수보다 약 78%( $e^{0.579} \approx 1.78$ ) 더 높은 연봉 수준에서 시작한다는 의미가 된다. 즉, 두 그룹 간의 기본 연봉

수준의 차이를 보여준다.

이번에는 경기 수에 따른 연봉 상승 폭이 보직에 따라 다를 수 있음을 가정하는 교호작용 (Interaction) 모형을 적합해 보자. 파이썬에서는 두 변수 사이에 \* 기호를 사용하여 간단히 표현할 수 있다.

#### [파이썬 코드 2-12]

```
# 교호작용을 고려한 모형
factor_fit2 = smf.ols("log_salary ~ G * closer", data = baseball).fit()
print(factor_fit2.summary())
```

결과표에서 **G:closer[T.1]**의 계수를 확인해 보자. 이 값은 -0.0124로 마무리 투수는 경기 수가 쌓일수록 불펜 투수보다 연봉 상승률이 상대적으로 낮다는 것을 의미한다. 마무리 투수라는 보직 자체에 따른 기본 연봉 수준(절편)은 높지만 (**closer[T.1]**이 양수) 한 경기를 더 뛴 때마다 증가하는 로그 연봉의 폭은 불펜 투수보다 작다고 해석할 수 있다.

이번에는 범주형 설명변수가 포함된 다중 선형 회귀모형을 시각화해보자. 교호작용을 고려하지 않은 경우를 시각화해 본다. 이 모형은 두 그룹의 기울기가 같다고 가정하므로, 그래프상에서는 평행한 두 직선으로 나타난다. 교호작용을 고려한 경우는 아래 코드에서 **factor\_fit1**을 **factor\_fit2**로 변경하면 된다.

#### [파이썬 코드 2-13]

```
# 시각화를 위한 설명변수의 range에 해당하는 x 값들을 구하기, 길이 100
var_range = np.linspace(baseball['G'].min(), baseball['G'].max(), 100)

# 적합한 값인 fitted value를 담아줄 데이터프레임 만들어서 넣기
pred_df1 = pd.concat([
    pd.DataFrame({'G': var_range, 'closer': 0}),
    pd.DataFrame({'G': var_range, 'closer': 1})
], ignore_index=True)
pred_df1['closer'] = pred_df1['closer'].astype('category')
pred_df1['log_salary'] = factor_fit1.predict(pred_df1)

# 그래프 그리기
plt.figure()
for label_val, color in zip([0, 1], ['red', 'blue']):
    subset_data = baseball[baseball['closer'] == label_val]
    subset_pred = pred_df1[pred_df1['closer'] == label_val]

    plt.scatter(subset_data['G'], subset_data['log_salary'], color=color,
                label='relief pitcher' if label_val == 0 else 'closer')
    plt.plot(subset_pred['G'], subset_pred['log_salary'], color=color)
```

```
plt.legend(loc='lower right')
plt.show()
```

빨간색 선(불펜)과 파란색 선(마무리)이 나란히 있는 것을 볼 수 있다. 만약 교호작용 모형 (`factor_fit2`)을 적용하면 두 선은 서로 다른 기울기를 가지며 경기 수가 많아질수록 간격이 좁아지는 형태를 띈다.

## 설명변수가 출전 경기 수인 경우 선형 회귀분석을 이용한 예측

다음은 출전 경기 수  $G$  만을 설명변수로 하는 선형 회귀모형을 적합하고, 특정 경기 수( $G=40$ )를 소환한 선수의 예상 연봉을 구해본다.

### [파이썬 코드 2-14]

```
# 선형회귀모형 적합 (연봉 ~ 경기 수)
fit_G = smf.ols('log_salary ~ G', data=baseball).fit()
# 새로운 데이터 설정 (출전 경기 수 = 40)
new_G = pd.DataFrame({'G':[40]})
# 예측값 및 예측구간 계산
fit_G.get_prediction(new_G).summary_frame(alpha=0.05)
```

출전 경기 수  $G=40$  일 때 로그 연봉의 평균 예측값은 9.0696이고, 예측구간은 (8.1159, 10.0233)이다. 이를 지수변환하면 평균 예측 연봉은 약 8,687만 원이고, 예측구간은 약 (3,347만 원, 2억 2,545만 원)이다. 예측구간이 넓은 주된 이유는 개별 선수의 연봉이 회귀선 주변에서 크게 변동하기 때문이다. 로그 연봉 모형에서 계산된 구간을 원래 연봉 단위로 지수변환하면 구간이 비대칭적이고 승법적인 형태로 나타나므로, 금액 단위에서는 폭이 더욱 크게 보일 수 있다.

### 데이터 분석가의 한마디

예측구간이 넓다는 것은 경기 수와 평균자책점만으로는 개별 선수의 연봉 변동을 충분히 설명하기 어렵다는 뜻이다. 로그 연봉 모형에서 얻은 예측구간을 원래 연봉 단위로 되돌리면 구간이 비대칭적이고 승법적인 형태로 표현되므로 금액 단위에서는 폭이 크게 보일 수 있다. 따라서 더 많은 설명변수를 고려하거나, 데이터의 비선형적 구조를 반영하는 모형을 함께 검토할 필요가 있다.

## A.2.3 비모수 회귀분석 실습

이제 비모수 회귀분석을 파이썬으로 실습해 보자. 선형 회귀가 "데이터는 직선이다"라는 가정을 전제로 했다면, 비모수 회귀는 "데이터가 들려주는 모양을 그대로 따라가 보겠다"는 유연한 접근법이다. 여기서도 반응변수는 로그 변환된 연봉(`log_salary`)을 사용하며, 먼저 설명변수 하나를 이용한 커널 회귀부터 시작해 본다.

## 커널 회귀분석

출전 경기 수(G)에 따른 로그 연봉의 변화를 커널 회귀로 포착해 보자. 파이썬에서는 `statsmodels` 라이브러리의 `KernelReg`를 이용한다. 커널 회귀에서 가장 중요한 결정은 **대역폭(Bandwidth, bw)**이다. 대역폭은 곡선이 얼마나 민감하게 데이터의 굴곡을 반영할지를 결정하는 '부드러움의 정도'라고 이해할 수 있다. 아래 코드에서는 가장 널리 쓰이는 가우시안 커널을 사용하고, 대역폭을 3으로 설정하여 적합을 진행한다.

### [파이썬 코드 2-15]

```
from statsmodels.nonparametric.kernel_regression import KernelReg
# 커널 회귀모형 선언 및 적합
# endog: 반응변수, exog: 설명변수, var_type='c': 연속형 변수 의미, bw=[3]: 대역폭 설정
kr = KernelReg(endog=baseball.log_salary, exog=baseball.G, var_type='c',
               reg_type='lc', bw=[3])
# 적합된 값 계산
y_fit, _ = kr.fit()
```

이제 추정된 곡선이 실제 데이터와 얼마나 잘 어우러지는지 시각화해 보자. 이를 위해 경기 수의 전체 범위(min ~ max)를 200개의 촘촘한 점으로 나눈 뒤, 각 점에서의 예측값을 계산하여 산점도 위에 그려본다.

### [파이썬 코드 2-16]

```
# 추정 곡선을 그리기 위한 촘촘한 x 축 값 (G) 생성
x_grid = np.linspace(baseball['G'].min(), baseball['G'].max(), 200)
# 생성한 x 축 값에 대한 예측값 계산
y_bw3, _ = kr.fit(x_grid)
# 산점도와 커널 회귀 곡선 시각화
plt.figure()
plt.scatter(baseball['G'], baseball['log_salary'], color='gray')
plt.plot(x_grid, y_bw3, color='red')
plt.show()
```

## 스플라인 회귀

이번에는 데이터를 구간별로 나누어 다항식을 연결하는 3차 B-스플라인(B-Spline) 회귀모형을 적합해 보자. 2.3절에서 강조했듯, 스플라인 회귀의 핵심은 곡선의 흐름이 바뀌는 지점인 **매듭점(Knots)**을 어디에 설정하느냐에 있다. 여기서는 경기 수(G)의 사분위수(25%, 50%, 75% 지점)를 매듭점으로 사용하여, 데이터가 집중된 구간을 세밀하게 나누어 본다.

### [파이썬 코드 2-17]

```
import statsmodels.api as sm
```

```

from statsmodels.gam.api import BSplines # B-스플라인 사용하기 위한 라이브러리

# 매듭점 설정: G의 1사분위수, 중앙값, 3사분위수를 기준으로 구간을 나눔
G_knots = baseball['G'].quantile([0.25, 0.50, 0.75]).to_list()

# B-스플라인 기저 생성
bs = BSplines(x=baseball[['G']], df=[7], degree=[3], knot_kwds={"knots":
    G_knots}))

# 반응변수와 설계행렬 생성
X_spline = sm.add_constant(bs.basis, has_constant='add')
y = baseball['log_salary']

# 회귀모형 적합
fit_spline = sm.GLM(y, X_spline, family=sm.families.Gaussian()).fit()

```

추정된 스플라인 곡선을 시각화하기 위해, 전체 경기 수 범위를 200개의 촘촘한 범위로 나눈 후 이에 대한 예측값을 계산하고 산점도 위에 그려보자.

#### [파이썬 코드 2-18]

```

# 시각화를 위한 촘촘한 경기 수 값 생성
G_grid = np.linspace(baseball['G'].min(), baseball['G'].max(), 200)
G_grid_df = pd.DataFrame({'G': G_grid})

# 예측용 데이터에 대한 B-스플라인 기저 생성
bs_pred = BSplines(x=G_grid_df[['G']], df=[7], degree=[3],
    knot_kwds={"knots":G_knots}))

# 예측값 계산
X_pred = sm.add_constant(bs_pred.basis, has_constant='add')
y_spline_pred = fit_spline.predict(X_pred)

# 산점도와 스플라인 곡선 시각화
plt.figure()
plt.scatter(baseball['G'], baseball['log_salary'],color='gray')
plt.plot(G_grid, y_spline_pred, color='blue')
plt.show()

```

## 스무딩 스플라인

이번에는 모든 데이터 포인트를 매듭점으로 사용하되, 곡선의 급격한 변화에 '별점'을 부여하여 전체적인 흐름을 매끄럽게 다듬는 스무딩 스플라인을 적합해 보자. 파이썬의 `scipy.interpolate` 모듈에 있는 `make_smoothing_spline` 함수를 사용한다.

이 함수는 설명변수(G)의 값이 중복되거나 정렬되어 있지 않으면 오류가 발생할 수 있다. 따라서 분

석 전에 동일한 경기 수에 대응하는 연봉들은 그 평균값을 구하고 사용하고, 경기 수 순서대로 정렬하는 전처리 과정이 선행되어야 한다.

#### [파이썬 코드 2-19]

```
from scipy.interpolate import make_smoothing_spline

# 전처리: 동일한 G 값에 대해서는 log_salary의 평균을 사용
smooth_df = baseball[['G', 'log_salary']].groupby('G', as_index=False).mean()

# 전처리: 경기수 G 기준 오름차순 정렬
smooth_df = smooth_df.sort_values(by='G')

# 스무딩 스플라인 적합
# lam: 매끄러움의 정도를 조절하는 조정 모수 (lambda)
fit_smooth = make_smoothing_spline(smooth_df['G'], smooth_df['log_salary'],
                                   lam=182)
```

이제 적합된 스무딩 스플라인 곡선이 전체 데이터의 흐름을 얼마나 안정적으로 따라가는지 시각화해 보자.

#### [파이썬 코드 2-20]

```
# 시각화를 위한 촘촘한 경기 수 G 생성
plot_x = np.linspace(smooth_df['G'].min(), smooth_df['G'].max(), 300)
# 적합된 모델을 사용하여 예측값 계산
plot_y = fit_smooth(plot_x)

# 결과 시각화
plt.figure()
plt.scatter(baseball['G'], baseball['log_salary'], color='gray')
plt.plot(plot_x, plot_y, color='blue')
plt.show()
```

## 일반화 가법모형

설명변수가 여러 개인 경우, 각 변수가 반응변수에 미치는 개별적인 비선형 효과를 동시에 고려하기 위해 **일반화 가법 모형(GAM)**을 사용한다. 여기서는 출전 경기 수(G)와 평균자책점(ERA)이 연봉에 미치는 영향을 각각 스플라인 항으로 구성하여 분석해 보자.

파이썬에서는 **pygam**의 **GAM()**을 사용할 수 있다. **pygam**이 설치되어 있지 않다면 코드 셀에서 `!pip install pygam`을 실행한다.

#### [파이썬 코드 2-21]

```
from pygam import GAM, s
```

```
# GAM 모형 정의: 각 설명변수에 대해 3 차 스플라인(spline_order=3) 적용
# n_splines: 매듭점의 개수와 유사한 개념으로, 곡선의 유연성을 결정
gam_fit = GAM( s(0, spline_order=3, n_splines=8)
               + s(1, spline_order=3, n_splines=8)).fit(baseball[['G', 'ERA']],
               baseball['log_salary'])
# 적합 결과 요약 확인
print(gam_fit.summary())
```

GAM의 가장 큰 장점은 다른 변수의 영향을 통제한 상태에서, 특정 변수 하나가 연봉에 미치는 '순수한 효과'를 그래프로 그려낼 수 있다는 점이다. 예를 들어, G의 효과를 보기 위해 ERA는 모든 선수가 동일하게 '평균 성적'을 냈다고 가정(고정)하고 그래프를 그린다.

#### [파이썬 코드 2-22]

```
# G의 변화에 따른 부분효과의 변화를 관찰하기 위한 촘촘한 G의 값 생성
G_grid = np.linspace(baseball['G'].min(), baseball['G'].max(), 100)

# G 부분효과 계산을 위해 다른 변수(ERA)를 평균값으로 고정하여 환경을 통제
ERA_mean = baseball['ERA'].mean()
X_G = np.column_stack([G_grid, np.full_like(G_grid, ERA_mean)])

# G의 부분효과 계산
G_partial = gam_fit.partial_dependence(term=0, X=X_G)

# --- 부분 잔차(Partial Residuals) 계산 ---
# 전체 예측값 계산
overall_predictions = gam_fit.predict(baseball[['G', 'ERA']])
# 전체 잔차(Full Residuals) 계산: 실제값 - 전체 예측값
full_residuals = baseball['log_salary'] - overall_predictions
# 부분 잔차 계산
G_resid = full_residuals + gam_fit.partial_dependence(term=0, X=baseball[['G',
    'ERA']]).values)
```

이제 앞서 구한 부분 효과 곡선과 실제 데이터의 흩어짐을 나타내는 부분 잔차를 함께 시각화하자.

#### [파이썬 코드 2-23]

```
plt.figure()
# 부분 효과 곡선 (파란선)
plt.plot(G_grid, G_partial, color='blue')
# 부분 잔차 산점도 (회색점)
plt.scatter(baseball['G'], G_resid, color='grey')
plt.show()
```

ERA의 부분효과와 부분 잔차를 구하고 시각화하기 위해서는 위 코드에서 G에 대응되는 부분을 ERA

로, ERA 에 대응되는 부분을 G로 변경하면 된다.

## A.3 3 부 실습

본 부록은 3부 본문 1-6장의 분류 방법을 Python으로 처음부터 끝까지 실행해 보는 자료이다. 주요 실습은 hn19.csv 데이터를 사용하며, 일부 예시는 별도의 예제 데이터를 사용한다. 본문에서 생략된 세부 코드와 출력을 모두 담는다.

### A.3.1 파이썬 환경 준비

#### 데이터 파일 위치

본 부록의 코드를 실행하려면 hn19.csv 파일을 책 루트의 데이터/ 폴더에 두어야 한다. 데이터의 출처와 변수 설명은 온라인 데이터 가이드를 참조하라.

#### 필수 Python 패키지

본 부록 전반에서 사용하는 필수 Python 패키지는 다음과 같다. 가상 환경에서 `pip install ...` 명령으로 설치한 뒤, 다음과 같이 임포트한다.

선택 실습용 추가 패키지: `xgboost`, `lightgbm`, `catboost`, `torch`. XGBoost, LightGBM, CatBoost, PyTorch 예제를 실행할 때에만 설치한다.

#### [파이썬 코드 3-1]

```
# 데이터 처리/시각화
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 분류 모형
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

본 부록의 그래프는 한글 레이블을 사용하므로, 한글이 깨질 경우 matplotlib의 한글 폰트 설정(예: `plt.rcParams["font.family"] = "AppleGothic"`)을 먼저 적용한다.

## A.3.2 분류 도입과 데이터 탐색

### 실습 1: 국민건강영양조사 데이터를 이용한 당뇨 위험군 분류

#### 분석을 위해 필요한 라이브러리 및 데이터 불러오기

데이터 확인 및 정제, 시각화를 위해 필요한 모듈인 `pandas`, `numpy`, `matplotlib.pyplot`, `seaborn`을 불러온다. 분석에 필요한 모듈인 `statsmodels`(로지스틱 회귀)와 `scikit-learn`(KNN, 분류 나무, 랜덤 포레스트, LDA/QDA, 나이브 베이즈)을 함께 불러온다.

#### [파이썬 코드 3-2]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

이제, `pandas` 모듈의 `read_csv()`를 이용하여 `hn19.csv` 파일을 불러온다. 이 데이터는 2019년 국민건강영양조사 자료에서 분석에 필요한 변수만을 추출한 것이다. 변수들은 다음과 같다.

- `age`: 만 나이 (단위: 세)
- `HE_BMI`: 체질량지수 BMI (단위:  $\text{kg}/\text{m}^2$ )
- `HE_HP`: 혈압 상태 (1 = 정상, 2 = 고혈압 전단계, 3 = 고혈압)
- `HE_DM_HbA1c`: 당뇨 상태 (1 = 정상, 2 = 당뇨 전단계, 3 = 당뇨)

주의: 데이터를 불러오기 위해 경로를 정확하게 설정해 주어야 한다.

#### [파이썬 코드 3-3]

```
hn_data_file = "데이터/hn19.csv"
data = pd.read_csv(hn_data_file, encoding="utf-8")
data.head(n=5)
```

결측값(NA)이 포함된 행은 분석에서 제외한다. `dropna()` 메서드를 이용하면 데이터프레임 내의 결측값이 있는 모든 행을 제거할 수 있다.

#### [파이썬 코드 3-4]

```
data = data[["age", "HE_BMI", "HE_HP",
            "HE_DM_HbA1c"]].dropna().reset_index(drop=True)
print("결측값 제거 후 데이터의 행 수:", len(data))
```

분류 모형의 반응변수와 설명변수를 정의하기 위해 변수 변환을 진행한다.

본 실습에서는 당뇨 또는 당뇨 전단계 여부를 반응변수로 설정한다. 즉, HE\_DM\_HbA1c가 2 이상이면 1(당뇨 위험군), 1이면 0(정상)으로 변환한다. 고혈압 또는 고혈압 전 단계 여부를 나타내는 범주형 설명변수 hp도 동일한 방식으로 생성한다.

BMI 변수도 보다 직관적인 이름인 bmi로 변경하며, 범주형 변수가 올바르게 인식될 수 있도록 .astype("category")를 이용하여 형 변환을 진행한다.

#### [파이썬 코드 3-5]

```
data["diabetes"] = (data["HE_DM_HbA1c"] >= 2).astype(int).astype("category")
data["hp"]       = (data["HE_HP"] >= 2).astype(int).astype("category")
data["bmi"]      = data["HE_BMI"]

data = data[["age", "bmi", "hp", "diabetes"]]
data.head(n=3)
```

반응변수의 분포를 확인한다. value\_counts() 메서드를 이용하면 범주형 변수의 빈도와 비율을 확인할 수 있다.

#### [파이썬 코드 3-6]

```
print(data["diabetes"].value_counts())
print()
print(data["diabetes"].value_counts(normalize=True).round(4))
```

### 변수 간의 상관관계 및 분포 파악하기

우선 수치형 변수인 age, bmi와 반응변수 diabetes 사이의 관계를 살펴보기 위해, 반응변수의 범주별로 설명변수의 분포를 비교하는 상자 그림(boxplot)을 그린다.

seaborn의 boxplot()을 이용해 두 그래프를 가로로 배치한다.

#### [파이썬 코드 3-7]

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.boxplot(x="diabetes", y="age", data=data, ax=axes[0],
            palette=["#6888BE", "#E07B91"], hue=None)
axes[0].set_title("당뇨 위험군별 나이 분포")

sns.boxplot(x="diabetes", y="bmi", data=data, ax=axes[1],
            palette=["#6888BE", "#E07B91"], hue=None)
axes[1].set_title("당뇨 위험군별 BMI 분포")

plt.tight_layout()
plt.show()
```

범주형 설명변수 hp와 반응변수 diabetes의 관계는 교차표(cross-tabulation)로 확인할 수 있다.

**[파이썬 코드 3-8]**

```
pd.crosstab(data["hp"], data["diabetes"], rownames=["hp"], colnames=["diabetes"])
```

두 범주형 변수의 관계를 비율로 시각화하기 위해 누적 막대그림을 그린다.

**[파이썬 코드 3-9]**

```
ct = pd.crosstab(data["hp"], data["diabetes"], normalize="index")

ax = ct.plot(kind="bar", stacked=True, figsize=(7, 5),
             color=["#6888BE", "#E07B91"], alpha=0.8)
ax.set_ylabel("비율")
ax.set_title("혈압 상태별 당뇨 위험군 비율")
ax.legend(title="diabetes")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

## A.3.3 로지스틱 회귀와 다범주 분류

### 단순 로지스틱 회귀모형 적합

우선 bmi 하나만을 설명변수로 이용하여 단순 로지스틱 회귀모형을 적합한다.

Python에서 로지스틱 회귀를 적합할 때는 `statsmodels.formula.api` 모듈의 `logit()` 을 이용한다.

함수 호출 시 전달해야 하는 인자는 다음과 같다.

- **formula**: 적합할 회귀모형의 형태를 나타낸다. 반응변수 ~ 설명변수 1 + ... + 설명변수 p 의 형태와 같이 반응변수와 설명변수는 ~로 연결하고, 설명변수끼리는 +로 연결한다.
- **data**: 사용할 데이터를 명시한다.

`logit()`을 이용해 모형을 선언하고 `fit()`을 통해 모형 적합을 완료한 후, `summary()`를 이용해 적합 결과를 확인할 수 있다.

**[파이썬 코드 3-10]**

```
import statsmodels.formula.api as smf

# diabetes 를 정수형으로 다시 변환 (statsmodels 의 요구사항)
data["diabetes_int"] = data["diabetes"].astype(int)

simple_fit = smf.logit("diabetes_int ~ bmi", data=data).fit(dis= False)
print(simple_fit.summary())
```

`summary()`의 결과에서 다음 정보를 확인할 수 있다.

- coef: 추정된 회귀계수  $\hat{\beta}$
- std\_err: 회귀계수의 표준오차
- z: Wald 검정통계량  $z = \hat{\beta} / \widehat{se}(\hat{\beta})$
- P>z: 검정의 p-value

적합된 모형식은 다음과 같다.

$$\log\left(\frac{\hat{p}}{1-\hat{p}}\right) = \hat{\beta}_0 + \hat{\beta}_1 \cdot \text{bmi}$$

회귀계수의 **오즈비(odds ratio)**는  $e^{\hat{\beta}}$ 로 계산할 수 있다.

#### [파이썬 코드 3-11]

```
print("회귀계수:")
print(simple_fit.params.round(4))
print()
print("오즈비 (exp(coef)):")
print(np.exp(simple_fit.params).round(4))
```

BMI가 1 증가할 때, 당뇨 위험군일 오즈가 몇 배 변하는지를 의미한다. BMI 5 증가 시의 오즈비도 함께 확인해보자.

#### [파이썬 코드 3-12]

```
or_5 = np.exp(simple_fit.params["bmi"] * 5)
print(f"BMI 5 증가 시 오즈비: {or_5:.4f}")
```

### 다중 로지스틱 회귀모형 적합

이번에는 age, bmi, hp를 모두 설명변수로 이용하여 다중 로지스틱 회귀모형을 적합한다.

적합 방법은 단순 로지스틱 회귀와 동일하게 smf.logit()을 이용한다. formula에서 설명변수를 +로 연결하면 된다. 범주형 변수는 c(...)로 감싸 명시적으로 처리한다.

#### [파이썬 코드 3-13]

```
multi_fit = smf.logit("diabetes_int ~ age + bmi + C(hp)",
    data=data).fit(dis=False)
print(multi_fit.summary())
```

추정된 회귀계수와 오즈비를 한눈에 확인하기 위한 표를 만들어보자.

#### [파이썬 코드 3-14]

```
coef_table = pd.DataFrame({
    "Estimate": multi_fit.params,
    "StdError": multi_fit.bse,
```

```

    "z_value":    multi_fit.tvalues,
    "p_value":    multi_fit.pvalues,
    "OddsRatio":  np.exp(multi_fit.params)
}).round(4)
coef_table

```

각 회귀계수에 대한 해석은 다음과 같다.

- **age**: 다른 설명변수가 같을 때, 나이가 1 세 증가하면 당뇨 위험균일 오즈가 약  $e^{\hat{\beta}_{age}}$  배 변한다.
- **bmi**: 다른 설명변수가 같을 때, BMI 가 1 증가하면 당뇨 위험균일 오즈가 약  $e^{\hat{\beta}_{bmi}}$  배 변한다.
- **C(hp)[T.1]**: 다른 설명변수가 같을 때, 고혈압(전단계 포함)인 사람은 정상 혈압인 사람에 비해 당뇨 위험균일 오즈가 약  $e^{\hat{\beta}}$  배 높다.

적합된 모형을 이용해 예측을 진행하자. 새로운 관측치를 데이터프레임으로 만들고, `predict()`를 이용해 추정 확률을 계산할 수 있다.

#### [파이썬 코드 3-15]

```

new_obs = pd.DataFrame({
    "age": [30, 30, 60, 60],
    "bmi": [22, 27, 22, 27],
    "hp":  pd.Categorical([0, 1, 0, 1], categories=[0, 1])
})

pred_probs = multi_fit.predict(new_obs)
new_obs["predicted_prob"] = pred_probs.round(3)
new_obs

```

### 분류 성능의 평가 — 혼동행렬, 민감도, 특이도

적합한 로지스틱 회귀모형으로부터 추정 확률을 구한 후, 임계값(threshold)을 적용하여 범주(0 또는 1)로 변환한다. 임계값의 기본값은 0.5이다. 추정 확률이 임계값을 초과하면 1(당뇨 위험균), 그렇지 않으면 0(정상)으로 분류한다.

#### [파이썬 코드 3-16]

```

pred_prob = multi_fit.predict(data)
pred_class = (pred_prob > 0.5).astype(int)

```

**혼동행렬(confusion matrix)**은 `sklearn.metrics`의 `confusion_matrix()` 또는 `pandas.crosstab()`을 이용해 만들 수 있다.

#### [파이썬 코드 3-17]

```

from sklearn.metrics import confusion_matrix

```

```
conf_mat = pd.crosstab(pred_class, data["diabetes_int"],
                        rownames=["predicted"], colnames=["actual"])
conf_mat
```

혼동행렬로부터 **정확도(accuracy)**, **민감도(sensitivity)**, **특이도(specificity)**를 계산한다.

- **정확도** =  $(TP + TN) / \text{전체}$
- **민감도** =  $TP / (TP + FN)$ , 실제 위험군 중 위험군으로 정확히 예측한 비율
- **특이도** =  $TN / (TN + FP)$ , 실제 정상 중 정상으로 정확히 예측한 비율

#### [파이썬 코드 3-18]

```
TN = conf_mat.loc[0, 0]
FN = conf_mat.loc[0, 1]
FP = conf_mat.loc[1, 0]
TP = conf_mat.loc[1, 1]

accuracy = (TP + TN) / conf_mat.values.sum()
sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)

print(f"정확도 (accuracy): { accuracy:.4f}")
print(f"민감도 (sensitivity): {sensitivity:.4f}")
print(f"특이도 (specificity): {specificity:.4f}")
```

임계값을 변화시키면서 민감도-특이도가 어떻게 변화하는지를 확인해보자. 임계값을 0.5 외에 0.3, 0.7 등으로 바꾸어 비교한다.

#### [파이썬 코드 3-19]

```
def calc_metrics(prob, actual, threshold):
    pred = (prob > threshold).astype(int)
    cm = pd.crosstab(pred, actual).reindex(index=[0, 1], columns=[0, 1],
                                           fill_value=0)
    TN, FN = cm.loc[0, 0], cm.loc[0, 1]
    FP, TP = cm.loc[1, 0], cm.loc[1, 1]
    return {
        "임계값": threshold,
        "정확도": (TP + TN) / cm.values.sum(),
        "sensitivity": TP / (TP + FN) if (TP + FN) else np.nan,
        "specificity": TN / (TN + FP) if (TN + FP) else np.nan,
    }
thresholds = [0.3, 0.4, 0.5, 0.6, 0.7]
metrics_df = pd.DataFrame([calc_metrics(pred_prob, data["diabetes_int"], t) for t
                           in thresholds])
metrics_df.round(4)
```

임계값에 따른 민감도, 특이도의 변화를 시각화하면 **상충관계(trade-off)**를 한눈에 확인할 수 있다.

**[파이썬 코드 3-20]**

```
thr_seq = np.arange(0.05, 0.96, 0.01)
trade_df = pd.DataFrame([calc_metrics(pred_prob, data["diabetes_int"], t) for t in
    thr_seq])

plt.figure(figsize=(8, 5))
plt.plot(trade_df["임계값"], trade_df["sensitivity"],
    color="#E07B91", linewidth=1.5, label="sensitivity")
plt.plot(trade_df["임계값"], trade_df["specificity"],
    color="#6888BE", linewidth=1.5, label="specificity")
plt.xlabel("임계값")
plt.ylabel("값")
plt.title("임계값에 따른 민감도와 특이도")
plt.legend()
plt.tight_layout()
plt.show()
```

**다항 로지스틱 회귀**

다범주 반응변수의 예시를 위해 여기서는 별도로 iris 데이터를 사용한다.

**[파이썬 코드 3-21]**

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

iris = load_iris()
X_multi_train, X_multi_test, y_multi_train, y_multi_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=2025, stratify=iris.target
)
mlr = LogisticRegression(solver="lbfgs", max_iter=1000)
mlr.fit(X_multi_train, y_multi_train)
print(f"accuracy: {mlr.score(X_multi_test, y_multi_test):.3f}")
```

**A.3.4 이웃에게 물어보는 분류 — 비모수적 접근(KNN)****K-최근접 이웃(KNN) 분류기**

비모수적 분류 방법인 KNN을 적용해보자. KNN은 가까운  $k$ 개의 이웃 중 다수 범주로 분류하는 방법이다. KNN을 적용하기 전에는 변수 표준화가 필수적이다. 변수마다 척도가 다르면 거리 계산이 척도가 큰 변수에 의해 지배되기 때문이다.

`sklearn.preprocessing.StandardScaler`를 이용하여 수치형 설명변수 `age`와 `bmi`를 표준화한다.

**[파이썬 코드 3-22]**

```

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split, StratifiedKFold,
    cross_val_score
from sklearn.pipeline import make_pipeline

X = data[["age", "bmi"]].values
y = data["diabetes_int"].values

```

데이터를 훈련 세트와 검증 세트로 나누어 KNN을 적합한다. seed를 2025로 고정하여 재현성을 확보한다.

### [파이썬 코드 3-23]

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=2025, stratify=y
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
knn_pred = knn.predict(X_test_scaled)

print("Confusion matrix:")
print(pd.crosstab(knn_pred, y_test, rownames=["predicted"], colnames=["actual"]))
print(f"\naccuracy: {(knn_pred == y_test).mean():.4f}")

```

$K$  선택을 위해  $K$  값을 변화시키면서 검증 세트의 오류율을 비교한다.

### [파이썬 코드 3-24]

```

k_vals = [1, 3, 5, 10, 20, 50, 100]
err_rates = []
for k in k_vals:
    pred = KNeighborsClassifier(n_neighbors=k).fit(X_train_scaled,
        y_train).predict(X_test_scaled)
    err_rates.append(np.mean(pred != y_test))

knn_results = pd.DataFrame({"K": k_vals, "error_rate": err_rates})
knn_results

```

결과를 시각화하여  $K$ 에 따른 검증 오류율의 U자 패턴을 확인한다.

### [파이썬 코드 3-25]

```

plt.figure(figsize=(8, 5))

```

```
plt.plot(knn_results["K"], knn_results["error_rate"],
         color="#6888BE", marker="o", linewidth=1.5)
plt.xscale("log")
plt.xlabel("K (로그 척도)")
plt.ylabel("오류율")
plt.title("K 에 따른 검정 오류율")
plt.tight_layout()
plt.show()
```

보다 정확한 선택을 위해 5-fold 교차 검증을 이용한다.

#### [파이썬 코드 3-26]

```
k_vals_cv = [1, 3, 5, 10, 15, 20, 30, 50, 100]
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=2025)
cv_err = []
for k in k_vals_cv:
    model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    cv_err.append(1 - cross_val_score(model, X, y, cv=cv,
                                       scoring="accuracy").mean())

cv_results = pd.DataFrame({"K": k_vals_cv, "cv_error": cv_err})
cv_results
```

교차 검증 오류율이 가장 작은 를 최적의 이웃 수로 선택한다.

#### [파이썬 코드 3-27]

```
best_k = int(cv_results.loc[cv_results["cv_error"].idxmin(), "K"])
print("최적의 K:", best_k)
```

## 변수 표준화

#### [파이썬 코드 3-28]

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

## KNN 실습

#### [파이썬 코드 3-29]

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
```

```
y_pred = knn.predict(X_test_scaled)
print(f"정확도: {accuracy_score(y_test, y_pred):.3f}")
```

## 거리 가중 KNN

### [파이썬 코드 3-30]

```
knn_w = KNeighborsClassifier(n_neighbors=10, weights='distance')
knn_w.fit(X_train_scaled, y_train)
y_pred = knn_w.predict(X_test_scaled)
```

## A.3.5 나무로 분류하기 — 단일 나무에서 앙상블까지

### 분류 나무

분류 나무를 적합하기 위해 `sklearn.tree`의 `DecisionTreeClassifier`를 이용한다. 가지치기(pruning)는 `max_depth`나 `ccp_alpha` 인자로 조절한다.

### [파이썬 코드 3-31]

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

# 범주형 변수를 더미로 변환하여 사용
X_tree = data[["age", "bmi", "hp"]].copy()
X_tree["hp"] = X_tree["hp"].astype(int)
y_tree = data["diabetes_int"].values

tree = DecisionTreeClassifier(max_depth=3, random_state=2025)
tree.fit(X_tree, y_tree)

print(f"훈련 정확도: {tree.score(X_tree, y_tree):.4f}")
```

적합된 분류 나무를 시각화한다.

### [파이썬 코드 3-32]

```
plt.figure(figsize=(12, 6))
plot_tree(tree, feature_names=["age", "bmi", "hp"],
          class_names=["normal", "risk"], filled=True, rounded=True, fontsize=10)
plt.title("분류 나무 (max_depth=3)")
plt.show()
```

가지치기를 위해 비용 복잡도 가지치기(cost-complexity pruning) 경로를 확인한다.

`cost_complexity_pruning_path()`를 이용해 가능한 값들을 추출한다.

### [파이썬 코드 3-33]

```

path = tree.cost_complexity_pruning_path(X_tree, y_tree)
ccp_alphas = path.ccp_alphas

# 각 alpha 에 대해 5-fold CV 를 수행하여 검증 오류율을 측정
from sklearn.model_selection import cross_val_score

# 너무 많은 alpha 후보 중 일정 개수만 선택
ccp_alphas_subset = np.unique(np.round(ccp_alphas, 5))[:15]

cv_scores = []
for alpha in ccp_alphas_subset:
    clf = DecisionTreeClassifier(ccp_alpha=alpha, random_state=2025)
    scores = cross_val_score(clf, X_tree, y_tree, cv=5,
                             scoring="accuracy")
    cv_scores.append(scores.mean())

ccp_results = pd.DataFrame({"ccp_alpha": ccp_alphas_subset,
                           "cv_accuracy": cv_scores}).round(5)
ccp_results

```

교차 검증 정확도가 가장 큰 값으로 가지치기된 나무를 적합한다.

#### [파이썬 코드 3-34]

```

best_alpha = ccp_results.loc[ccp_results["cv_accuracy"].idxmax(), "ccp_alpha"]
print("최적 ccp_alpha:", best_alpha)

pruned_tree = DecisionTreeClassifier(ccp_alpha=best_alpha, random_state=2025)
pruned_tree.fit(X_tree, y_tree)

plt.figure(figsize=(12, 6))
plot_tree(pruned_tree, feature_names=["age", "bmi", "hp"],
          class_names=["normal", "risk"], filled=True, rounded=True, fontsize=10)
plt.title("가지치기된 분류 나무")
plt.show()

```

## 랜덤 포레스트

`sklearn.ensemble` 의 `RandomForestClassifier` 를 이용해 랜덤 포레스트를 적합한다.

주요 인자는 다음과 같다.

- `n_estimators`: 적합할 나무의 개수 ( $B$ )
- `max_features`: 각 분할에서 후보로 고려할 변수의 수 ( $m$ ). 분류 문제에서는 일반적으로  $m \approx \sqrt{p}$ 를 사용 ("sqrt"로 지정)
- `random_state`: 재현성을 위한 시드

#### [파이썬 코드 3-35]

```

from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=500, max_features="sqrt",
                           random_state=2025, oob_score=True)
rf.fit(X_tree, y_tree)

print(f"OOB 정확도: {rf.oob_score_:.4f}")
print(f"훈련 정확도: {rf.score(X_tree, y_tree):.4f}")

```

랜덤 포레스트의 OOB(Out-of-Bag) 점수는 별도의 검증 세트나 교차 검증 없이 테스트 정확도를 추정하는 도구이다. 변수 중요도(variable importance)를 시각화하여 어떤 변수가 예측에 가장 크게 기여하는지 파악한다.

### [파이썬 코드 3-36]

```

importance = pd.DataFrame({
    "feature": ["age", "bmi", "hp"],
    "importance": rf.feature_importances_
}).sort_values("importance", ascending=True)

plt.figure(figsize=(7, 4))
plt.barh(importance["feature"], importance["importance"], color="#6888BE",
         alpha=0.8)
plt.xlabel("지니 지수 평균 감소량")
plt.title("변수 중요도")
plt.tight_layout()
plt.show()

print(importance.sort_values("importance", ascending=False))

```

## 분류 나무 실습

### [파이썬 코드 3-37]

```

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Dedicated three-feature input for the tree and ensemble examples.
X_tree_compare = data[["age", "bmi", "hp"]].copy()
X_tree_compare["hp"] = X_tree_compare["hp"].astype(int)
y_tree_compare = data["diabetes_int"].values
X_train_tree, X_test_tree, y_train_tree, y_test_tree = train_test_split(
    X_tree_compare, y_tree_compare, test_size=0.3,
    random_state=2025, stratify=y_tree_compare
)

tree = DecisionTreeClassifier(max_depth=3, random_state=42)

```

```
tree.fit(X_train_tree, y_train_tree)

# Tree visualization
plt.figure(figsize=(12, 6))
plot_tree(
    tree, feature_names=["age", "bmi", "hp"],
    class_names=["Normal", "Risk"], filled=True
)
plt.show()
```

## 랜덤 포레스트

### [파이썬 코드 3-38]

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=500, max_features="sqrt",
                           random_state=42)
rf.fit(X_train_tree, y_train_tree)
print(f"정확도: {rf.score(X_test_tree, y_test_tree):.3f}")
```

## XGBoost

### [파이썬 코드 3-39]

```
from xgboost import XGBClassifier

xgb = XGBClassifier(n_estimators=500, learning_rate=0.05,
                   max_depth=3, random_state=42)
xgb.fit(X_train_tree, y_train_tree)
print(f"정확도: {xgb.score(X_test_tree, y_test_tree):.3f}")
```

## LightGBM

### [파이썬 코드 3-40]

```
from lightgbm import LGBMClassifier

lgbm = LGBMClassifier(n_estimators=500, learning_rate=0.05,
                     num_leaves=31, random_state=42)
lgbm.fit(X_train_tree, y_train_tree)
print(f"정확도: {lgbm.score(X_test_tree, y_test_tree):.3f}")
```

## CatBoost

### [파이썬 코드 3-41]

```
from catboost import CatBoostClassifier

cat = CatBoostClassifier(iterations=500, learning_rate=0.05,
                        depth=6, verbose=0, random_state=42)
cat.fit(X_train_tree, y_train_tree)
print(f"정확도: {cat.score(X_test_tree, y_test_tree):.3f}")
```

## A.3.6 신경망 기반 분류

### MLP (scikit-learn)

[파이썬 코드 3-42]

```
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(hidden_layer_sizes=(64, 32),
                    activation="relu", solver="adam",
                    max_iter=200, random_state=42)
mlp.fit(X_train_tree, y_train_tree)
print(f"정확도: {mlp.score(X_test_tree, y_test_tree):.3f}")
```

### CNN (PyTorch)

아래 CNN은 이미지 분류용 별도 구조 예시이며 hn19 데이터에는 직접 적용하지 않는다.

[파이썬 코드 3-43]

```
import torch
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 64), nn.ReLU(),
            nn.Linear(64, num_classes), # logits
        )
    def forward(self, x):
        return self.classifier(self.features(x)) # softmax 는 손실함수에서 처리
```

## A.3.7 생성 모형 기반 분류

### LDA 와 QDA

생성 모형 분류기인 LDA(선형판별분석)와 QDA(이차판별분석)를 `sklearn.discriminant_analysis`를 이용해 적합한다.

#### [파이썬 코드 3-44]

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
```

LDA는 모든 클래스가 동일한 공분산 행렬을 공유한다고 가정한다.

#### [파이썬 코드 3-45]

```
lda = LinearDiscriminantAnalysis()
lda.fit(X_tree, y_tree)
lda_pred = lda.predict(X_tree)

print("Confusion matrix:")
print(pd.crosstab(lda_pred, y_tree, rownames=["predicted"], colnames=["actual"]))
print(f"\n 정확도: {(lda_pred == y_tree).mean():.4f}")
```

QDA는 클래스마다 서로 다른 공분산 행렬을 가진다고 가정한다.

#### [파이썬 코드 3-46]

```
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_tree, y_tree)
qda_pred = qda.predict(X_tree)

print("Confusion matrix:")
print(pd.crosstab(qda_pred, y_tree, rownames=["predicted"], colnames=["actual"]))
print(f"\n 정확도: {(qda_pred == y_tree).mean():.4f}")
```

### 나이브 베이즈

나이브 베이즈 분류기는 조건부 독립 가정을 이용하여 결합 분포를 단순화한다.

연속형 변수에는 `GaussianNB`(연속 변수의 조건부 분포를 정규분포로 가정)를 이용한다.

#### [파이썬 코드 3-47]

```
from sklearn.naive_bayes import GaussianNB

nb = GaussianNB()
nb.fit(X_tree, y_tree)
```

```
nb_pred = nb.predict(X_tree)

print("Confusion matrix:")
print(pd.crosstab(nb_pred, y_tree, rownames=["predicted"], colnames=["actual"]))
print(f"\n 정확도: {(nb_pred == y_tree).mean():.4f}")
```

## 여러 분류 방법의 성능 비교

로지스틱 회귀, KNN, 분류 나무, 랜덤 포레스트, LDA, QDA, 나이브 베이즈의 성능을 한 표로 비교한다. 모든 모형은 age, bmi, hp의 동일한 설명변수와 동일한 70/30 훈련·검증 분할을 사용한다. KNN의 이웃 수는 훈련 세트 안에서만 5-겹 층화 교차 검증으로 선택하므로, 검증 세트는 최종 평가에만 사용한다.

### [파이썬 코드 3-48]

```
from sklearn.model_selection import train_test_split, StratifiedKFold,
    cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

# Split first: the holdout set is reserved for the final comparison.
X_all = data[["age", "bmi", "hp"]].copy()
X_all["hp"] = X_all["hp"].astype(int)
y_all = data["diabetes_int"].values
X_tr, X_te, y_tr, y_te = train_test_split(
    X_all, y_all, test_size=0.3, random_state=2025, stratify=y_all
)

# Tune K only within the training data; scaling occurs inside each fold.
k_vals = [1, 3, 5, 10, 15, 20, 30, 50, 100]
inner_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=2025)
cv_error = []
for k in k_vals:
    model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    cv_error.append(1 - cross_val_score(model, X_tr, y_tr, cv=inner_cv,
        scoring="accuracy").mean())
best_k = int(k_vals[int(np.argmax(cv_error))])
print("training-set CV best K:", best_k)
```

공통적으로 정확도, 민감도, 특이도를 계산하는 함수를 정의한다.

### [파이썬 코드 3-49]

```
def metrics(pred, actual):
    cm = pd.crosstab(pd.Series(pred), pd.Series(actual)).reindex(
        index=[0, 1], columns=[0, 1], fill_value=0)
    TN, FN = cm.loc[0, 0], cm.loc[0, 1]
```

```

FP, TP = cm.loc[1, 0], cm.loc[1, 1]
return {
    "정확도": (TP + TN) / cm.values.sum(),
    "sensitivity": TP / (TP + FN) if (TP + FN) else np.nan,
    "specificity": TN / (TN + FP) if (TN + FP) else np.nan,
}

```

7개 모형을 동일한 훈련 데이터로 적합한 후, 검증 데이터에 대한 성능을 계산한다.

#### [파이썬 코드 3-50]

```

# 1) Logistic regression
train_data = X_tr.copy(); train_data["y"] = y_tr
test_data = X_te.copy(); test_data["y"] = y_te
lr_fit = smf.logit("y ~ age + bmi + C(hp)", data=train_data).fit(dis=False)
lr_pred = (lr_fit.predict(test_data) > 0.5).astype(int).values

# 2) KNN: the same three predictors, with a pipeline fitted on training data only
knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=best_k))
knn.fit(X_tr, y_tr)
knn_pred = knn.predict(X_te)

# 3) Classification tree
tree2 = DecisionTreeClassifier(max_depth=4, random_state=2025).fit(X_tr, y_tr)
tree_pred_test = tree2.predict(X_te)

# 4) Random forest
rf2 = RandomForestClassifier(n_estimators=500, max_features="sqrt",
    random_state=2025).fit(X_tr, y_tr)
rf_pred_test = rf2.predict(X_te)

# 5, 6) LDA and QDA
lda_pred_test = LinearDiscriminantAnalysis().fit(X_tr, y_tr).predict(X_te)
qda_pred_test = QuadraticDiscriminantAnalysis().fit(X_tr, y_tr).predict(X_te)

# 7) Naive Bayes
nb_pred_test = GaussianNB().fit(X_tr, y_tr).predict(X_te)

```

각 모형의 성능 지표를 한 표로 정리한다.

#### [파이썬 코드 3-51]

```

result_table = pd.DataFrame({
    "Logistic": metrics(lr_pred, y_te),
    "KNN": metrics(knn_pred, y_te),
    "Tree": metrics(tree_pred_test, y_te),
    "RandomForest": metrics(rf_pred_test, y_te),
    "LDA": metrics(lda_pred_test, y_te),
    "QDA": metrics(qda_pred_test, y_te),
    "NaiveBayes": metrics(nb_pred_test, y_te),
})

```

```
}).T.round(4)
result_table
```

각 모형의 정확도를 막대그림으로 시각화하여 비교한다.

#### [파이썬 코드 3-52]

```
plt.figure(figsize=(9, 5))
bars = plt.bar(result_table.index, result_table["정확도"], alpha=0.8,
               color=["#6888BE", "#E07B91", "#88B04B", "#F5B041",
                     "#5DADE2", "#AF7AC5", "#EC7063"])

plt.ylim(0, 1)
plt.ylabel("정확도")
plt.title("분류기별 검정 정확도")
plt.xticks(rotation=30, ha="right")

for bar, val in zip(bars, result_table["정확도"]):
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01,
             f"{val:.3f}", ha="center", fontsize=10)

plt.tight_layout()
plt.show()
```

No Free Lunch 정리 — 모든 상황에서 최고인 분류기는 존재하지 않는다. 데이터의 특성과 분석의 목적(해석 가능성 vs 예측 정확도)에 맞게 적절한 도구를 선택해야 한다.

## LDA 와 QDA

#### [파이썬 코드 3-53]

```
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis)

lda = LinearDiscriminantAnalysis().fit(X_train, y_train)
qda = QuadraticDiscriminantAnalysis().fit(X_train, y_train)
print(f"LDA: {lda.score(X_test, y_test):.3f}")
print(f"QDA: {qda.score(X_test, y_test):.3f}")
```

## 나이브 베이즈

#### [파이썬 코드 3-54]

```
from sklearn.naive_bayes import GaussianNB

import numpy as np

# 나이브 베이즈
nb = GaussianNB().fit(X_train, y_train)
```

```
print(f"NB 정확도: {nb.score(X_test, y_test):.3f}")
```

## A.3.8 실습 2: 고혈압 위험군 분류

### 분석을 위해 필요한 라이브러리 및 데이터 불러오기

#### [파이썬 코드 3-55]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import statsmodels.formula.api as smf
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split, KFold, cross_val_score

data = pd.read_csv("데이터/hn19.csv", encoding="utf-8")
data.head(3)
```

#### [파이썬 코드 3-56]

```
data = data[["age", "HE_BMI", "HE_HP",
    "HE_DM_HbA1c"]].dropna().reset_index(drop=True)
print("결측값 제거 후 데이터의 행 수:", len(data))
```

#### [파이썬 코드 3-57]

```
data["hp"] = (data["HE_HP"] >= 2).astype(int).astype("category")
data["dm"] = (data["HE_DM_HbA1c"] >= 2).astype(int).astype("category")
data["bmi"] = data["HE_BMI"]

data = data[["age", "bmi", "dm", "hp"]]
data.head(3)
```

#### [파이썬 코드 3-58]

```
print(data["hp"].value_counts())
print()
print(data["hp"].value_counts(normalize=True).round(4))
```

## 변수 간의 상관관계 및 분포 파악하기

### [파이썬 코드 3-59]

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.boxplot(x="hp", y="age", data=data, ax=axes[0],
            palette=["#6888BE", "#E07B91"], hue=None)
axes[0].set_title("혈압 상태별 나이 분포")

sns.boxplot(x="hp", y="bmi", data=data, ax=axes[1],
            palette=["#6888BE", "#E07B91"], hue=None)
axes[1].set_title("혈압 상태별 BMI 분포")

plt.tight_layout()
plt.show()
```

### [파이썬 코드 3-60]

```
pd.crosstab(data["dm"], data["hp"], rownames=["dm"], colnames=["hp"])
```

### [파이썬 코드 3-61]

```
ct = pd.crosstab(data["dm"], data["hp"], normalize="index")
ax = ct.plot(kind="bar", stacked=True, figsize=(7, 5),
            color=["#6888BE", "#E07B91"], alpha=0.8)
ax.set_ylabel("비율")
ax.set_title("당뇨 상태별 고혈압 비율")
ax.legend(title="hp")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

## 단순 로지스틱 회귀모형 적합

### [파이썬 코드 3-62]

```
data["hp_int"] = data["hp"].astype(int)

simple_fit = smf.logit("hp_int ~ bmi", data=data).fit(dispc=False)
print(simple_fit.summary())
```

### [파이썬 코드 3-63]

```
print("오즈비:")
print(np.exp(simple_fit.params).round(4))
```

## 다중 로지스틱 회귀모형 적합

**[파이썬 코드 3-64]**

```
multi_fit = smf.logit("hp_int ~ age + bmi + C(dm)", data=data).fit(dis=False)
print(multi_fit.summary())
```

**[파이썬 코드 3-65]**

```
coef_table = pd.DataFrame({
    "Estimate": multi_fit.params,
    "StdError": multi_fit.bse,
    "z_value": multi_fit.tvalues,
    "p_value": multi_fit.pvalues,
    "OddsRatio": np.exp(multi_fit.params)
}).round(4)
coef_table
```

**[파이썬 코드 3-66]**

```
new_obs = pd.DataFrame({
    "age": [30, 30, 60, 60],
    "bmi": [22, 27, 22, 27],
    "dm": pd.Categorical([0, 1, 0, 1], categories=[0, 1])
})
new_obs["predicted_prob"] = multi_fit.predict(new_obs).round(3)
new_obs
```

**분류 성능의 평가****[파이썬 코드 3-67]**

```
pred_prob = multi_fit.predict(data)
pred_class = (pred_prob > 0.5).astype(int)

conf_mat = pd.crosstab(pred_class, data["hp_int"],
                        rownames=["predicted"], colnames=["actual"])
conf_mat
```

**[파이썬 코드 3-68]**

```
TN = conf_mat.loc[0, 0]; FN = conf_mat.loc[0, 1]
FP = conf_mat.loc[1, 0]; TP = conf_mat.loc[1, 1]

print(f"정확도: { (TP+TN)/conf_mat.values.sum():.4f}")
print(f"민감도: { TP/(TP+FN):.4f}")
print(f"특이도: { TN/(TN+FP):.4f}")
```

**[파이썬 코드 3-69]**

```
def calc_metrics(prob, actual, threshold):
```

```

pred = (prob > threshold).astype(int)
cm = pd.crosstab(pred, actual).reindex(index=[0, 1], columns=[0, 1],
    fill_value=0)
TN, FN = cm.loc[0, 0], cm.loc[0, 1]
FP, TP = cm.loc[1, 0], cm.loc[1, 1]
return {
    "임계값": threshold,
    "정확도": (TP + TN) / cm.values.sum(),
    "sensitivity": TP / (TP + FN) if (TP + FN) else np.nan,
    "specificity": TN / (TN + FP) if (TN + FP) else np.nan,
}

thr_seq = np.arange(0.05, 0.96, 0.01)
trade_df = pd.DataFrame([calc_metrics(pred_prob, data["hp_int"], t) for t in
    thr_seq])

plt.figure(figsize=(8, 5))
plt.plot(trade_df["임계값"], trade_df["sensitivity"],
    color="#E07B91", linewidth=1.5, label="sensitivity")
plt.plot(trade_df["임계값"], trade_df["specificity"],
    color="#6888BE", linewidth=1.5, label="specificity")
plt.xlabel("임계값"); plt.ylabel("값")
plt.title("임계값에 따른 민감도와 특이도")
plt.legend(); plt.tight_layout(); plt.show()

```

## K-최근접 이웃(KNN) 분류기

### [파이썬 코드 3-70]

```

X = data[["age", "bmi"]].values
y = data["hp_int"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=2025, stratify=y
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

### [파이썬 코드 3-71]

```

k_vals_cv = [1, 3, 5, 10, 15, 20, 30, 50, 100]
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=2025)
cv_err = []
for k in k_vals_cv:
    model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    cv_err.append(1 - cross_val_score(model, X, y, cv=cv,
        scoring="accuracy").mean())

```

```
cv_results = pd.DataFrame({"K": k_vals_cv, "cv_error": cv_err})
cv_results
```

**[파이썬 코드 3-72]**

```
best_k = int(cv_results.loc[cv_results["cv_error"].idxmin(), "K"])
print("최적의 K:", best_k)
```

**분류 나무****[파이썬 코드 3-73]**

```
X_tree = data[["age", "bmi", "dm"]].copy()
X_tree["dm"] = X_tree["dm"].astype(int)
y_tree = data["hp_int"].values

tree = DecisionTreeClassifier(max_depth=3, random_state=2025)
tree.fit(X_tree, y_tree)
print(f"훈련 정확도: {tree.score(X_tree, y_tree):.4f}")
```

**[파이썬 코드 3-74]**

```
plt.figure(figsize=(12, 6))
plot_tree(tree, feature_names=["age", "bmi", "dm"],
          class_names=["normal", "risk"], filled=True, rounded=True, fontsize=10)
plt.title("분류 나무 (max_depth=3)")
plt.show()
```

**[파이썬 코드 3-75]**

```
path = tree.cost_complexity_pruning_path(X_tree, y_tree)
ccp_alphas_subset = np.unique(np.round(path.ccp_alphas, 5))[:15]

cv_scores = []
for alpha in ccp_alphas_subset:
    clf = DecisionTreeClassifier(ccp_alpha=alpha, random_state=2025)
    cv_scores.append(cross_val_score(clf, X_tree, y_tree, cv=5,
                                     scoring="accuracy").mean())

ccp_results = pd.DataFrame({"ccp_alpha": ccp_alphas_subset,
                           "cv_accuracy": cv_scores}).round(5)

best_alpha = ccp_results.loc[ccp_results["cv_accuracy"].idxmax(), "ccp_alpha"]
print("최적 ccp_alpha:", best_alpha)

pruned_tree = DecisionTreeClassifier(ccp_alpha=best_alpha, random_state=2025)
pruned_tree.fit(X_tree, y_tree)

plt.figure(figsize=(12, 6))
```

```
plot_tree(pruned_tree, feature_names=["age", "bmi", "dm"],
          class_names=["normal", "risk"], filled=True, rounded=True, fontsize=10)
plt.title("가지치기된 분류 나무")
plt.show()
```

## 랜덤 포레스트

### [파이썬 코드 3-76]

```
rf = RandomForestClassifier(n_estimators=500, max_features="sqrt",
                           random_state=2025, oob_score=True)
rf.fit(X_tree, y_tree)
print(f"OOB 정확도: {rf.oob_score_:.4f}")
```

### [파이썬 코드 3-77]

```
importance = pd.DataFrame({
    "feature": ["age", "bmi", "dm"],
    "importance": rf.feature_importances_
}).sort_values("importance", ascending=True)

plt.figure(figsize=(7, 4))
plt.barh(importance["feature"], importance["importance"], color="#6888BE",
         alpha=0.8)
plt.xlabel("지니 지수 평균 감소량")
plt.title("변수 중요도")
plt.tight_layout()
plt.show()
```

## LDA와 QDA

### [파이썬 코드 3-78]

```
lda = LinearDiscriminantAnalysis().fit(X_tree, y_tree)
lda_pred = lda.predict(X_tree)
print("Confusion matrix:")
print(pd.crosstab(lda_pred, y_tree, rownames=["predicted"], colnames=["actual"]))
print(f"\n 정확도: {(lda_pred == y_tree).mean():.4f}")
```

### [파이썬 코드 3-79]

```
qda = QuadraticDiscriminantAnalysis().fit(X_tree, y_tree)
qda_pred = qda.predict(X_tree)
print("Confusion matrix:")
print(pd.crosstab(qda_pred, y_tree, rownames=["predicted"], colnames=["actual"]))
print(f"\n 정확도: {(qda_pred == y_tree).mean():.4f}")
```

## 나이브 베이즈

**[파이썬 코드 3-80]**

```
nb = GaussianNB().fit(X_tree, y_tree)
nb_pred = nb.predict(X_tree)
print("Confusion matrix:")
print(pd.crosstab(nb_pred, y_tree, rownames=["predicted"], colnames=["actual"]))
print(f"\n 정확도: {(nb_pred == y_tree).mean():.4f}")
```

**여러 분류 방법의 성능 비교**

모든 비교 모형은 age, bmi, dm의 동일한 설명변수를 사용한다. KNN의 이웃 수는 훈련 세트 내부의 5-겹 층화 교차 검증으로 선택한 뒤 검증 세트에서 평가한다.

**[파이썬 코드 3-81]**

```
from sklearn.model_selection import train_test_split, StratifiedKFold,
    cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

X_all = data[["age", "bmi", "dm"]].copy()
X_all["dm"] = X_all["dm"].astype(int)
y_all = data["hp_int"].values
X_tr, X_te, y_tr, y_te = train_test_split(
    X_all, y_all, test_size=0.3, random_state=2025, stratify=y_all
)

# Select K only with training-set folds, including fold-specific scaling.
k_vals = [1, 3, 5, 10, 15, 20, 30, 50, 100]
inner_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=2025)
cv_error = []
for k in k_vals:
    model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    cv_error.append(1 - cross_val_score(model, X_tr, y_tr, cv=inner_cv,
        scoring="accuracy").mean())
best_k = int(k_vals[int(np.argmin(cv_error))])

def metrics(pred, actual):
    cm = pd.crosstab(pd.Series(pred), pd.Series(actual)).reindex(
        index=[0, 1], columns=[0, 1], fill_value=0)
    TN, FN = cm.loc[0, 0], cm.loc[0, 1]
    FP, TP = cm.loc[1, 0], cm.loc[1, 1]
    return {
        "정확도": (TP + TN) / cm.values.sum(),
        "sensitivity": TP / (TP + FN) if (TP + FN) else np.nan,
        "specificity": TN / (TN + FP) if (TN + FP) else np.nan,
    }
```

```
# Logistic regression
train_data = X_tr.copy(); train_data["y"] = y_tr
test_data = X_te.copy(); test_data["y"] = y_te
lr_fit = smf.logit("y ~ age + bmi + C(dm)", data=train_data).fit(dis= False)
lr_pred = (lr_fit.predict(test_data) > 0.5).astype(int).values

# KNN uses the same predictors as every other comparison model.
knn = make_pipeline(StandardScaler(),
    KNeighborsClassifier(n_neighbors=best_k)).fit(X_tr, y_tr)
knn_pred = knn.predict(X_te)

tree2 = DecisionTreeClassifier(max_depth=4, random_state=2025).fit(X_tr, y_tr)
tree_pred_test = tree2.predict(X_te)
rf2 = RandomForestClassifier(n_estimators=500, max_features="sqrt",
    random_state=2025).fit(X_tr, y_tr)
rf_pred_test = rf2.predict(X_te)
lda_pred_test = LinearDiscriminantAnalysis().fit(X_tr, y_tr).predict(X_te)
qda_pred_test = QuadraticDiscriminantAnalysis().fit(X_tr, y_tr).predict(X_te)
nb_pred_test = GaussianNB().fit(X_tr, y_tr).predict(X_te)
```

**[파이썬 코드 3-82]**

```
result_table = pd.DataFrame({
    "Logistic":    metrics(lr_pred, y_te),
    "KNN":         metrics(knn_pred, y_te),
    "Tree":        metrics(tree_pred_test, y_te),
    "RandomForest": metrics(rf_pred_test, y_te),
    "LDA":         metrics(lda_pred_test, y_te),
    "QDA":         metrics(qda_pred_test, y_te),
    "NaiveBayes":  metrics(nb_pred_test, y_te),
}).T.round(4)
result_table
```

**[파이썬 코드 3-83]**

```
plt.figure(figsize=(9, 5))
bars = plt.bar(result_table.index, result_table["정 확도"], alpha=0.8,
    color=["#6888BE", "#E07B91", "#88B04B", "#F5B041",
    "#5DADE2", "#AF7AC5", "#EC7063"])

plt.ylim(0, 1)
plt.ylabel("정 확도")
plt.title("분류기별 검정 정 확도")
plt.xticks(rotation=30, ha="right")

for bar, val in zip(bars, result_table["정 확도"]):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
        f"{val:.3f}", ha="center", fontsize=10)

plt.tight_layout()
```

```
plt.show()
```

## A.4 4 부 실습

### A.4.1 확률과 확률분포

앞서 우리는 실제 세계의 데이터를 설명하기 위해 이론 세계의 도구인 '확률분포'를 도입했다. 이제 파이썬(Python)을 이용해 이 추상적인 개념들을 구체적인 숫자로 확인해 보자. 이 실습에서는 가장 대표적인 이산형 분포인 **이항분포**와 연속형 분포인 **정규분포**를 다룬다.

#### 이항분포: 동전 던지기의 확률 계산

성공 확률이 50%( $p = 0.5$ )인 공정한 동전을 10번 던지는 실험( $n = 10$ )을 생각해 보자. 이때 앞면이 나오는 횟수를 확률변수  $X$ 라고 할 때, 우리는 `scipy.stats` 라이브러리의 `binom` 모듈을 이용해 이론적인 확률을 계산할 수 있다.

##### [파이썬 코드 4-1]

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import binom, norm

# 설정: 시행 횟수 n=10, 성공 확률 p=0.5
n = 10
p = 0.5

# 1) P(X = 2): 앞면이 정확히 2 회 나올 확률 (확률질량함수 pmf 사용)
prob_exact = binom.pmf(2, n=n, p=p)
print(f"P(X = 2) = {prob_exact:.4f}")

# 2) P(X <= 3): 앞면이 3 회 이하로 나올 확률 (누적분포함수 cdf 사용)
prob_less_equal = binom.cdf(3, n=n, p=p)
print(f"P(X <= 3) = {prob_less_equal:.4f}")

# 3) P(X > 7): 앞면이 7 회 초과로 나올 확률 (1 - cdf 사용)
prob_over = 1 - binom.cdf(7, n=n, p=p)
print(f"P(X > 7) = {prob_over:.4f}")
```

결과는 다음과 같다.

```
P(X = 2) = 0.0439
P(X <= 3) = 0.1719
P(X > 7) = 0.0547
```

## 분석 결과

- $P(X = 2)$ : 동전을 10 번 던져 앞면이 2 번 나올 확률은 약 **0.0439(4.4%)**로 꽤 드문 일이다.
- $P(X \leq 3)$ : 앞면이 3 번 이하로 나올 확률은 약 **0.1719(17.2%)**다.
- $P(X > 7)$ : 반대로 앞면이 8 번, 9 번, 10 번 나올 확률, 즉 7 번을 초과할 확률은 **0.0547(5.5%)**로 계산된다.

## 정규분포: 이론과 현실의 대조

다음으로 통계학의 꽃인 정규분포를 살펴보자. 평균이 0이고 표준편차가 1인 표준정규분포( $Z \sim N(0,1)$ )에서 데이터를 무작위로 추출(Simulation)해 보자. 이론적으로 완벽한 곡선(PDF)과 무작위로 뽑힌 데이터의 분포(히스토그램)가 어떻게 다른지 비교하는 것이 핵심이다.

### [파이썬 코드 4-2]

```
# 설정: 평균(mu)=0, 표준편차(sigma)=1
mu = 0
sigma = 1
size = 1000 # 표본 크기

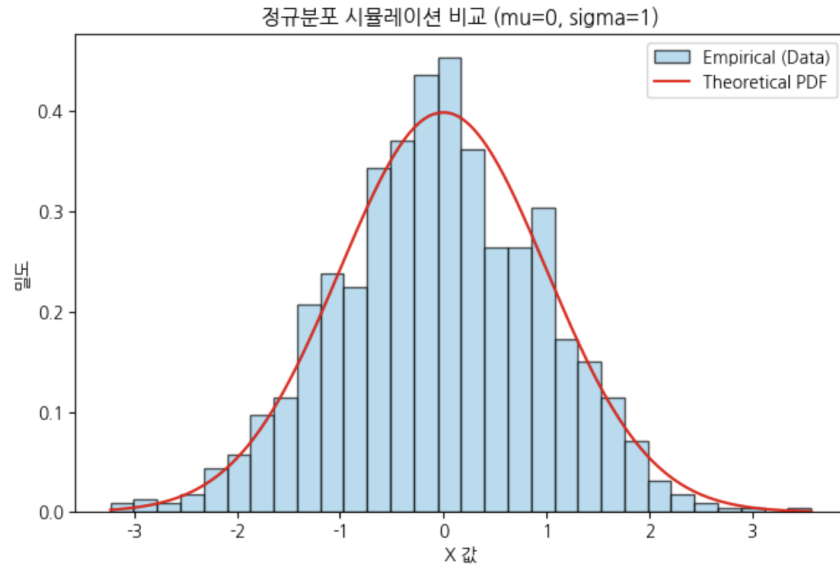
# 데이터 생성 (시뮬레이션: 현실의 데이터)
np.random.seed(123)
simulated_data = np.random.normal(loc=mu, scale=sigma, size=size)

# 시각화: 경험적 분포(히스토그램) vs 이론적 분포(선 그래프)
plt.figure(figsize=(8, 5))

# 히스토그램 (경험적 확률)
plt.hist(simulated_data, bins=30, density=True, color='skyblue',
         edgecolor='black', alpha=0.7, label='Empirical (Data)')

# 이론적 밀도곡선
x = np.linspace(min(simulated_data), max(simulated_data), 1000)
pdf = norm.pdf(x, loc=mu, scale=sigma)
plt.plot(x, pdf, 'r-', label='Theoretical PDF')

plt.xlabel('X 값')
plt.ylabel('밀도')
plt.title(f'정규분포 시뮬레이션 비교 (mu={mu}, sigma={sigma})')
plt.legend()
plt.show()
```



[그림 A-4-1] 표준정규분포의 경험적 분포(히스토그램)와 이론적 분포(선 그래프)

## 결과 해석

[그림 A-4-1]에서 붉은색 실선은 수학적으로 정의된 **이론적 분포**다. 반면 파란색 막대는 우리가 컴퓨터를 통해 무작위로 뽑은 1,000개의 **데이터(경험적 분포)**다. 데이터의 수가 많아질수록 막대 그래프의 모양은 붉은 실선에 점점 더 가까워진다. 이것이 통계적 추론이 가능한 이유다. 우리는 불확실한 데이터(파란 막대)를 보고, 그 배후에 있는 이론적 진실(붉은 선)을 추측해 낸다.

## 분포의 모양을 결정하는 두 가지 열쇠: $\mu$ 와 $\sigma$

정규분포의 모양은 평균( $\mu$ )과 표준편차( $\sigma$ )에 의해 결정된다. 이 파라미터들이 변할 때 분포가 어떻게 바뀌는지 확인해 보자. 다음 코드는  $\mu = 0, 10$ ,  $\sigma = 1, 4$  각각 경우의 정규분포로부터 표집한 1000개의 표본을 보여준다.

### [파이썬 코드 4-3]

```
# 시뮬레이션 설정
mu_list = [0, 10]
sigma_list = [1, 4]
sim_size = 1000

# 서브플롯 설정
fig, axes = plt.subplots(len(mu_list), len(sigma_list), figsize=(12, 8),
                          constrained_layout=True)

# 시각화 반복
for i, mu_val in enumerate(mu_list):
    for j, sigma_val in enumerate(sigma_list):
```

```

ax = axes[i, j]

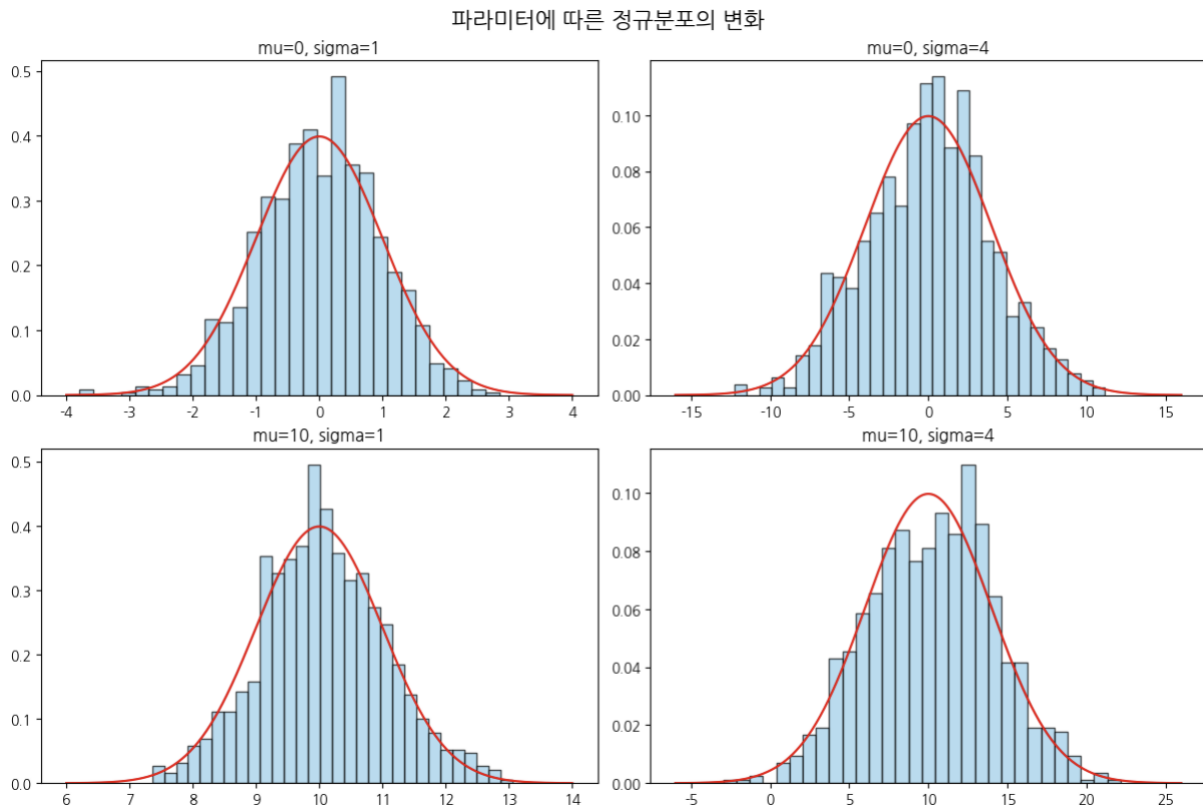
# 데이터 생성 및 히스토그램
data = np.random.normal(loc=mu_val, scale=sigma_val, size=sim_size)
ax.hist(data, bins=30, density=True, color='skyblue', edgecolor='black',
        alpha=0.7)

# 이론적 곡선
x_rng = np.linspace(mu_val - 4*sigma_val, mu_val + 4*sigma_val, 500)
ax.plot(x_rng, norm.pdf(x_rng, mu_val, sigma_val), 'r-')

ax.set_title(f"mu={mu_val}, sigma={sigma_val}")

plt.suptitle("파라미터에 따른 정규분포의 변화", fontsize=16)
plt.show()

```



[그림 A-4-2] 파라미터에 따른 정규분포의 변화

[그림 A-4-2]를 보면,  $\mu$ 가 변하면 분포의 중심이 이동하고,  $\sigma$ 가 커지면 분포가 옆으로 퍼지며 높이가 낮아지는 것을 확인할 수 있다. (변화가 없는 것처럼 보인다면, X축의 위치를 확인해 보자.)

### 정규분포의 확률 계산: 중요한 숫자들

추론에서 자주 사용하게 될 중요한 확률 값들을 계산해 보자. 특히 **1.96**이라는 숫자에 주목하라.

**[파이썬 코드 4-4]**

```

mu = 0
sigma = 1

# 1) P(X <= 1.64555): 상위 5% 지점 (단측 검정에서 주로 사용)
prob1 = norm.cdf(1.64555, loc=mu, scale=sigma)
print(f"P(X <= 1.64555) = {prob1:.4f}")

# 2) P(-1 < X < 1): 평균에서 1 표준편차 이내에 있을 확률
prob2 = norm.cdf(1, loc=mu, scale=sigma) - norm.cdf(-1, loc=mu, scale=sigma)
print(f"P(-1 < X < 1) = {prob2:.4f}")

# 3) P(|X| >= 1.96): 양쪽 꼬리 확률의 합 (95% 신뢰구간의 근거)
prob3 = norm.cdf(-1.96, loc=mu, scale=sigma) + (1 - norm.cdf(1.96, loc=mu,
    scale=sigma))
print(f"P(|X| >= 1.96) = {prob3:.4f}")

P(X <= 1.64555) = 0.9500
P(-1 < X < 1) = 0.6827
P(|X| >= 1.96) = 0.0500

```

**핵심 결과**

$P(|X| \geq 1.96) \approx 0.05$ : 표준정규분포에서 1.96보다 크거나 -1.96보다 작은 값은 전체의 5%에 불과하다. 즉, 데이터의 **95%**는 -1.96과 1.96 사이에 존재한다. 이것이 우리가 95% 신뢰구간을 구할 때 '표준오차의 약 2배(정확히는 1.96배)'를 사용하는 수학적 근거다.

**분위수(Quantile) 구하기**

마지막으로, 확률이 주어졌을 때 그에 해당하는  $x$ 값을 찾아보자. 이를 분위수라고 한다.

**[파이썬 코드 4-5]**

```

# 25%(Q1), 50%(중앙값), 75%(Q3) 분위수 구하기
quantiles = [0.25, 0.5, 0.75, 0.975]
q_values = norm.ppf(quantiles, loc=mu, scale=sigma)

for q, val in zip(quantiles, q_values):
    print(f"{int(q*100)}% 이론적 분위수: X = {val:.4f}")

25% 이론적 분위수: X = -0.6745
50% 이론적 분위수: X = 0.0000
75% 이론적 분위수: X = 0.6745

```

결과를 보면 50% 분위수는 평균인 0이 나오며, 25%와 75% 분위수는 0을 중심으로 대칭인 값을 가진다는 것을 확인할 수 있다.

## A.4.2 모집단과 표집

통계적 추론의 핵심은 '부분(표본)'을 통해 '전체(모집단)'를 짐작하는 것이다. 이 개념을 명확히 이해하기 위해, 우리는 2023년 보스턴 마라톤 대회 데이터를 활용한 시뮬레이션을 진행해 보자.

여기서 우리는 **전체 참가자 데이터를 이미 가지고 있다고** 가정한다. 즉, 우리는 신(God)의 관점에서 모집단의 모든 정보를 알고 있다. 이 상태에서 일부를 무작위로 뽑았을 때, 과연 그 표본이 모집단을 얼마나 잘 닮아 있는지 확인해 보는 것이 이번 실습의 목표다.

### 데이터 준비: 모집단 정의하기

먼저 데이터를 불러온다. 분석의 대상을 명확히 하기 위해, 전체 참가자 중 '**40-44세 여성**' 그룹만 추출하여 이를 우리의 **관심 모집단(Population)**으로 정의하자.

#### [파이썬 코드 4-6]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 데이터 불러오기
boston = pd.read_csv("데이터/boston_marathon_2023.csv")

# 분석에 필요한 변수의 결측값만 제거
boston = boston.dropna(
    subset=["age_group", "gender", "finish_net_minutes"]
)

# 1. Data Filtering: 40-44 세 여성 데이터 추출 (모집단 정의)
boston40 = boston[(boston["age_group"] == "40-44") & (boston["gender"] == "W")]

# 데이터 확인
print(f"모집단 크기: {len(boston40)}명")
print(boston40[["age_group", "gender", "finish_net_minutes"]].head(5))
```

모집단 크기: 1878 명

|       | age_group | gender | finish_net_minutes |
|-------|-----------|--------|--------------------|
| 20059 | 40-44     | W      | 145.800000         |
| 20060 | 40-44     | W      | 154.666667         |
| 20061 | 40-44     | W      | 159.266667         |

|       |       |   |            |
|-------|-------|---|------------|
| 20062 | 40-44 | W | 165.366667 |
| 20063 | 40-44 | W | 165.683333 |

이 코드를 실행하면 약 1,878명의 데이터가 추출된다. 이 1,878명이 우리가 알고자 하는 전체 모집단이다.

## 모집단의 분포 확인

이제 모집단 전체의 완주 시간 기록이 어떻게 분포되어 있는지 히스토그램으로 확인해 보자. 이것은 우리가 현실에서는 결코 알 수 없는 '진실(Truth)'의 모습이다.

### [파이썬 코드 4-7]

```
# 2. 모집단의 히스토그램 그리기 (40-44 세 여성 전체)
plt.figure(figsize=(10, 6))
sns.histplot(boston40['finish_net_minutes'], binwidth=5, color='lightcoral',
              edgecolor='black', alpha=0.8, kde=False)
plt.title('2023 보스턴 마라톤: 40-44 세 여성 완주 시간 분포 (모집단)', fontsize=15)
plt.xlabel('완주 시간 (분)', fontsize=12)
plt.ylabel('빈도', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

전체 분포는 평균(약 230분)을 중심으로 종 모양과 유사하게 퍼져 있음을 알 수 있다.

## 표집(Sampling)

이제 현실의 상황을 가정해 보자. 우리는 시간과 비용의 제약으로 전체 1,878명을 다 조사할 수 없다. 대신 무작위로 **100명(n=100)**을 뽑아 그들의 기록만 확인할 수 있다고 하자. 이를 표집(Sampling)이라고 한다.

결과의 재현성을 위해 `random_state` (또는 `seed`)를 설정하여 추출한다.

### [파이썬 코드 4-8]

```
# 3. 표본 추출: 재현성을 위해 seed 설정 후 100 개 표본 추출
np.random.seed(123)
sample_data = boston40.sample(n=100, random_state=123) # random_state 는 seed 역할

print(f"표본 평균: {sample_data['finish_net_minutes'].mean():.2f}분")
```

표본 평균: 233.02 분

객체 `sample_data`에 우리의 표본인 100개의 관측값이 저장되어 있다.

## 표본의 분포: 부분은 전체를 닮았는가?

우리가 손에 쥐게 된 100명의 데이터, 즉 **표본(Sample)**의 분포를 그려보자. 과연 이 작은 조각이 거대한 모집단의 모습을 얼마나 잘 반영하고 있을까?

### [파이썬 코드 4-9]

```
# 4. 표본의 상대빈도 분포 그리기
plt.figure(figsize=(10, 6))
# 히스토그램 (상대빈도, density)
sns.histplot(sample_data['finish_net_minutes'], binwidth=5, stat='density',
             color='steelblue', edgecolor='black', alpha=0.8)
# 밀도 곡선 추가 (KDE)
sns.kdeplot(sample_data['finish_net_minutes'], color='red', linewidth=2)

plt.title('표본의 상대빈도 분포 (100 개 표본)', fontsize=15)
plt.xlabel('완주 시간 (분)', fontsize=12)
plt.ylabel('밀도 (상대빈도)', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

## 분석 결과

표본의 분포는 모집단과 비슷해 보이지만 완벽히 같지는 않다. 울퉁불퉁한 정도나 봉우리의 높이가 미세하게 다르다. 이것이 바로 **표본의 변동성(Sampling Variability)**이다. 만약 우리가 `random_state`를 바꾸어 다시 100명을 뽑는다면, 표본의 평균과 분포 모양은 또 달라질 것이다.

## A.4.3 표본크기와 표본의 변동성

앞선 실습에서 우리는 100명의 표본을 뽑아 평균을 계산했다. 하지만 만약 우리가 타임머신을 타고 시간을 되돌려 다시 100명을 뽑는다면, 과연 똑같은 평균이 나올까? 당연히 아닐 것이다. 구성원이 달라지므로 평균값도 달라질 수밖에 없다.

이처럼 표집을 무한히 반복했을 때, 계산된 평균들이 어떻게 퍼져 있는지를 보여주는 것이 **표본분포(Sampling Distribution)**다. 이번 실습에서는 파이썬 시뮬레이션을 통해 표본의 크기( $n$ )가 커질수록 이 분포가 어떻게 변하는지, 즉 추정의 정확도가 어떻게 달라지는지 눈으로 확인해 본다.

## 기준점 확인: 모집단의 참값

시뮬레이션의 결과를 평가하기 위해, 먼저 우리가 이미 알고 있는 모집단(40-44세 여성 참가자 전체)의 진짜 평균 기록( $\mu$ )을 계산해 둔다. 현실에서는 알 수 없는 '신의 영역'에 있는 값이지만, 시뮬레이션에서는 이 값이 기준점이 된다.

**[파이썬 코드 4-10]**

```
# 모집단 평균 계산 (참값)
pop_mean = boston40['finish_net_minutes'].mean()
print(f"모집단 평균: {pop_mean:.2f}분")
```

```
모집단 평균: 230.74 분
```

모집단의 평균은 약 230.74분이다. 이제 우리가 뽑는 표본들의 평균이 이 값 주변에 어떻게 분포하는지 살펴볼 것이다.

**"오늘을 다시 사는" 함수 정의**

현실에서는 데이터를 한 번만 수집할 수 있지만, 컴퓨터 세상에서는 얼마든지 가상의 표집을 반복할 수 있다. 다음 함수는 모집단에서 지정된 크기(**sample\_size**)만큼 표본을 뽑아 평균을 구하는 작업을 수천 번(**num\_simulations**) 반복하여 리스트로 반환한다.

**[파이썬 코드 4-11]**

```
def simulate_sampling_distribution(population_df, sample_size, num_simulations):
    sample_means = []
    for _ in range(num_simulations):
        # 모집단에서 무작위 표본 추출 (복원 추출)
        sample = population_df['finish_net_minutes'].sample(n=sample_size,
                                                             replace=True, random_state=None)
        # 표본 평균 계산 및 저장
        sample_mean = sample.mean()
        sample_means.append(sample_mean)
    return sample_means
```

**시뮬레이션:  $n = 100$  vs  $n = 400$** 

이제 두 가지 시나리오를 비교해 보자.

1.  $n = 100$ : 100 명씩 뽑는 조사를 5,000 번 반복한다.
2.  $n = 400$ : 400 명씩 뽑는 조사를 5,000 번 반복한다.

데이터 수가 4배 늘어났을 때, 평균들의 분포가 어떻게 달라지는지 확인하는 것이 핵심이다.

**[파이썬 코드 4-12]**

```
num_simulations = 5000
np.random.seed(123) # 출판용 그림의 재현성 확보

# 시뮬레이션 실행
sample_means_100 = simulate_sampling_distribution(boston40, sample_size=100,
                                                  num_simulations=num_simulations)
```

```
sample_means_400 = simulate_sampling_distribution(boston40, sample_size=400,
num_simulations=num_simulations)
```

## 시각화 및 결과 해석: 대수의 법칙 확인

이제 두 시나리오의 결과를 히스토그램과 밀도 곡선으로 그려 비교해 보자. 붉은색 점선은 모집단의 진짜 평균이다.

### [파이썬 코드 4-13]

```
plt.figure(figsize=(8, 12))

# 전체 X 축 범위 설정
min_x = min(min(sample_means_100), min(sample_means_400))
max_x = max(max(sample_means_100), max(sample_means_400))

# Plot 1: 표본 크기 n=100 일 때
plt.subplot(2, 1, 1) # 2 행 1 열의 첫 번째 플롯
sns.histplot(sample_means_100, bins=30, stat='density', color='skyblue',
edgecolor='black', alpha=0.7, label='n=100')
sns.kdeplot(sample_means_100, color='blue', linewidth=2)
plt.axvline(pop_mean, color='red', linestyle='--', label='Population Mean')
plt.title('Sampling Distribution of Means (n=100)', fontsize=14)
plt.xlabel('Sample Mean (minutes)', fontsize=12)
plt.ylabel('Density', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.xlim(min_x, max_x) # X 축 범위 동일하게 설정

# Plot 2: 표본 크기 n=400 일 때
plt.subplot(2, 1, 2) # 2 행 1 열의 두 번째 플롯
sns.histplot(sample_means_400, bins=30, stat='density', color='lightgreen',
edgecolor='black', alpha=0.7, label='n=400')
sns.kdeplot(sample_means_400, color='darkgreen', linewidth=2)
plt.axvline(pop_mean, color='red', linestyle='--', label='Population Mean')
plt.title('Sampling Distribution of Means (n=400)', fontsize=14)
plt.xlabel('Sample Mean (minutes)', fontsize=12)
plt.ylabel('Density', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.xlim(min_x, max_x) # X 축 범위 동일하게 설정

plt.suptitle('Comparison of Sampling Distributions for Different Sample Sizes',
fontsize=16, y=1.02)
plt.tight_layout(rect=[0, 0.03, 1, 0.98]) # 전체 제목과 플롯 간 간격 조절
plt.show()
```

위 코드를 실행하면  $n=100$  과  $n=400$  에서의 표본평균 분포를 비교하는 그래프를 얻을 수 있다.

## A.4.4 신뢰구간: 100 개의 그물 던지기 시뮬레이션

앞서, 95% 신뢰구간의 진짜 의미는 "동일한 방법으로 표집과 구간 계산을 100번 반복한다면, 그중 약 95개의 그물은 참값을 포함하겠지만, 5개 정도는 진실을 놓칠 것이다"라고 했다.

이 철학적 개념을 코드로 증명해 보자. 모집단(`boston40`)의 평균을 알고 있는 상태에서, 100명씩 뽑아 신뢰구간을 만드는 과정을 100번 반복한다. 과연 몇 개의 그물이 진실을 가두는지 확인해 보자.

### [파이썬 코드 4-14]

```
# 100 번의 신뢰구간 생성 시뮬레이션
n, num_sim = 100, 100
pop_mean = boston40['finish_net_minutes'].mean()
count = 0

for _ in range(num_sim):
    sample = boston40['finish_net_minutes'].sample(n)
    se = sample.std() / np.sqrt(n)
    low, high = sample.mean() - 1.96*se, sample.mean() + 1.96*se

    # 참값이 구간 안에 포함되는지 확인
    if low <= pop_mean <= high:
        count += 1

print(f"100 개의 신뢰구간 중 참값을 포함한 개수: {count}")
```

100개의 신뢰구간 중 참값을 포함한 개수가 대략 95개인 것을 확인하자.

## A.4.5 관악초등학교 학생의 평균 에너지 섭취량에 대한 신뢰구간

관악초등학교 학생의 에너지 섭취량 추정 예제를 파이썬으로 구현하여 t-분포를 사용한 신뢰구간을 계산해 보고 결측값(Missing Value)은 어떻게 처리해야 하는지 확인해 보자.

### 데이터 입력 및 전처리: 결측값과의 조우

우리는 관악초등학교 소득 상위 20% 가구 학생 10명을 무작위로 선정하여 조사했다. 하지만 현실 조사에서는 늘 예상치 못한 변수가 발생한다. 8번 학생이 응답을 거부하거나 기록이 누락된 것이다. 먼저 수집된 데이터를 입력하고, 분석 가능한 상태로 만들어보자. 결측값은 NumPy 라이브러리의 NaN(Not a Number)으로 표현한다.

### [파이썬 코드 4-15]

```
import numpy as np
import scipy.stats as stats
```

```
# 1. 데이터 입력 (10 명, 8 번 학생은 결측)
# 결측값은 np.nan 으로 표현
raw_data = np.array([1531, 1809, 2094, 1625, 1115,
                     1630, 1963, np.nan, 1744, 2485])

# 2. 전처리: 결측값 제거
# 통계 계산을 위해 결측값을 제외한 유효한 데이터만 남긴다.
data = raw_data[~np.isnan(raw_data)]

print(f"원래 표본 크기: {len(raw_data)}명")
print(f"유효 표본 크기(n): {len(data)}명")
print(f"데이터: {data}")
```

```
원래 표본 크기: 10 명
유효 표본 크기(n): 9 명
데이터: [1531. 1809. 2094. 1625. 1115. 1630. 1963. 1744. 2485.]
```

원래 10명을 계획했으나, 유효한 표본의 크기( $n$ )는 9명이 되었다. 이제 이 9명의 데이터만이 우리가 가진 정보의 전부다.

## 기초 통계량 계산

이제 9명의 데이터를 바탕으로 표본평균( $\bar{x}$ )과 표본표준편차( $s$ )를 계산한다. 여기서 주의할 점은 표준편차를 계산할 때 자유도를 고려하여 `ddof=1` 옵션을 사용해야 한다는 점이다.

### [파이썬 코드 4-16]

```
# 3. 통계량 계산
n = len(data)
x_bar = np.mean(data)
s = np.std(data, ddof=1) # 표본표준편차 (자유도 n-1)

# 4. 표준오차(SE) 계산
# 모표준편차를 모르므로 표본표준편차(s)를 이용해 추정한다.
se = s / np.sqrt(n)

print("-" * 30)
print(f"표본 평균(x_bar): {x_bar:.2f} kcal")
print(f"표본 표준편차(s): {s:.2f} kcal")
print(f"표준오차(SE): {se:.2f} kcal")
print("-" * 30)
```

```
-----
표본 평균(x_bar): 1777.33 kcal
표본 표준편차(s): 384.37 kcal
표준오차(SE): 128.12 kcal
```

## t-분포를 사용한 신뢰구간의 계산

자유도( $df = n - 1 = 8$ )를 고려한 t-값을 계산하고, 관악초등학교 학생의 평균 에너지 섭취량에 대한 신뢰구간을 구한다.

### [파이썬 코드 4-17]

```
# 5. 신뢰구간
alpha = 0.05
df = n - 1
t_crit = stats.t.ppf(1 - alpha/2, df) # 자유도 8 인 t-분포의 상위 2.5% 지점

margin_t = t_crit * se
lower_t = x_bar - margin_t
upper_t = x_bar + margin_t

print(f"\n 자유도(df): {df}")
print(f"t-임계값: {t_crit:.4f}") # Z 값 1.96 보다 크다
print(f"[t-분포 사용 시] 95% 신뢰구간: [{lower_t:.2f}, {upper_t:.2f}]")
print(f"구간의 폭: {upper_t - lower_t:.2f}")
```

```
자유도(df): 8
t-임계값: 2.3060
[t-분포 사용 시] 95% 신뢰구간: [1481.88, 2072.79]
구간의 폭: 590.91
```

## A.4.6 일표본 모중앙값에 대한 추론 (신뢰구간)

앞서 우리는 평균에 대한 추론을 다루었다. 하지만 소득이나 에너지 섭취량처럼 데이터가 한쪽으로 치우친 경우, 평균보다는 **중앙값(Median)**이 집단의 대표값으로 더 적절할 때가 많다.

문제는 여기서 발생한다. 표본평균은 중심극한정리라는 강력한 이론 덕분에 표준오차를 쉽게 구할 수 있고, 이를 통해 신뢰구간을 계산하는 공식( $\bar{x} \pm 2SE$ )이 존재한다. 하지만 **표본중앙값의 표준오차**를 구하는 공식은 매우 복잡하거나, 데이터의 분포에 대한 까다로운 가정을 필요로 한다. 공식이 없거나 복잡할 때, 우리는 어떻게 신뢰구간을 구할 수 있을까? 이때야말로 **부트스트랩**이 빛을 발하는 순간이다. 복잡한 수식 없이 컴퓨터의 연산 능력만으로 중앙값의 신뢰구간을 구해 보자.

### 데이터 준비: 분석 대상 정의하기

이번 실습에서는 국민건강영양조사(KNHANES) 2019년 데이터를 사용한다. 분석의 초점을 명확히 하기 위해, '**20세 이하 남성**' 그룹을 특정하여 그들의 에너지 섭취량(**N\_EN**)을 분석 대상으로 삼았다.

**[파이썬 코드 4-18]**

```
import pandas as pd
import numpy as np

# 데이터 불러오기
hn19 = pd.read_csv("데이터/hn19.csv")

# 데이터 필터링: 에너지 섭취량이 기록된 20 세 이하 남성
hn19_15 = hn19[
    hn19['N_EN'].notna() &
    (hn19['sex'] == 1) &
    (hn19['age'] <= 20)
].copy()

# 100 명의 표본 추출 (랜덤 시드 고정)
sampled_data = hn19_15.sample(n=100, random_state=123, replace=False)
```

분석을 간단히 하기 위해 위 코드에서는 100명의 표본을 추출했다. 이제 이 100명의 데이터만 가지고 전체 20세 이하 남성의 에너지 섭취량 중앙값이 어디에 있을지 추론해야 한다.

**부트스트랩 함수 정의: 컴퓨터로 만드는 가상의 표본들**

중앙값의 표본분포를 알 수 없으므로, 우리는 **부트스트랩 재표집**을 통해 이를 직접 만들어낸다. 원리는 간단하다.

1. 우리가 가진 100 명의 표본에서 복원 추출(중복 허용)을 통해 다시 100 명을 뽑는다.
2. 이 가상의 표본(부트스트랩 표본)에서 중앙값을 계산한다.
3. 이 과정을 수천 번 반복하여 중앙값들의 분포(부트스트랩 분포)를 만든다.

**[파이썬 코드 4-19]**

```
def bootstrap_median_ci(data, B=2000, conf=0.95):
    n = len(data)
    medians = []

    # 부트스트랩 반복: 가상의 표본을 만들고 중앙값을 기록한다
    for i in range(B):
        np.random.seed(i) # 재현성을 위해 시드 설정
        sample_i = np.random.choice(data, size=n, replace=True)
        medians.append(np.median(sample_i))

    # 퍼센타일(백분위수)을 이용한 신뢰구간 계산
    alpha = 1 - conf
    lower = np.percentile(medians, 100 * (alpha / 2))
    upper = np.percentile(medians, 100 * (1 - alpha / 2))
```

```
estimate = np.median(data) # 원래 표본의 중앙값

return {
    'estimate': float(round(estimate, 3)),
    'ci_lower': float(round(lower, 3)),
    'ci_upper': float(round(upper, 3))
}
```

## 신뢰구간 계산 및 해석

이제 함수를 실행해 95% 신뢰구간을 구해보자. 반복 횟수 B는 2,000번으로 설정한다.

### [파이썬 코드 4-20]

```
# 'sampled_data'의 'N_EN' 컬럼(에너지 섭취량)에 대해 부트스트랩 수행
result = bootstrap_median_ci(sampled_data['N_EN'], B=2000, conf=0.95)
print(result)

{'estimate': 1824.3, 'ci_lower': 1698.687, 'ci_upper': 2010.36}
```

- **점 추정값(estimate):** 표본에서 계산된 중앙값 1824.3 kcal 이다.
- **부트스트랩 퍼센타일 신뢰구간(ci\_lower, ci\_upper):** 부트스트랩 분포의 하위 2.5%와 상위 97.5% 지점 (1698.687, 2010.36)이다.

다음과 같이 결론 내릴 수 있다. 20세 이하 남성의 에너지 섭취량 중앙값은 약 1824.3 kcal로 추정되며, 모중앙값에 대한 95% 부트스트랩 퍼센타일 신뢰구간은 약 1698.7 kcal에서 2010.4 kcal이다.

이처럼 부트스트랩을 사용하면 복잡한 수식 없이도 어떤 통계량(중앙값, 최빈값, 분산 등)에 대해서든 신뢰구간을 유연하게 구해낼 수 있다. 이것이 현대 통계학에서 부트스트랩이 강력한 도구로 각광받는 이유다.

## A.4.7 모평균에 대한 검정

이번 실습에서는 국민건강영양조사(KNHANES) 2019년 데이터를 사용하여 다음 두 가지 질문에 답할 것이다.

1. **일표본 t-검정:** 저소득층 여성의 평균 에너지 섭취량은 권장량인 1,600kcal 보다 적은가?
2. **이표본 t-검정:** 저소득층과 고소득층 여성 간에 평균 에너지 섭취량의 차이가 존재하는가?

### 데이터 준비: 분석 대상 정의하기

먼저 분석에 필요한 데이터를 추출한다. 위의 예제에서는 표본 수  $n = 100$ 인 표본에 대한 예를 들었지만, 이 실습에서는 KNHANES 데이터의 모든 데이터를 전부 사용한다. 성별이 여성(`sex == 2`)이

고 만 20세 이상인 성인(`age >= 20`) 중, 소득 수준이 하위 20%인 1분위(저소득층)와 상위 20%인 5분위(고소득층) 데이터를 선별한다.

#### [파이썬 코드 4-21]

```
import pandas as pd
import numpy as np
from scipy import stats

# 데이터 불러오기 (앞선 실습에서 로드된 hn19 데이터프레임 사용 가정)
# hn19 = pd.read_csv("데이터/hn19.csv")

# 1. 데이터 필터링 및 income_group 변수 생성
hn19_filtered = hn19[
    hn19['ho_incm5'].isin([1, 5]) &
    hn19['N_EN'].notna() &
    (hn19['sex'] == 2) &
    (hn19['age'] >= 20)
].copy()

# 가독성을 위해 그룹 이름 매핑
hn19_filtered['income_group'] = hn19_filtered['ho_incm5'].map({
    1: '저소득층',
    5: '고소득층'
}).astype('category')

# 2. 그룹별 요약 통계 계산
summary_stats = hn19_filtered.groupby('income_group')['N_EN'].agg(
    평균=('mean'),
    표준편차=('std'),
    n=('count')
).reset_index()

print("그룹별 칼로리 섭취량 요약:")
display(summary_stats)
```

그룹별 칼로리 섭취량 요약:

|   | income_group | 평균          | 표준편차       | n   |
|---|--------------|-------------|------------|-----|
| 0 | 고소득층         | 1685.674243 | 684.270612 | 724 |
| 1 | 저소득층         | 1307.914137 | 528.468172 | 541 |

이 코드를 실행하면 표본 수가 높은 데이터에 대한 집단별 통계량을 보여준다. 저소득층의 평균 섭취량은 약 1,308kcal, 고소득층은 약 1,686kcal로 관측된다. 또한 표준편차를 보면 저소득층(약 528)보다 고소득층(약 684)이 훨씬 크게 나타나, 두 집단의 퍼짐 정도가 다를 수 있다.

## 일표본 t-검정: 기준값과의 비교 stats.ttest\_1samp

첫 번째 질문은 "저소득층 여성의 섭취량이 1,600kcal보다 유의하게 적은가?"이다. 우리가 입증하고 싶은 것은 "적다"이므로, `alternative='less'` 옵션을 사용하여 단측 검정을 수행한다.

### [파이썬 코드 4-22]

```
# 저소득층 데이터 추출
low_income_data = hn19_filtered[hn19_filtered['income_group'] ==
    '저소득층']['N_EN']

# 귀무가설의 모평균 값 (mu_0)
mu_0 = 1600

# 1. 일표본 t-검정 수행
# alternative='less': 대립가설이 '평균 < mu_0' 인 단측 검정
t_statistic_one_sample, p_value_one_sample = stats.ttest_1samp(
    low_income_data, popmean=mu_0, alternative='less'
)

print(f"저소득층 표본 크기: {len(low_income_data)}명")
print(f"저소득층 평균 칼로리 섭취량: {low_income_data.mean():.2f} kcal")
print("-" * 30)

print(f"일표본 t-통계량: {t_statistic_one_sample:.3f}")
print(f"일표본 P-value (단측, less): {p_value_one_sample:.3f}")

저소득층 표본 크기: 541 명
저소득층 평균 칼로리 섭취량: 1307.91 kcal
-----
일표본 t-통계량: -12.856
일표본 P-value (단측, less): 0.000
```

## 결과 해석

분석 결과 t-통계량은 약 -12.856로 계산된다. 이는 관측된 평균(1307.91)이 기준값(1600)보다 표준 오차의 12배 넘게 작다는 뜻이다. 이에 따른 P-값은 0에 가까운 수치가 나오므로, 우리는 귀무가설을 강력하게 기각하고 "저소득층 여성의 평균 에너지 섭취량은 1,600kcal보다 통계적으로 유의하게 적다"고 결론 내린다.

## 이표본 t-검정: 두 집단 간의 비교 stats.ttest\_ind

두 번째 질문은 "저소득층과 고소득층 간에 섭취량 차이가 있는가?"이다. 앞서 기초 통계량에서 확인했듯, 두 집단의 표준편차(분산)가 꽤 다르다. 따라서 등분산을 가정하는 일반적인 t-검정 대신, 분산이 달라도 안전하게 사용할 수 있는 **웰치의 t-검정(Welch's t-test)**을 수행해야 한다. 파이썬에서

는 `equal_var=False` 옵션으로 이를 간단히 적용할 수 있다.

#### [파이썬 코드 4-23]

```
# 고소득층 데이터 추출
high_income_data = hn19_filtered[hn19_filtered['income_group'] ==
    '고소득층']['N_EN']

print(f"저소득층 표본 크기: {len(low_income_data)}명")
print(f"고소득층 표본 크기: {len(high_income_data)}명")
print("-" * 30)

# 2. 이표본 t-검정 (Welch's t-test)
# equal_var=False: 등분산 가정을 하지 않음 (Welch's t-test 수행)
t_statistic, p_value = stats.ttest_ind(
    low_income_data, high_income_data, equal_var=False
)

print(f"Welch's t-statistic: {t_statistic:.3f}")
print(f"P-value: {p_value:.3f}")

저소득층 표본 크기: 541 명
고소득층 표본 크기: 724 명
-----
Welch's t-statistic: -11.077
P-value: 0.000
```

#### 결과 해석

t-통계량은 약 -11.077, P-값은 거의 0으로 유의수준 0.05보다 훨씬 작다. 따라서 "소득 수준에 따라 성인 여성의 평균 에너지 섭취량에는 통계적으로 유의한 차이가 있다"는 결론을 내릴 수 있다.

### A.4.8 분산분석

이번 실습에서는 소득 수준(1~5분위)에 따른 대한민국 성인 여성의 에너지 섭취량의 차이를 분산분석(ANOVA)을 이용하여 검정한다.

#### 데이터 준비 및 탐색

먼저 국민건강영양조사 데이터에서 성인 여성을 대상으로 소득 분위별(1~5분위)로 각각 100명씩을 무작위 추출하여 분석 데이터를 생성한다.

#### [파이썬 코드 4-24]

```
import pandas as pd
import numpy as np
```

```
# (데이터 로드 과정 생략: hn19 데이터프레임이 준비되었다고 가정)
# 1. 데이터 필터링: 성인 여성, 소득분위 1~5, 에너지 섭취량(N_EN) 있는 경우
data_anova = hn19[
    hn19['ho_incm5'].isin([1, 2, 3, 4, 5]) & # 소득 1~5 분위 모두 포함
    hn19['N_EN'].notna() &
    (hn19['sex'] == 2) &
    (hn19['age'] >= 19)
].copy()

# 2. 그룹별 100명 무작위 표집
sampled_data = data_anova.groupby('ho_incm5', group_keys=False).sample(n=100,
    replace=False, random_state=123)

# 3. 그룹별 기초 통계량 확인
summary = sampled_data.groupby('ho_incm5')['N_EN'].agg(['mean', 'std', 'count'])
print(summary)
```

|          | mean        | std        | count |
|----------|-------------|------------|-------|
| ho_incm5 |             |            |       |
| 1.0      | 1226.856822 | 476.428809 | 100   |
| 2.0      | 1505.740736 | 509.251808 | 100   |
| 3.0      | 1599.797384 | 618.530528 | 100   |
| 4.0      | 1638.810116 | 629.228414 | 100   |
| 5.0      | 1707.945572 | 720.338576 | 100   |

## 결과 해석

기초 통계량을 살펴보면 소득 1분위의 평균 섭취량은 약 1,226kcal인 반면, 5분위는 약 1,707kcal로 꽤 큰 차이를 보인다. 하지만 표준편차를 보면 그룹 내에서도 개인차가 상당히 큼을 알 수 있다. 과연 이 평균의 차이가 통계적으로 유의미한지 검토해 보자.

## 분산분석(ANOVA) 수행

분산분석을 위해 `statsmodels` 라이브러리를 사용한다. 여기서 핵심은 `N_EN ~ C(ho_incm5)` 식이다. 이는 "에너지 섭취량(`N_EN`)을 소득 분위(`ho_incm5`)라는 범주형 변수(`C`)로 설명하겠다"는 의미다.

### [파이썬 코드 4-25]

```
import statsmodels.api as sm
from statsmodels.formula.api import ols

# 4. ANOVA 모델 적합
# C(ho_incm5): 소득분위를 연속형 숫자가 아닌 범주형(Category) 변수로 인식하게 함
model = ols('N_EN ~ C(ho_incm5)', data=sampled_data).fit()

# 5. 분산분석표(ANOVA Table) 출력
```

```
anova_table = sm.stats.anova_lm(model, typ=2)
print(anova_table)
```

|             | sum_sq       | df    | F        | PR(>F)       |
|-------------|--------------|-------|----------|--------------|
| C(ho_incm5) | 1.406903e+07 | 4.0   | 9.859343 | 1.109332e-07 |
| Residual    | 1.765881e+08 | 495.0 | NaN      | NaN          |

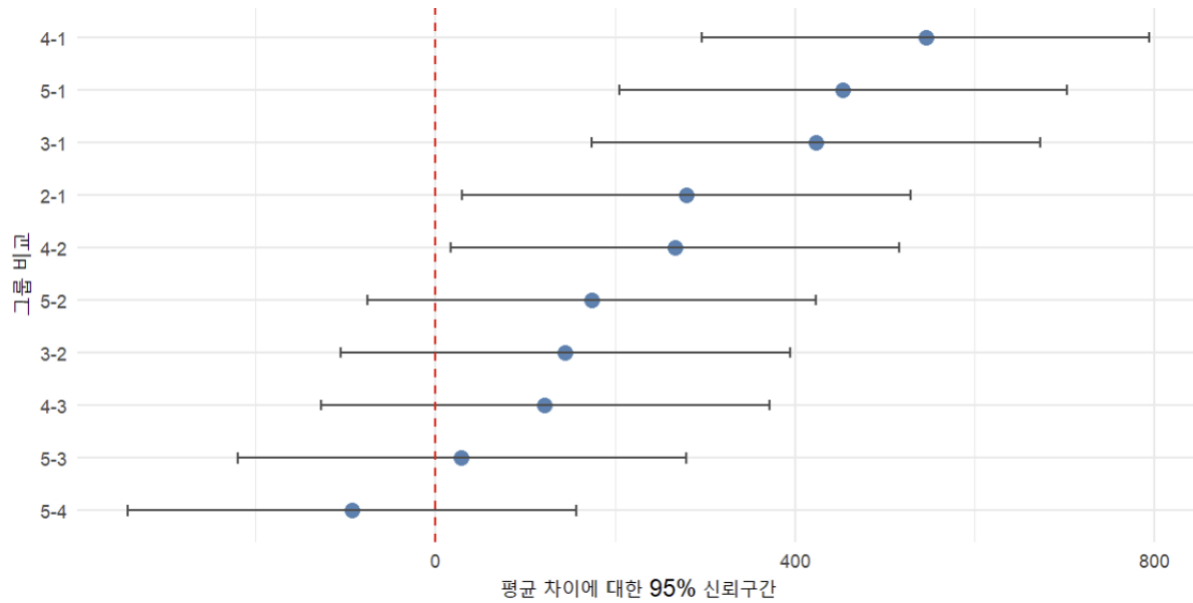
분산분석표에서 가장 중요한 두 가지 숫자는 F-값 9.86과 P-값( **PR(>F)** ) 1.109332e-07 ( $1.109332 \times 10^{-7}$ )이다. P-값이 0.05보다 훨씬 작으므로, 우리는 귀무가설을 기각하고, "**적어도 하나의 소득 그룹은 다른 그룹들과 평균 섭취량이 다르다**"고 결론 내린다.

분산분석의 결론은 "뭔가 다르긴 하다"는 것일 뿐, 구체적으로 "1분위와 2분위가 다른지", 아니면 "4분위와 5분위가 다른지"는 알려주지 않는다. 이를 확인하기 위해 추가로 수행하는 것이 **사후 검정 (Post-hoc Test)**이며, 가장 널리 쓰이는 방법이 **튜키의 HSD(Tukey's Honest Significant Difference)** 검정이다. Tukey HSD는 모든 집단 쌍을 동시에 비교하면서 가족별 제1종 오류율 (family-wise error rate)을 통제하는 다중비교 방법이다. 따라서 개별 쌍을 각각 따로 검정하는 경우보다 귀무가설 기각에 더 보수적인 편이다.

튜키의 HSD 검정을 소득 수준별 평균 에너지 섭취량 데이터에 적용한 결과를 [그림 A-4-3]에 표시하였다. [그림 A-4-3]은 모든 가능한 집단 쌍의 평균 차이에 대한 95% 신뢰구간을 보여준다. 그래프 Y축의 "4-1"은 4분위 집단과 1분위 집단의 모평균 차이를 뜻한다.

- **1 분위 vs 5 분위 (5-1):** 신뢰구간이 0 을 포함하지 않고 양수 쪽에 있다. 즉, 5 분위의 평균 칼로리 섭취량이 1 분위보다 통계적으로 유의하게 높다.
- **4 분위 vs 5 분위 (5-4):** 신뢰구간이 0 을 포함하고 있다. 즉, 통계적으로 두 집단 간에는 차이가 있다고 말하기 어렵다.

이처럼 분산분석으로 전체적인 차이의 유무를 먼저 판별(숲을 봄)한 뒤, 사후 검정을 통해 구체적인 차이의 관계(나무를 봄)를 파악하는 것이 올바른 순서다.



[그림 A-4-3] 튜키의 HSD 검정에 의한 모든 집단 쌍별 신뢰구간

## 튜키의 HSD 검정

ANOVA는 "차이가 있다"는 사실만 알려줄 뿐, 구체적으로 "1분위와 5분위가 다른지", "3분위와 4분위가 다른지"는 알려주지 않는다. 이를 확인하기 위해 **Tukey의 HSD(Honest Significant Difference)** 검정을 수행한다.

### [파이썬 코드 4-26]

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd

# 6. 사후 검정 (Post-hoc Test)
tukey = pairwise_tukeyhsd(endog=sampled_data['N_EN'],      # 데이터
                           groups=sampled_data['ho_incm5'], # 그룹
                           alpha=0.05)                    # 유의수준

print(tukey)
```

```
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====
group1 group2 meandiff p-adj    lower    upper  reject
-----
1.0     2.0  278.8839 0.0091   47.6219 510.1459   True
1.0     3.0  372.9406 0.0001  141.6786 604.2026   True
1.0     4.0  411.9533   0.0   180.6913 643.2153   True
1.0     5.0  481.0887   0.0   249.8268 712.3507   True
2.0     3.0   94.0566 0.7994  -137.2053 325.3186  False
2.0     4.0  133.0694 0.5139  -98.1926 364.3314  False
2.0     5.0  202.2048 0.1187  -29.0572 433.4668  False
3.0     4.0   39.0127 0.9906  -192.2493 270.2747  False
```

```
3.0    5.0 108.1482 0.7035 -123.1138 339.4102 False
4.0    5.0  69.1355 0.9249 -162.1265 300.3974 False
-----
```

## 결과 해석

출력된 표에서 **reject** 컬럼이 **True**인 쌍을 찾으면 된다.

- **1 분위 vs 5 분위:** **reject=True**. 두 그룹 간에는 확실한 차이가 있다.
- **3 분위 vs 4 분위:** **reject=False**. 두 그룹 간의 차이는 통계적으로 유의하지 않다.

이처럼 분산분석으로 전체 차이 유무를 먼저 판단한 후, 사후 검정으로 개별 그룹 간 차이를 확인하는 것이 정석적인 분석 절차다.

## A.4.9 카이제곱 독립성 검정

앞서 우리는 범주형 데이터의 관계를 파악하기 위해 카이제곱 검정을 배운 바 있다. 이번 실습에서는 국민건강영양조사 데이터를 이용하여 **"20대 여성의 소득 수준과 주관적 건강 상태 인식 사이에 관련이 있는가?"**라는 질문에 답해보고자 한다.

### 데이터 준비 및 교차표 생성

먼저 분석 대상을 명확히 한다. 전체 데이터에서 '만 20세 이상 29세 이하의 성인 여성'만을 추출하고, 결측값이 있는 데이터는 제거한다. 분석할 변수는 소득 분위(**ho\_incm5**)와 주관적 건강 상태 (**D\_1\_1**)다.

#### [파이썬 코드 4-27]

```
import pandas as pd
from scipy.stats import chi2_contingency, permutation_test
import numpy as np

# 1. 데이터 필터링: 20대 여성 데이터 추출
# 결측값(9) 제거 및 필요한 변수만 선택
health_income_data = hn19[
    (hn19['sex'] == 2) &
    (hn19['age'] >= 20) &
    (hn19['age'] <= 29) &
    (hn19['D_1_1'] != 9)
][['D_1_1', 'ho_incm5']].dropna()

# 범주형 변수로 변환
health_income_data['D_1_1'] = health_income_data['D_1_1'].astype('category')
health_income_data['ho_incm5'] = health_income_data['ho_incm5'].astype('category')
```

```
print(f"분석 대상 데이터 크기: {len(health_income_data)}명")

# 2. 분할표 (Contingency Table) 생성
contingency_table = pd.crosstab(health_income_data['D_1_1'],
    health_income_data['ho_incm5'])

print("건강 상태(D_1_1)와 소득 분위(ho_incm5)의 교차표:")
display(contingency_table)
```

분석 대상 데이터 크기: 314 명  
건강 상태(D\_1\_1)와 소득 분위(ho\_incm5)의 교차표:

| ho_incm5 | 1.0 | 2.0 | 3.0 | 4.0 | 5.0 |
|----------|-----|-----|-----|-----|-----|
| D_1_1    |     |     |     |     |     |
| 1.0      | 4   | 1   | 1   | 2   | 7   |
| 2.0      | 2   | 3   | 19  | 24  | 40  |
| 3.0      | 11  | 15  | 40  | 38  | 53  |
| 4.0      | 5   | 8   | 13  | 9   | 11  |
| 5.0      | 0   | 2   | 1   | 1   | 4   |

분할표를 살펴보면, 특정 셀(Cell)의 빈도가 매우 적은 것을 확인할 수 있다. 예를 들어 건강이 '매우 좋음(1)'이면서 소득 2분위인 표본은 단 1명이다.

## 카이제곱 검정 수행

이제 `scipy.stats`의 `chi2_contingency` 함수를 이용해 카이제곱 검정을 수행한다. 이 때의 귀무가설과 대립가설은 다음과 같다.

- 귀무가설  $H_0$ : 소득 분위와 주관적 건강 상태는 서로 독립이다(관련이 없다).
- 대립가설  $H_1$ : 소득 분위와 주관적 건강 상태는 서로 독립이 아니다(관련이 있다).

### [파이썬 코드 4-28]

```
# 3. 카이제곱 독립성 검정
chi2, p_value, dof, expected = chi2_contingency(contingency_table)

print(f"카이제곱 통계량: {chi2:.3f}")
print(f"P-value: {p_value:.3f}")
print(f"자유도 (Degrees of Freedom): {dof}")
```

```
print("기대 빈도표:\n", pd.DataFrame(expected, index=contingency_table.index,
                                     columns=contingency_table.columns))
```

카이제곱 통계량: 31.501

P-value: 0.012

자유도 (Degrees of Freedom): 16

기대 빈도표:

| ho_incm5 | 1.0       | 2.0       | 3.0       | 4.0       | 5.0       |
|----------|-----------|-----------|-----------|-----------|-----------|
| D_1_1    |           |           |           |           |           |
| 1.0      | 1.050955  | 1.385350  | 3.535032  | 3.535032  | 5.493631  |
| 2.0      | 6.165605  | 8.127389  | 20.738854 | 20.738854 | 32.229299 |
| 3.0      | 11.000000 | 14.500000 | 37.000000 | 37.000000 | 57.500000 |
| 4.0      | 3.222930  | 4.248408  | 10.840764 | 10.840764 | 16.847134 |
| 5.0      | 0.560510  | 0.738854  | 1.885350  | 1.885350  | 2.929936  |

## 결과 해석의 주의점

여기서 계산된 P-값을 그대로 믿어도 될까? 카이제곱 검정은 표본의 크기가 충분히 클 때(보통 기대 빈도가 5 이상일 때) 검정통계량이 카이제곱 분포를 따른다는 이론에 근거한다. 하지만 앞서 확인했듯이 우리 데이터에는 **빈도가 1, 2 (심지어 0)과 같이 매우 작은 셀들이 존재한다**. 이런 경우 이론적인 카이제곱 분포 근사가 부정확할 수 있어, 계산된 P-값이 왜곡될 위험이 있다.

이때의 해결책 중 하나는 재범주화를 진행하는 것이다. 즉, 의미가 서로 다르지 않은 두 개 이상의 범주를 하나로 합치는 방법으로 실전에서 제법 많이 쓰인다. 예를 들어 건강 상태 1-2는 " 좋음"으로, 3은 "보통"으로, 4-5는 "나쁨"으로 재범주화한다. 재범주화 후 원래 분할표보다 기대빈도 조건이 크게 개선되어 카이제곱 근사의 신뢰성이 높아진다. 이 예제에서 최소 기대빈도는 약 3.78이며, 15개 셀 중 2개는 5 미만이다. 따라서 결과를 해석할 때 이 점을 함께 고려해야 한다.

### [파이썬 코드 4-29]

```
# D_1_1의 범주 재정의: (1,2) -> ' 좋음', 3 -> '보통', (4,5) -> '나쁨'
def recode_health_status(d11_value):
    if d11_value in [1, 2]:
        return ' 좋음'
    elif d11_value == 3:
        return '보통'
    elif d11_value in [4, 5]:
        return '나쁨'
    else:
        return np.nan # 정의되지 않은 값은 NaN으로 처리

health_income_data['health_status_recoded'] =
    health_income_data['D_1_1'].apply(recode_health_status).astype('category')

# ho_incm5는 기존대로 범주형 변수로 변환
```

```
health_income_data['ho_incm5'] = health_income_data['ho_incm5'].astype('category')

print(f"분석 대상 데이터 크기: {len(health_income_data)}명")

# 2. 분할표 (Contingency Table) 생성
contingency_table_3 = pd.crosstab(health_income_data['health_status_recoded'],
    health_income_data['ho_incm5'])

print("재범주화된 건강 상태와 소득 분위의 교차표:")
display(contingency_table_3)
```

분석 대상 데이터 크기: 314 명  
재범주화된 건강 상태와 소득 분위의 교차표:

| ho_incm5              | 1.0 | 2.0 | 3.0 | 4.0 | 5.0 |
|-----------------------|-----|-----|-----|-----|-----|
| health_status_recoded |     |     |     |     |     |
| 나쁨                    | 5   | 10  | 14  | 10  | 15  |
| 보통                    | 11  | 15  | 40  | 38  | 53  |
| 좋음                    | 6   | 4   | 20  | 26  | 47  |

재범주화된 데이터를 이용하여 독립성 검정을 다시 시행한다.

#### [파이썬 코드 4-30]

```
chi2, p_value, dof, expected = chi2_contingency(contingency_table_3)
print(f"P-value: {p_value:.3f}")

P-value: 0.070
```

카이제곱 검정의 결과 P-값은 0.07로서 유의수준 0.05보다 크다! 귀무가설 (두 변수의 독립성)을 기각할 증거가 충분하지 않기 때문에, 우리는 20대 여성 집단에서 소득 수준과 주관적 건강 인지 사이에 통계적으로 유의미한 관련성을 발견하지 못한 것이다.

## A.5 5 부 실습

### A.5.1 변수 선택

이 장에서는 서울시 법정동 단위 아파트 매매가격 데이터를 이용하여 본문의 3부에서 다룬 변수 선택 방법을 파이썬으로 직접 실습해 본다. 어떤 설명변수를 모형에 포함할지 결정하는 것은 예측 성능과 해석 가능성 모두에 영향을 미치는 중요한 과정이다.

#### 탐색적 자료 분석

본격적인 분석에 앞서 데이터의 전반적인 구조와 변수 간의 관계를 파악하는 **탐색적 자료 분석 (EDA)**을 진행한다. 먼저 분석에 필요한 라이브러리를 불러온다.

##### [파이썬 코드 5-1]

```
# 라이브러리 로딩
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from itertools import combinations
from sklearn.model_selection import KFold
from sklearn.linear_model import LinearRegression
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.metrics import mean_squared_error
import warnings
warnings.filterwarnings("ignore")
# 폰트 설정
plt.rc('font', family='Malgun Gothic')
# 마이너스 부호 깨짐 방지
plt.rcParams['axes.unicode_minus'] = False
```

다음으로 데이터를 불러오고 분석에 필요한 전처리를 수행한다. 본 실습에서 사용하는 주요 라이브러리를 간략히 소개하면 다음과 같다. 변수 선택과 직접 관련된 라이브러리로는 `itertools`의 `combinations`가 있는데, 이는 변수들의 모든 조합을 자동으로 만들어주는 함수로 최적 부분집합 탐색에 활용된다. `scikit-learn`에서는 선형 회귀 모형을 적합하는 `LinearRegression`, 전진·후진 선택법을 수행하는 `SequentialFeatureSelector`, 교차 검증 데이터 분할을 위한 `KFold`, 그리고 예측 오차를 계산하는 `mean_squared_error`를 사용한다.

##### [파이썬 코드 5-2]

```
# 데이터 로드
apt = pd.read_csv("데이터/seoul_apt_dong_model.csv")

# 식별 변수 제거와 비율 단위 통일을 마친 분석용 데이터
X = apt.drop(columns="y")
y = apt["y"]

# 관측치 수(n)와 설명변수 수(p) 저장
n, p = len(y), X.shape[1]
```

데이터는 서울시 법정동 단위로 수집된 **268개의 관측치**와 **8개의 설명변수**로 이루어져 있다. 주요 변수를 소개하면 다음과 같다.

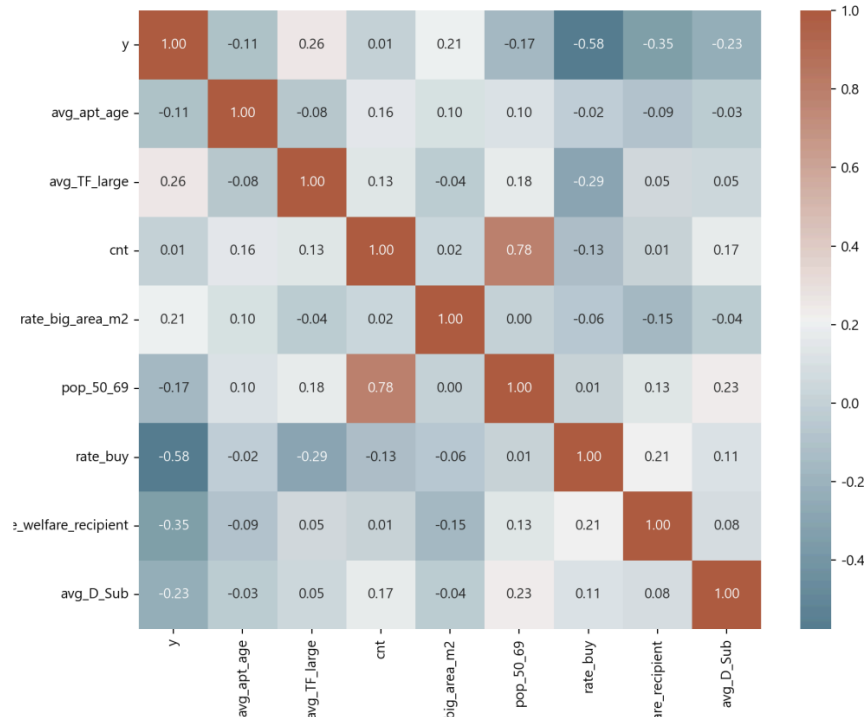
| 변수명                    | 설명                         |
|------------------------|----------------------------|
| y                      | 아파트 매매 평균가격(㎡당, 원), 로그 변환값 |
| avg_apt_age            | 평균 아파트 연령(2023년 기준)        |
| avg_TF_large           | 대단지 아파트 비율                 |
| cnt                    | 아파트 수                      |
| rate_big_area_m2       | 대형 평수(85㎡ 이상) 비율           |
| pop_50_69              | 50~69세 인구                  |
| rate_buy               | 매매 거래 비율                   |
| rate_welfare_recipient | 기초생활수급자 비율                 |
| avg_D_Sub              | 인근 지하철역까지의 평균 거리(m)        |

이제 변수 간의 선형적 연관성 파악을 위해 상관계수 행렬을 계산하고 히트맵으로 시각화해 보자.

### [파이썬 코드 5-3]

```
# 상관계수 행렬 계산 및 히트맵 시각화
cor_mat = apt.corr()

plt.figure(figsize=(10, 8))
sns.heatmap(cor_mat, annot=True, fmt=".2f",
             cmap=sns.diverging_palette(220, 20, as_cmap=True))
plt.show()
```



[그림 A-5-1] 상관계수 행렬 및 히트맵 시각화

[그림 A-5-1]의 히트맵을 통해 반응변수  $y$ (로그 아파트 가격)와 각 설명변수의 상관관계를 한눈에 파악할 수 있다. 예를 들어  $rate\_big\_area\_m2$ (대형 평수 비율)이나  $avg\_TF\_large$ (대단지 비율)는 가격과 양의 상관관계를 보이는 반면,  $rate\_welfare\_recipient$ (수급자 비율)는 음의 상관관계를 나타낼 것으로 예상된다. 이처럼 탐색적 자료 분석을 통해 예측에 유용한 변수들을 미리 파악해두면, 이후 변수 선택 결과를 해석하는 데 도움이 된다.

## 변수 선택 실습

### 최적 부분집합 선택법

**최적 부분집합 선택법(Best Subset Selection)**은 가능한 모든 변수 조합을 탐색하여 각 변수 개수별로 가장 성능이 좋은 모형을 찾는 방법이다. 변수가  $k$ 개인 모형들 중 최선의 모형  $M_k$ 를 구한 뒤,  $C_p$ , BIC, 수정된 결정계수 등의 기준으로 최종 모형을 결정한다.

먼저 모든 변수를 포함한 전체 모형(Full Model)을 적합하여 오차 분산  $\sigma^2$ 을 추정한다. 이 값은 이후  $C_p$  계산에서 패널티 항의 기준값으로 사용된다.

### [파이썬 코드 5-4]

```
# 전체 모형으로 오차분산(sigma^2) 추정
model_full = LinearRegression().fit(X, y)
```

```

rss_full = np.sum((y - model_full.predict(X)) ** 2)
sigma2_hat = rss_full / (n - p - 1) # 불편추정량
tss = np.sum((y - y.mean()) ** 2) # 총제곱합 (결정계수 계산용)

```

이제 변수 개수  $k$ 를 1개부터 8개까지 늘려가며 모든 조합을 탐색하고, 각 단계에서 잔차제곱합(RSS)이 가장 작은 최적 조합과 평가 지표를 함께 기록한다.

#### [파이썬 코드 5-5]

```

# 변수 개수별 최적 부분집합 탐색 및 BIC 계산
results = []

for k in range(1, p + 1):
    best_rss, best_vars = np.inf, None

    for cols in combinations(X.columns, k):
        rss = np.sum((y - LinearRegression().fit(X[list(cols)],
            y).predict(X[list(cols)])) ** 2)

        if rss < best_rss:
            best_rss, best_vars = rss, cols

    bic = n * np.log(best_rss / n) + k * np.log(n)
    results.append({"변수 개수": k, "선택 변수": best_vars, "BIC": bic})

reg_summary = pd.DataFrame(results).set_index("변수 개수")
print(reg_summary.round(3))

```

| 변수 개수 | 선택 변수                                              | BIC      |
|-------|----------------------------------------------------|----------|
| 1     | (rate_buy,)                                        | -722.365 |
| 2     | (rate_buy, rate_welfare_recipient)                 | -740.682 |
| 3     | (rate_buy, rate_welfare_recipient, avg_D_Sub)      | -746.173 |
| 4     | (avg_appt_age, rate_buy, rate_welfare_recipient... | -750.544 |
| 5     | (avg_appt_age, rate_big_area_m2, rate_buy, rate... | -755.349 |
| 6     | (avg_appt_age, avg_TF_large, rate_big_area_m2, ... | -758.523 |
| 7     | (avg_appt_age, avg_TF_large, rate_big_area_m2, ... | -758.599 |
| 8     | (avg_appt_age, avg_TF_large, cnt, rate_big_area... | -757.973 |

반복문이 끝나면 변수 개수별로 최적 조합과 BIC 값이 `reg_summary`에 정리된다. BIC가 가장 낮은 변수 개수가 이 기준에서의 최적 모형이다.

#### 평가 기준을 통한 변수 개수 선택

각 변수 개수에 따른 BIC 값을 시각화하면 최적의 변수 개수를 직관적으로 파악할 수 있다.

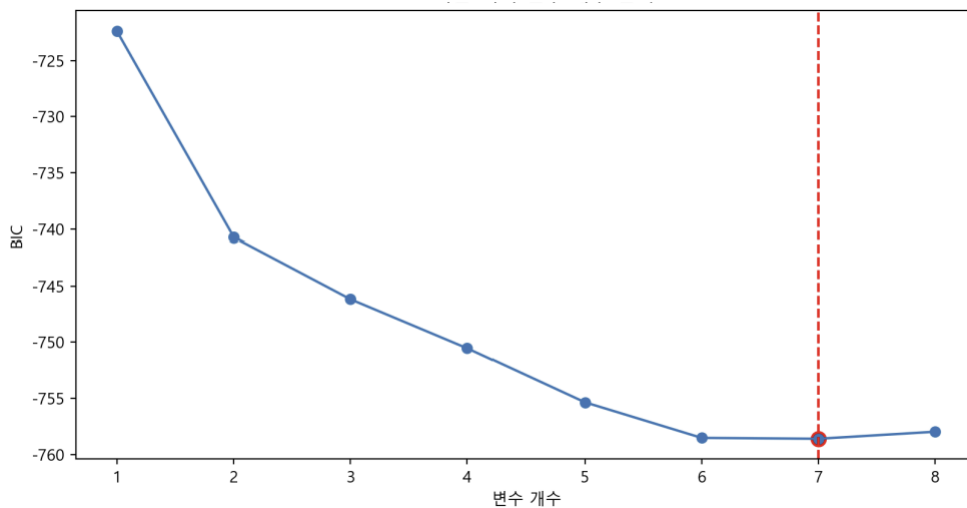
#### [파이썬 코드 5-6]

```
# BIC 시각화
best_k = reg_summary["BIC"].idxmin()

plt.figure()
plt.plot(reg_summary.index, reg_summary["BIC"], marker="o")
plt.axvline(best_k, color="red", linestyle="--")
plt.scatter(best_k, reg_summary.loc[best_k, "BIC"], color="red", s=80)
plt.xlabel("변수 개수")
plt.ylabel("BIC")
plt.title("BIC 기준 최적 변수 개수 선택")
plt.show()

print(f"BIC 기준 최적 변수 개수: {best_k}개")
print(f"선택된 변수: {list(reg_summary.loc[best_k, '선택 변수'])}")
```

BIC 기준 최적 변수 개수: 7개  
 선택된 변수: ['avg\_apartment\_age', 'avg\_TF\_large', 'rate\_big\_area\_m2', 'pop\_50\_69', 'rate\_buy', 'rate\_welfare\_recipient', 'avg\_D\_Sub']



[그림 A-5-2] BIC 기준 최적 변수 개수 선택

[그림 A-5-2]에서 BIC가 가장 낮은 지점이 빨간 점과 점선으로 표시된다. BIC는 변수가 늘어날수록 강한 패널티를 부여하기 때문에, 단순히 RSS가 가장 낮은 모형보다 더 간결한 모형을 선택하는 경향이 있다. 그래프를 살펴보면 BIC 기준에서는 **7개** 변수를 포함한 모형이 최적으로 나타난다.

### 교차 검증을 이용한 변수 개수 선택

앞에서는 훈련 데이터 안에서 계산된 지표로 변수 개수를 선택했다면, 이번에는 모형이 실제로 보이지 못한 데이터에서 얼마나 잘 예측하는지를 직접 측정하는 교차 검증을 수행한다. 여기서는 전체 데이터를 10개 조각으로 나누어 검증하는 10-겹 교차 검증(10-fold CV)을 사용한다.

**[파이썬 코드 5-7]**

```
# 10-겹 교차검증으로 변수 개수 선택
kf = KFold(n_splits=10, shuffle=True, random_state=1)
cv_errors = np.zeros((10, p))

for fold_idx, (train_idx, test_idx) in enumerate(kf.split(X)):
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    for k in range(1, p + 1):
        best_rss, best_vars = np.inf, None

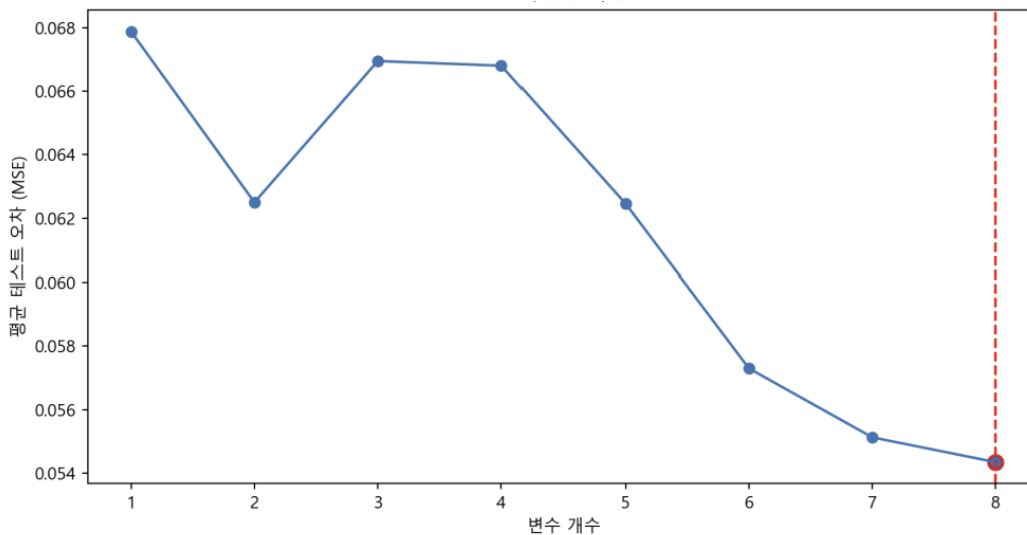
        for cols in combinations(X.columns, k):
            model = LinearRegression().fit(X_train[list(cols)], y_train)
            rss = np.sum((y_train - model.predict(X_train[list(cols)])) ** 2)
            if rss < best_rss:
                best_rss, best_vars = rss, cols

        model = LinearRegression().fit(X_train[list(best_vars)], y_train)
        cv_errors[fold_idx, k - 1] = mean_squared_error(
            y_test, model.predict(X_test[list(best_vars)])
        )

mean_cv = cv_errors.mean(axis=0)
optimal_k = np.argmin(mean_cv) + 1

print(optimal_k)
```

8

**[그림 A-5-3]** 교차 검증 결과 시각화

데이터를 순서에 상관없이 고르게 나누기 위해 `shuffle=True` 옵션을 적용한다. 결과는 `cv_errors` 배열에 저장되며, 행은 폴드 번호, 열은 변수 개수에 해당한다. 모든 반복이 끝나면 열 방향으로 평균을 내어 변수 개수별 평균 테스트 오차를 구하고, 이 값이 가장 작은 변수 개수를 최적으로 선택한다. 총 8개의 변수가 선택된 것을 확인할 수 있다. 즉, CV 방법을 사용했을 때 변수의 개수가 줄어들지 않는 것을 확인할 수 있다.

#### [파이썬 코드 5-8]

```
# 교차검증 결과 시각화
plt.figure()
plt.plot(range(1, p + 1), mean_cv, marker="o")
plt.axvline(optimal_k, color="red", linestyle="--")
plt.scatter(optimal_k, mean_cv[optimal_k - 1], color="red", s=80)
plt.xlabel("변수 개수")
plt.ylabel("평균 테스트 오차 (MSE)")
plt.title("교차검증 곡선")
plt.show()

print(f"교차검증 기준 최적 변수 개수: {optimal_k}개")
```

교차검증 기준 최적 변수 개수: 8개

[그림 A-5-3]에서 평균 테스트 오차가 가장 낮은 지점이 빨간 점으로 표시된다. 이 지점이 새로운 데이터에 대해 가장 안정적인 예측을 보이는 변수 개수다. 교차 검증 기준으로는 변수의 개수가 8개 선택됨을 확인할 수 있다.

### 최종 모형 적합과 해석

정보 기준과 교차 검증은 서로 다른 변수 수를 선택했다. 이 실습에서는 예측 성능을 우선하여 교차 검증이 선택한 8개 변수를 최종 모형으로 사용하고 회귀계수를 확인한다.

#### [파이썬 코드 5-9]

```
# 최종 모형 적합 및 회귀계수 확인
final_vars = reg_summary.loc[optimal_k, "선택 변수"]
final_model = LinearRegression().fit(X[list(final_vars)], y)

coef_series = pd.Series(final_model.coef_, index=final_vars).round(3)
print(coef_series)
```

|                  |        |
|------------------|--------|
| avg_apt_age      | -0.006 |
| avg_TF_large     | 0.299  |
| cnt              | 0.005  |
| rate_big_area_m2 | 0.402  |
| pop_50_69        | -0.000 |
| rate_buy         | -1.964 |

```
rate_welfare_recipient    -2.712
avg_D_Sub                  -0.000
dtype: float64
```

이 모형의 반응변수  $y$ 는 아파트 가격에 로그를 취한 값이므로, 회귀계수를 해석할 때는 지수함수 ( $exp$ )를 이용해 **배수(비율)** 개념으로 읽는 것이 정확하다. 예를 들어 다른 변수들이 모두 고정되어 있을 때, 평균 아파트 나이가 1년 증가하면 가격은  $exp(-0.006)$ 배로 미세하게 감소한다. 반면 대단지 비율이나 대형 평수 비율이 높을수록 가격은 상승하고, 기초생활수급자 비율이 높을수록 가격은 현저히 감소하는 경향이 있다.

### 데이터 분석가의 한마디

최적 부분집합 선택법은 모든 조합을 탐색하므로 이론적으로 가장 완벽한 방법이지만, 변수가 많아질수록 계산 시간이 기하급수적으로 늘어난다는 단점이 있다. 변수가  $p$ 개라면 탐색해야 할 조합의 수는  $2^p$ 개에 달한다. 예를 들어 변수가 20개만 되어도 100만 개 이상의 모형을 비교해야 한다. 이 때문에 실무에서는 아래에서 소개할 단계적 선택법처럼 계산 효율이 높은 방법을 함께 활용한다.

## 단계적 선택법: 전진선택법과 후진선택법

완전탐색 방식의 연산 부담을 줄이기 위해 실무에서는 **단계적 선택법(Stepwise Selection)**을 널리 활용한다. 변수를 하나씩 추가하거나 제거하면서 최적 모형을 탐색하는 방식이다.

**전진선택법(Forward Selection)**은 아무 변수도 없는 상태에서 시작하여, 모형 성능을 가장 크게 높이는 변수를 한 번에 하나씩 추가한다. 반대로 **후진선택법(Backward Selection)**은 모든 변수가 포함된 상태에서 시작하여, 기여도가 가장 낮은 변수를 하나씩 제거한다.

파이썬에서는 `SequentialFeatureSelector`를 사용해 두 방법 모두 손쉽게 구현할 수 있으며, `direction` 파라미터만 바꾸면 된다.

### [파이썬 코드 5-10]

```
# Forward selection: choose four variables intentionally for this example.
sfs_forward = SequentialFeatureSelector(
    LinearRegression(),
    direction="forward",
    n_features_to_select=4,
    scoring="r2",
    cv=None
)
sfs_forward.fit(X, y)
forward_vars = X.columns[sfs_forward.get_support()].tolist()
print("전진선택법 선택 변수:", forward_vars)
```

여기서는 4개 변수를 선택하도록 명시했으며, 전진선택법의 결과 선택된 변수를 확인할 수 있다.

#### [파이썬 코드 5-11]

```
# Backward selection uses the same intentional four-variable target.
sfs_backward = SequentialFeatureSelector(
    LinearRegression(),
    direction="backward",
    n_features_to_select=4,
    scoring="r2",
    cv=None
)
sfs_backward.fit(X, y)
backward_vars = X.columns[sfs_backward.get_support()].tolist()
print("후진선택법 선택 변수:", backward_vars)
```

후진선택법도 4개 변수를 선택하도록 명시한다. 두 방법의 결과를 비교하면 어떤 변수가 일관되게 선택되는지, 그리고 탐색 방향에 따라 결과가 달라지는 경우가 있는지 확인할 수 있다. 단순 전진선택법은 현재 선택 집합에 변수를 하나씩 추가하는 탐욕적 절차이므로, 한 번 추가한 변수를 이후 단계에서 제거하지 않는다. 따라서 전체 가능한 변수 조합 중 최적 모형을 보장하지 않는다.

#### 데이터 분석가의 한마디

전진선택법과 후진선택법은 빠르고 실용적이지만, 반드시 전체 최적 모형을 찾는다는 보장은 없다. 예를 들어 단순 전진선택법은 한 번 추가한 변수를 이후 단계에서 제거하지 않는 탐욕적 절차이므로, 초기 단계의 선택이 이후 결과에 영향을 줄 수 있다. 또한 변수들의 결합 효과를 전체 조합 수준에서 탐색하지 않으므로 전역적으로 최적인 변수 집합을 보장하지 않는다. 따라서 결과를 맹목적으로 따르기보다는, 본문에서 배운 다양한 기준(BIC, 교차검증 오차 등)과 도메인 지식을 함께 활용하여 최종 모형을 결정하는 것이 바람직하다.

## A.5.2 정규화

이 장에서는 서울시 법정동 단위 아파트 매매가격 데이터를 이용하여 본문에서 다룬 **정규화 (Regularization)** 방법을 파이썬으로 직접 실습해 본다. 앞서 변수 선택에서는 변수를 넣거나 뺀다면, 정규화는 모든 변수를 유지하면서 회귀계수의 크기에 제약을 걸어 모형이 과적합되는 것을 방지하는 방식이다. 여기서는 라쏘(LASSO), 릿지(Ridge), Elastic Net 세 가지 방법을 순서대로 실습한다.

## 사전 준비

데이터 로드와 전처리는 변수 선택 실습과 동일하다. 이번 장에서 추가로 필요한 라이브러리를 불러온다.

### [파이썬 코드 5-12]

```
# Additional libraries for regularization
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import (
    Lasso, LassoCV,
    Ridge, RidgeCV,
    ElasticNet, ElasticNetCV
)
```

정규화 실습에서 새로 등장하는 주요 도구를 소개한다. `train_test_split`은 데이터를 훈련용과 테스트용으로 나누는 함수이고, `LassoCV`, `RidgeCV`, `ElasticNetCV`는 각 방법의 교차 검증 버전으로, 최적의 규제 강도를 자동으로 찾아준다.

## 훈련/테스트 데이터 분할

정규화 모형의 예측 성능을 객관적으로 평가하기 위해 데이터를 훈련(Train)과 테스트(Test)로 나눈다. 최적 규제 강도는 훈련 데이터 안에서 교차 검증으로 선택하고, 각 방법의 테스트 MSE는 한 번도 보지 않은 테스트 데이터에서 계산한다. 계수 해석용 최종 모형은 같은 규제 강도로 전체 표준화 데이터에 다시 적합한다.

### [파이썬 코드 5-13]

```
# 훈련/테스트 데이터 50:50 분할
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=1
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

위 코드에서 `test_size=0.5`는 전체 데이터의 50%를 테스트용으로 분리한다는 의미이다. `random_state=1`을 고정하면 분석을 반복해도 동일한 분할 결과를 재현할 수 있다. `StandardScaler()`는 훈련 데이터에서만 적합한 뒤 테스트 데이터에는 변환만 적용한다.

## 라쏘 (LASSO)

**라쏘(LASSO; Least Absolute Shrinkage and Selection Operator)**는 회귀계수의 **절댓값**에 패널티를 부여하는 방법이다. 패널티가 강해질수록 일부 변수의 계수가 정확히 **0**으로 수축되어, 변수 선택 효과까지 함께 얻을 수 있다는 것이 가장 큰 특징이다. 규제 강도는 **alpha**(본문의  $\lambda$ )로 조절하며, 값이 클수록 패널티가 강해진다.

### 교차 검증으로 최적 $\lambda$ 선택

임의의 alpha 값을 직접 지정하는 대신, LassoCV를 사용하면 여러 후보 값에 대해 10-겹 교차 검증을 수행하여 평균 예측 오차가 가장 낮은 최적의 alpha를 자동으로 찾아준다.

#### [파이썬 코드 5-14]

```
# Candidate alpha values on a log scale
alphas = np.logspace(2, -5, 100)

# Select the LASSO penalty with training-set cross-validation
lasso_cv = LassoCV(alphas=alphas, cv=10, max_iter=10000)
lasso_cv.fit(X_train_scaled, y_train)

best_lambda_lasso = lasso_cv.alpha_
print(f"LASSO 최적 lambda: {best_lambda_lasso:.5f}")
```

LASSO 최적 lambda: 0.00572

교차 검증 결과, 최적  $\lambda = 0.00572$ 로 선택되었다. 이 규제 강도에서는 표준화된 최종 LASSO 모형의 8개 계수가 모두 0이 아니므로, 이 결과를 변수 제거의 근거로 해석해서는 안 된다.

### 교차 검증 곡선 시각화

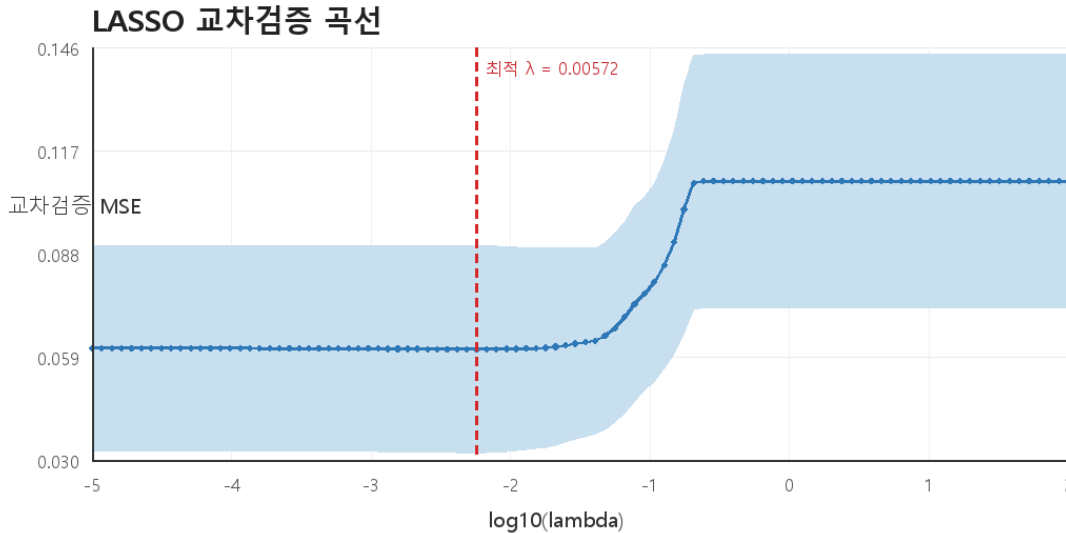
각 alpha 값에 따른 교차 검증 평균 오차(MSE)를 그래프로 확인한다. 빨간 점선이 가장 오차가 낮은 최적 지점을 나타낸다.

#### [파이썬 코드 5-15]

```
mean_mse = lasso_cv.mse_path_.mean(axis=1)
std_mse = lasso_cv.mse_path_.std(axis=1)

plt.figure()
plt.plot(np.log10(lasso_cv.alphas_), mean_mse, marker="o")
plt.fill_between(
    np.log10(lasso_cv.alphas_),
    mean_mse - std_mse,
    mean_mse + std_mse,
    alpha=0.3
)
plt.axvline(np.log10(best_lambda_lasso), color="red", linestyle="--")
```

```
plt.xlabel("log10(lambda)")
plt.ylabel("CV MSE")
plt.title("LASSO 교차검증 곡선")
plt.show()
```



[그림 A-5-4] 라쏘 교차 검증 곡선

[그림 A-5-4]의 그래프에서 음영 영역은 10개 폴드의 오차 변동성(표준편차)을 나타낸다. 빨간 점선은 현재 교차 검증에서 선택된 최적  $\lambda = 0.00572$ 의 위치를 표시한다.

### 최종 라쏘 모형 적합과 해석

최적의  $\lambda$ 를 찾았으므로, 전체 데이터셋에 최종 라쏘 모형을 적합하고 회귀계수를 확인한다.

#### [파이썬 코드 5-16]

```
# Refit on all standardized predictors for coefficient interpretation
final_scaler = StandardScaler()
X_scaled_all = final_scaler.fit_transform(X)
lasso_final = Lasso(alpha=best_lambda_lasso, max_iter=10000)
lasso_final.fit(X_scaled_all, y)

lasso_coef = pd.Series(lasso_final.coef_, index=X.columns).round(5)
print(lasso_coef)

# Test performance uses a model fitted only on the training data
lasso_test = Lasso(alpha=best_lambda_lasso, max_iter=10000).fit(X_train_scaled,
    y_train)
print(f"LASSO test MSE: {mean_squared_error(y_test,
    lasso_test.predict(X_test_scaled)):.5f}")
```

```

avg_apt_age          -0.03841
avg_TF_large         0.04501
cnt                  0.02372
rate_big_area_m2     0.04379
pop_50_69            -0.04905
rate_buy             -0.13859
rate_welfare_recipient -0.06600
avg_D_Sub            -0.04177
dtype: float64
LASSO test MSE: 0.05569

```

표준화된 설명변수 기준의 최종 LASSO 모형에서는 8개 계수가 모두 0이 아니다. 절대값이 가장 큰 계수는 rate\_buy(매매 거래 비율)의 -0.13859로, 다른 조건이 같을 때 매매 거래 비율이 상대적으로 높은 지역에서 종속변수가 낮아지는 방향의 관계가 나타난다. avg\_TF\_large(대단지 비율)와 rate\_big\_area\_m2(대형 평수 비율)는 양의 계수를 보인다. 표준화 계수는 원래 변수 1단위 증가가 아니라 설명변수 1표준편차 증가에 대응하는 계수라는 점에 유의한다.

### 데이터 분석가의 한마디

LASSO는 변수 선택과 정규화를 동시에 수행한다는 점에서 강력하지만, 한 가지 주의할 점이 있다. 설명변수들 사이의 상관관계가 매우 높을 때, LASSO는 관련 변수 중 하나를 임의로 선택하고 나머지를 0으로 만들어 버릴 수 있다. 이런 경우에는 아래에서 소개할 Elastic Net이 더 안정적인 대안이 될 수 있다.

## 릿지 (Ridge)

릿지(Ridge)는 회귀계수의 **제곱합**에 패널티를 부여하는 방법이다. 라쏘처럼 계수를 정확히 0으로 만들지는 않지만, 모든 변수의 계수를 전반적으로 줄여 모형을 안정화시킨다. 특히 설명변수들 사이에 **상관관계가 높아 추정이 불안정한 상황(다중공선성)**에서 릿지는 매우 효과적이다.

### 교차 검증으로 최적 $\lambda$ 선택

#### [파이썬 코드 5-17]

```

# Select the Ridge penalty with training-set cross-validation
ridge_cv = RidgeCV(alphas=alphas, cv=10)
ridge_cv.fit(X_train_scaled, y_train)

best_lambda_ridge = ridge_cv.alpha_
print(f"Ridge 최적 lambda: {best_lambda_ridge:.5f}")

```

```
Ridge 최적 lambda: 19.63041
```

## 최종 릿지 모형 적합과 해석

### [파이썬 코드 5-18]

```
ridge_final = Ridge(alpha=best_lambda_ridge)
ridge_final.fit(X_scaled_all, y)

ridge_coef = pd.Series(ridge_final.coef_, index=X.columns).round(5)
print(ridge_coef)

ridge_test = Ridge(alpha=best_lambda_ridge).fit(X_train_scaled, y_train)
print(f"Ridge test MSE: {mean_squared_error(y_test,
      ridge_test.predict(X_test_scaled)):.5f}")
```

```
avg_apt_age          -0.04199
avg_TF_large         0.04958
cnt                  0.03701
rate_big_area_m2     0.04658
pop_50_69           -0.06044
rate_buy             -0.13095
rate_welfare_recipient -0.06695
avg_D_Sub            -0.04481
dtype: float64
Ridge test MSE: 0.05180
```

라쏘와 달리 릿지의 계수는 작아지기는 하지만 **0이 되지는 않는다**. 즉 모든 변수가 모형에 남아 있  
되, 각 변수의 영향력이 완만하게 조정된다. 계수가 0이 아니므로 변수 선택 효과는 없지만, 그만큼  
다중공선성 상황에서 추정치가 안정적으로 유지된다.

### 데이터 분석가의 한마디

LASSO와 Ridge 중 어떤 방법이 더 좋은 질문에는 정답이 없다. 일반적으로 중요한 변수가 소  
수이고 나머지 변수는 거의 영향이 없다고 판단될 때는 LASSO가, 많은 변수가 비슷한 수준으로  
영향을 미치거나 변수 간 상관관계가 높을 때는 Ridge가 유리하다. 두 방법의 장점을 동시에 취  
하고 싶다면 아래에서 소개하는 Elastic Net을 고려해 볼 수 있다.

## Elastic Net

**Elastic Net**은 라쏘(L1 패널티)와 릿지(L2 패널티)를 동시에 사용하는 방법이다. `l1_ratio` 파라미터  
가 두 패널티의 혼합 비율을 결정하는데, 이 값이 1에 가까울수록 라쏘에, 0에 가까울수록 릿지에 가

값이다. 교차 검증을 통해 최적의 규제 강도(alpha)와 혼합 비율(l1\_ratio)을 동시에 탐색할 수 있다.

## 교차 검증으로 최적 $\lambda$ 와 l1\_ratio 동시 선택

### [파이썬 코드 5-19]

```
# Candidate L1 mixing ratios
l1_ratios = np.linspace(0.1, 1.0, 10)

enet_cv = ElasticNetCV(
    l1_ratio=l1_ratios,
    alphas=alphas,
    cv=10,
    max_iter=10000,
    random_state=1
)
enet_cv.fit(X_train_scaled, y_train)

best_l1_ratio = enet_cv.l1_ratio_
best_lambda_enet = enet_cv.alpha_
print(f"Elastic Net 최적 l1_ratio: {best_l1_ratio:.2f}")
print(f"Elastic Net 최적 lambda: {best_lambda_enet:.5f}")
```

```
Elastic Net 최적 l1_ratio: 0.10
Elastic Net 최적 lambda: 0.05591
```

`best_l1_ratio` 값을 보면 현재 데이터에 대해 Elastic Net이 라쏘와 릿지 중 어느 쪽에 더 가까운지 직관적으로 파악할 수 있다. 값이 1에 가까우면 변수 선택이 중요한 데이터, 0에 가까우면 계수 안정화가 더 중요한 데이터라고 해석할 수 있다.

## 최종 Elastic Net 모형 적합과 해석

### [파이썬 코드 5-20]

```
enet_final = ElasticNet(
    alpha=best_lambda_enet,
    l1_ratio=best_l1_ratio,
    max_iter=10000
)
enet_final.fit(X_scaled_all, y)

enet_coef = pd.Series(enet_final.coef_, index=X.columns).round(5)
print(enet_coef)

enet_test = ElasticNet(alpha=best_lambda_enet, l1_ratio=best_l1_ratio,
    max_iter=10000)
```

```
enet_test.fit(X_train_scaled, y_train)
print(f"Elastic Net test MSE: {mean_squared_error(y_test,
enet_test.predict(X_test_scaled)):.5f}")
```

```
avg_apr_age      -0.03620
avg_TF_large     0.04411
cnt              0.01924
rate_big_area_m2 0.04221
pop_50_69        -0.04400
rate_buy         -0.13336
rate_welfare_recipient -0.06466
avg_D_Sub        -0.04085
dtype: float64
```

```
Elastic Net test MSE: 0.05453
```

## A.5.3 차원 축소: 주성분 분석

### 주성분 분석(PCA)

**주성분 분석(Principal Component Analysis, PCA)**은 여러 개의 변수를 서로 상관되지 않는 새로운 축, 즉 주성분(Principal Components)으로 변환하는 차원 축소 기법이다. 원래 변수들이 가진 전체 변동을 최대한 보존하면서도 더 적은 수의 축으로 데이터를 요약할 수 있어, 변수가 많고 서로 상관 관계가 높은 데이터를 단순화하고 시각화하는 데 매우 유용하다.

### 탐색적 자료 분석

#### 데이터 로드

이번 분석에서는 서울시 지하철 역별 시간대별 승하차 인원 데이터를 활용한다. 2024년 1월 1일부터 12월 31일까지 서울시 관할 1~8호선의 각 역에서 시간대별 승하차 인원을 집계한 자료로, 서울 열린데이터광장에서 제공된다.

#### [파이썬 코드 5-21]

```
# 라이브러리 로딩
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
plt.rc('font', family='Malgun Gothic')
plt.rc('axes', unicode_minus=False)

# 데이터 로드
subway = pd.read_csv(
    "데이터/seoul_subway_hourly_2024.csv",
    encoding='utf-8'
)
```

이 데이터는 역별·호선별로 하루를 1시간 단위로 나눈 총 20개 시간 구간의 승하차 인원을 담고 있다. 즉, 하나의 관측치는 특정 역의 특정 날짜에 대한 **20차원의 시간대별 승하차 벡터**로 표현된다.

### 전처리: 요일 및 평일/주말 변수 생성

요일별 패턴을 분석하기 위해 날짜 정보로부터 요일 변수와 평일/주말 구분 변수를 추가한다.

#### [파이썬 코드 5-22]

```
subway["날짜"] = pd.to_datetime(subway["날짜"])

# 요일 변수 생성 (월~일)
subway["요일"] = subway["날짜"].dt.weekday + 1
subway["요일"] = subway["요일"].map({
    1:"월", 2:"화", 3:"수", 4:"목", 5:"금", 6:"토", 7:"일"
})
subway["요일"] = pd.Categorical(
    subway["요일"],
    categories=["월", "화", "수", "목", "금", "토", "일"],
    ordered=True
)

# 평일=1, 주말=0
subway["평일/주말"] = (
    subway["요일"].isin(["토", "일"])
    .map({True: 0, False: 1})
    .astype("category")
)
```

`pd.Categorical`을 이용해 요일에 순서를 지정해두면, 이후 시각화에서 요일이 월~일 순서로 자동 정렬된다.

### 승차 데이터 추출

주성분 분석은 수치형 변수에만 적용할 수 있다. 또한 승차와 하차 패턴이 서로 다를 수 있으므로 따로 분석하는 것이 바람직하다. 여기서는 먼저 승차 인원 데이터만 추출하여 분석한다.

**[파이썬 코드 5-23]**

```
# 승차 데이터 추출 및 수치형 변수(20 개 시간대) 선택
subway_boarding = subway[subway["구분"] == "승차"]
subway_boarding_vars = subway_boarding.iloc[:, 6:26]

print(subway_boarding_vars.shape)  # (관측치 수, 20)

(99705, 20)
```

추출된 데이터는 각 행이 특정 역·특정 날짜의 20개 시간대 승차 인원으로 구성된다.

**주성분 분석 실습****표준화 및 PCA 수행**

PCA를 적용하기 전에 반드시 **표준화**를 수행해야 한다. 시간대마다 승차 인원의 절대적인 규모가 다를 수 있어, 표준화 없이 PCA를 수행하면 인원이 많은 시간대가 분산을 지배하는 왜곡이 생긴다.

**[파이썬 코드 5-24]**

```
# 표준화
scaler = StandardScaler()
X_scaled = scaler.fit_transform(subway_boarding_vars)

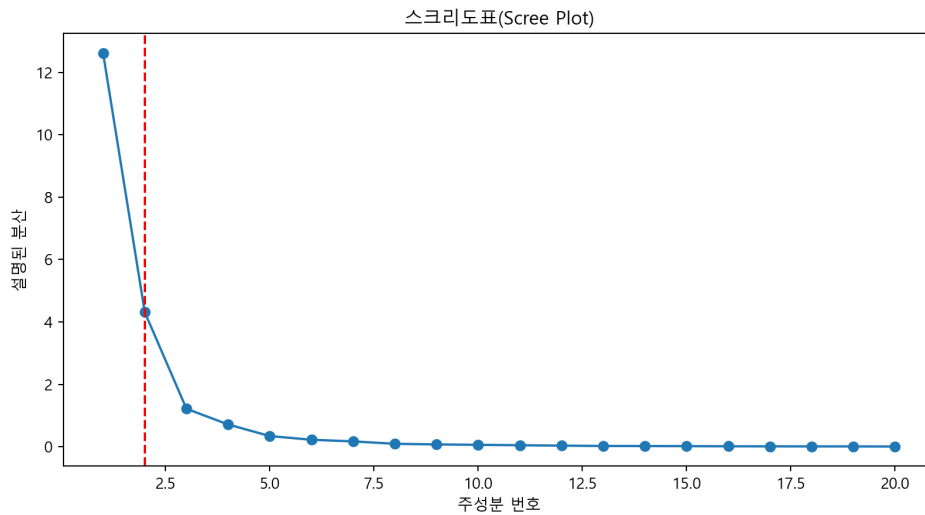
# 전체 주성분 계산
pca = PCA()
pca.fit(X_scaled)
```

**스크리도표로 주성분 개수 선택**

주성분의 개수를 결정하기 위해 **스크리도표(Scree Plot)**를 활용한다. 스크리도표는 각 주성분이 설명하는 분산의 크기를 순서대로 시각화한 것으로, 분산이 급격히 감소하다가 **완만해지는 지점(팔꿈치)**을 기준으로 주성분 개수를 선택한다.

**[파이썬 코드 5-25]**

```
plt.figure()
plt.plot(range(1, len(pca.explained_variance_) + 1),
         pca.explained_variance_, marker='o')
plt.axvline(2, color='red', linestyle='--')
plt.xlabel("주성분 번호")
plt.ylabel("설명된 분산")
plt.title("스크리도표(Scree Plot)")
plt.show()
```



[그림 A-5-5] 스크리도표(Scree Plot)

[그림 A-5-5]의 스크리도표를 보면 세 번째 주성분 이후부터 분산 감소 폭이 완만해지는 것을 확인할 수 있다. 이는 앞의 **두 개 주성분**이 데이터의 대부분의 변동을 설명하고 있음을 시사한다. 따라서 주성분의 개수를 2개로 설정하여 차원 축소를 수행한다.

## [파이썬 코드 5-26]

```
# 주성분 2 개로 축소
pca2 = PCA(n_components=2)
pca_scores = pca2.fit_transform(X_scaled)

subway_pc_df = pd.DataFrame(pca_scores, columns=["PC1", "PC2"])

# 누적 설명 분산 비율 확인
print(f"PC1 설명 분산 비율: {pca2.explained_variance_ratio_[0]:.3f}")
print(f"PC2 설명 분산 비율: {pca2.explained_variance_ratio_[1]:.3f}")
print(f"누적 설명 분산 비율: {pca2.explained_variance_ratio_.sum():.3f}")
```

```
PC1 설명 분산 비율: 0.631
PC2 설명 분산 비율: 0.216
누적 설명 분산 비율: 0.847
```

누적 설명 분산 비율을 확인하면 PC1과 PC2 두 개만으로 전체 변동의 몇 퍼센트를 설명하는지 한눈에 파악할 수 있다. 현재 PC1은 63%, PC2는 21%를 설명하고 있고 두 개만으로 약 84.7% 분산을 설명하고 있음을 확인할 수 있다.

## 주성분 해석

## 로딩(Loading) 확인

주성분의 의미를 해석하기 위해 각 변수의 **로딩(Loading)**을 확인한다. 로딩은 각 원래 변수(시간대)가 특정 주성분에 얼마나 기여하는지를 나타내는 값으로, 절댓값이 클수록 해당 주성분에 대한 영향력이 크다.

### [파이썬 코드 5-27]

```
loading_df = pd.DataFrame(
    pca2.components_.T,
    index=subway_boarding_vars.columns,
    columns=["PC1", "PC2"]
)
print(loading_df.round(3))
```

|         | PC1   | PC2    |
|---------|-------|--------|
| 06시 이전  | 0.139 | 0.360  |
| 06시-07시 | 0.134 | 0.403  |
| 07시-08시 | 0.122 | 0.403  |
| 08시-09시 | 0.142 | 0.390  |
| 09시-10시 | 0.189 | 0.340  |
| 10시-11시 | 0.233 | 0.214  |
| 11시-12시 | 0.253 | 0.093  |
| 12시-13시 | 0.258 | 0.055  |
| 13시-14시 | 0.263 | -0.007 |
| 14시-15시 | 0.263 | -0.073 |
| 15시-16시 | 0.263 | -0.106 |
| 16시-17시 | 0.262 | -0.122 |
| 17시-18시 | 0.240 | -0.167 |
| 18시-19시 | 0.219 | -0.174 |
| 19시-20시 | 0.248 | -0.171 |
| 20시-21시 | 0.251 | -0.172 |
| 21시-22시 | 0.244 | -0.174 |
| 22시-23시 | 0.236 | -0.154 |
| 23시-24시 | 0.229 | -0.125 |
| 24시 이후  | 0.182 | -0.046 |

로딩 값을 PC1과 PC2 각각 기준으로 정렬하면 어떤 시간대가 각 주성분에 큰 영향을 미치는지 쉽게 확인할 수 있다.

### [파이썬 코드 5-28]

```
# PC1 기준 정렬
print("=== PC1 기여도 ===")
print(loading_df.sort_values('PC1', ascending=False).round(3))

# PC2 기준 정렬
```

```
print("\n=== PC2 기여도 ===")
print(loading_df.sort_values('PC2', ascending=False).round(3))
```

```
=== PC1 기여도 ===
```

|         | PC1   | PC2    |
|---------|-------|--------|
| 15시-16시 | 0.263 | -0.106 |
| 13시-14시 | 0.263 | -0.007 |
| 14시-15시 | 0.263 | -0.073 |
| 16시-17시 | 0.262 | -0.122 |
| 12시-13시 | 0.258 | 0.055  |
| 11시-12시 | 0.253 | 0.093  |
| 20시-21시 | 0.251 | -0.172 |
| 19시-20시 | 0.248 | -0.171 |
| 21시-22시 | 0.244 | -0.174 |
| 17시-18시 | 0.240 | -0.167 |
| 22시-23시 | 0.236 | -0.154 |
| 10시-11시 | 0.233 | 0.214  |
| 23시-24시 | 0.229 | -0.125 |
| 18시-19시 | 0.219 | -0.174 |
| 09시-10시 | 0.189 | 0.340  |
| 24시 이후  | 0.182 | -0.046 |
| 08시-09시 | 0.142 | 0.390  |
| 06시 이전  | 0.139 | 0.360  |
| 06시-07시 | 0.134 | 0.403  |
| 07시-08시 | 0.122 | 0.403  |

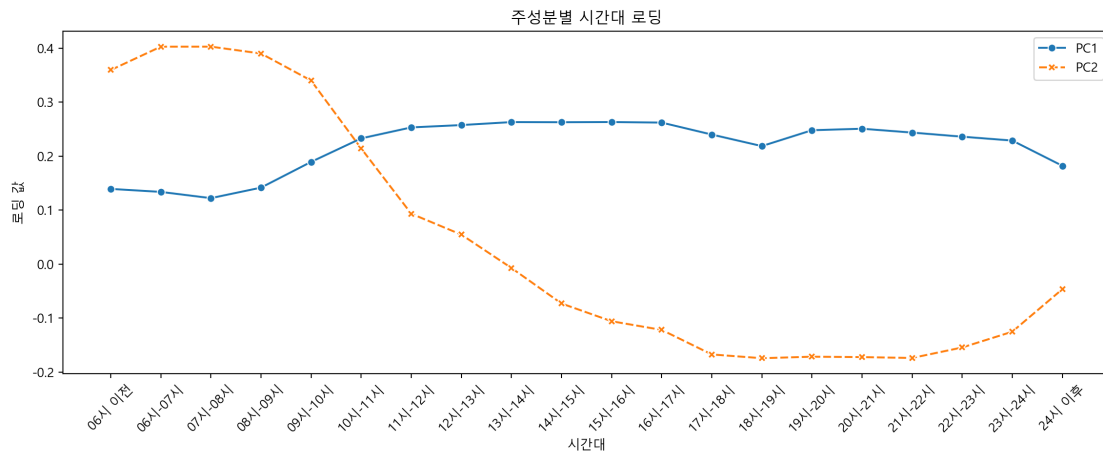
```
=== PC2 기여도 ===
```

|         | PC1   | PC2    |
|---------|-------|--------|
| 07시-08시 | 0.122 | 0.403  |
| 06시-07시 | 0.134 | 0.403  |
| 08시-09시 | 0.142 | 0.390  |
| 06시 이전  | 0.139 | 0.360  |
| 09시-10시 | 0.189 | 0.340  |
| 10시-11시 | 0.233 | 0.214  |
| 11시-12시 | 0.253 | 0.093  |
| 12시-13시 | 0.258 | 0.055  |
| 13시-14시 | 0.263 | -0.007 |
| 24시 이후  | 0.182 | -0.046 |
| 14시-15시 | 0.263 | -0.073 |
| 15시-16시 | 0.263 | -0.106 |
| 16시-17시 | 0.262 | -0.122 |
| 23시-24시 | 0.229 | -0.125 |
| 22시-23시 | 0.236 | -0.154 |
| 17시-18시 | 0.240 | -0.167 |
| 19시-20시 | 0.248 | -0.171 |
| 20시-21시 | 0.251 | -0.172 |
| 21시-22시 | 0.244 | -0.174 |
| 18시-19시 | 0.219 | -0.174 |

이를 선 그래프로 시각화하면 시간대별 패턴이 더욱 명확하게 드러난다.

#### [파이썬 코드 5-29]

```
plt.figure(figsize=(12, 5))
sns.lineplot(data=loading_df, markers=True)
plt.xticks(rotation=45)
plt.xlabel("시간대")
plt.ylabel("로딩 값")
plt.title("주성분별 시간대 로딩")
plt.tight_layout()
plt.show()
```



[그림 A-5-6] 주성분별 시간대 로딩

[그림 A-5-6]의 그래프를 해석하면 다음과 같다. **PC1**은 낮과 오후 시간대(10시~18시)에서 높은 로딩 값을 가지며, 전반적인 유동 인구 규모를 반영하는 축으로 볼 수 있다. **PC2**는 출근 시간대(7~9시)와 퇴근 시간대(18~20시)에서 서로 반대 방향의 로딩을 가지며, 전형적인 직장인 출퇴근 패턴을 요약하는 주성분으로 해석할 수 있다.

### 주성분 점수 시각화

이제 차원 축소된 PC1, PC2 값을 이용하여 관측치들의 분포를 살펴본다. 원래 데이터의 요일, 평일/주말, 호선, 역명 정보를 주성분 점수 데이터와 결합한다.

#### [파이썬 코드 5-30]

```
subway_boarding_reset = subway_boarding.reset_index(drop=True)

subway_pc_df["요일"] = subway_boarding_reset["요일"]
subway_pc_df["평일/주말"] = subway_boarding_reset["평일/주말"]
subway_pc_df["호선"] = subway_boarding_reset["호선"]
```

```
subway_pc_df["역명"] = subway_boarding_reset["역명"]
```

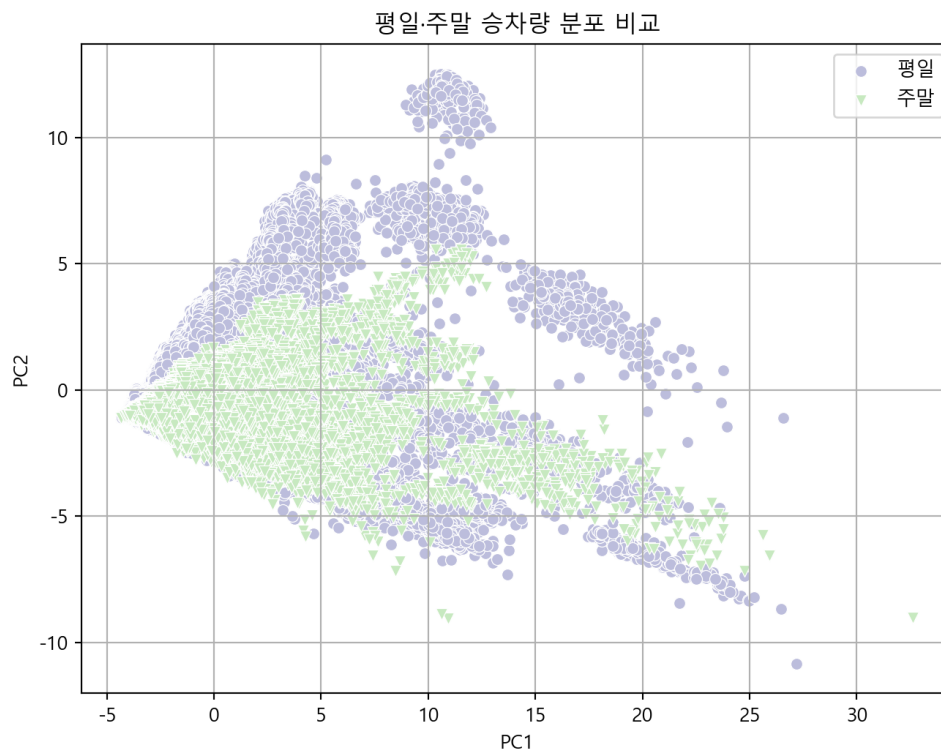
평일과 주말을 구분해 산점도를 그리면 두 집단이 어느 정도 분리되는 양상을 확인할 수 있다.

### [파이썬 코드 5-31]

```
plt.figure(figsize=(8, 6))

sns.scatterplot(
    data=subway_pc_df[subway_pc_df["평일/주말"] == 1],
    x="PC1", y="PC2",
    color="#bcbddc", marker="o", label="평일"
)
sns.scatterplot(
    data=subway_pc_df[subway_pc_df["평일/주말"] == 0],
    x="PC1", y="PC2",
    color="#c7e9c0", marker="v", label="주말"
)

plt.title("평일·주말 승차량 분포 비교")
plt.legend()
plt.grid(True)
plt.show()
```



**[그림 A-5-7] 평일·주말 승차량 분포 비교**

[그림 A-5-7]에서 PC2 값이 양의 방향으로 클수록 평일 패턴이 두드러지게 나타난다. 이는 출퇴근 시간대 승차량이 많은 평일의 특성이 PC2에 반영되었기 때문으로 해석할 수 있다. 반면 주말 관측치들은 PC2 값이 낮고 PC1 값이 넓게 분포하는 경향을 보이는데, 이는 주말에는 출퇴근 패턴 없이 낮 시간대 전반에 걸쳐 다양한 유동 인구 패턴이 나타나기 때문이다.

**데이터 분석가의 한마디**

PCA는 데이터를 "압축"하는 도구이기도 하지만, 동시에 데이터 속에 숨겨진 패턴을 발견하는 **탐색적 분석 도구**이기도 하다. 이번 분석에서 우리는 20개의 시간대 변수를 단 2개의 주성분으로 줄였음에도 불구하고, 평일과 주말의 승차 패턴 차이를 뚜렷하게 시각화할 수 있었다. 다만 PCA는 원래 변수들의 선형결합으로만 주성분을 구성하기 때문에, 비선형적인 구조가 강한 데이터에서는 한계가 있다. 이런 경우에는 t-SNE, UMAP 등 비선형 차원 축소 기법을 함께 고려해 볼 수 있다.

**A.5.4 차원 축소: 요인 분석**

**요인 분석(Factor Analysis, FA)**은 여러 개의 변수들을 소수의 공통 요인(Common Factor)으로 요약하는 차원 축소 기법이다. 주성분 분석(PCA)이 변수들의 전체 분산을 최대한 보존하는 새로운 축을 찾는 것이라면, 요인 분석은 변수들 간의 상관관계 구조를 설명하는 잠재적인 공통 원인을 발견하는데 초점을 맞춘다. 파이썬에서는 `factor_analyzer` 패키지를 사용하여 요인 분석을 수행할 수 있다.

**데이터 소개**

이번 분석에서는 K공기업에 재직 중인 임직원 **510명**을 대상으로 수집한 설문조사 데이터를 활용한다. 이 데이터는 선호 보상방식(외재적 보상, 내재적 보상, 복합적 보상)이 직무성과에 미치는 영향을 분석하기 위해 수집되었다.

요인 분석에 사용할 변수는 선호 보상방식에 관한 12개 설문 문항으로, 각 문항은 5점 리커트 척도로 측정되었다. 이론적으로 아래 세 가지 보상 유형으로 구분된다.

| 보상 유형  | 변수명      | 설명                           |
|--------|----------|------------------------------|
| 외재적 보상 | pay      | 금전적 보수(기본급, 성과급 등) 중요도       |
|        | welfare  | 금전적 복리후생(주택자금, 의료비 지원 등) 중요도 |
|        | training | 교육훈련 기회(학위 취득, 외부연수 등) 중요도   |

|        |                |                    |
|--------|----------------|--------------------|
|        | position       | 원하는 보직·인사이드 중요도    |
|        | work           | 유연근무·휴가 등 근무여건 중요도 |
| 내재적 보상 | achievement    | 직무 성취감 중요도         |
|        | responsibility | 직무 책임감 중요도         |
|        | interest       | 직무 흥미성 중요도         |
|        | pride          | 조직에 대한 자부심 중요도     |
|        | recognition    | 조직 내 인정 중요도        |
| 복합적 보상 | bonding        | 조직 내 유대감 중요도       |
|        | promotion      | 승진 중요도             |

요인 분석의 목적은 이 12개 문항이 이론적 구조(내재적·외재적·복합적 보상)에 실제로 부합하는지 타당도를 검토하는 것이다.

## 탐색적 자료 분석

### 데이터 로드

#### [파이썬 코드 5-32]

```
# 라이브러리 로딩
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from factor_analyzer import FactorAnalyzer

plt.rc('font', family='Malgun Gothic')
plt.rc('axes', unicode_minus=False)

# 데이터 로드
k_survey = pd.read_csv("데이터/k_corporation_reward_survey.csv")
```

**factor\_analyzer**는 파이썬에서 요인 분석을 수행하는 전용 패키지로, R의 `psych::fa()`와 유사한 기능을 제공한다. 설치가 필요한 경우 `pip install factor_analyzer`로 설치한다.

### 분석 변수 추출 및 표준화

요인 분석에 사용할 선호 보상방식 관련 12개 변수를 추출하고 표준화한다.

### [파이썬 코드 5-33]

```
# 선호 보상방식 변수(12 개) 추출
survey_vars = k_survey.copy()

# 변수 표준화
scaler = StandardScaler()
survey_scaled = pd.DataFrame(
    scaler.fit_transform(survey_vars),
    columns=survey_vars.columns
)
print(survey_scaled.head())
```

|   | pay       | bonding  | promotion | ... | pride     | work      | interest  |
|---|-----------|----------|-----------|-----|-----------|-----------|-----------|
| 0 | 0.702907  | 0.717274 | 1.470549  | ... | -0.526513 | -0.947570 | -0.389610 |
| 1 | 0.702907  | 0.717274 | 0.461155  | ... | -0.526513 | 0.652631  | -0.389610 |
| 2 | 0.702907  | 0.717274 | 0.461155  | ... | -0.526513 | -0.947570 | -0.389610 |
| 3 | -2.727549 | 0.717274 | 1.470549  | ... | 0.857620  | -2.547772 | 1.009694  |
| 4 | 0.702907  | 0.717274 | 0.461155  | ... | 0.857620  | 0.652631  | -1.788915 |

[5 rows x 12 columns]

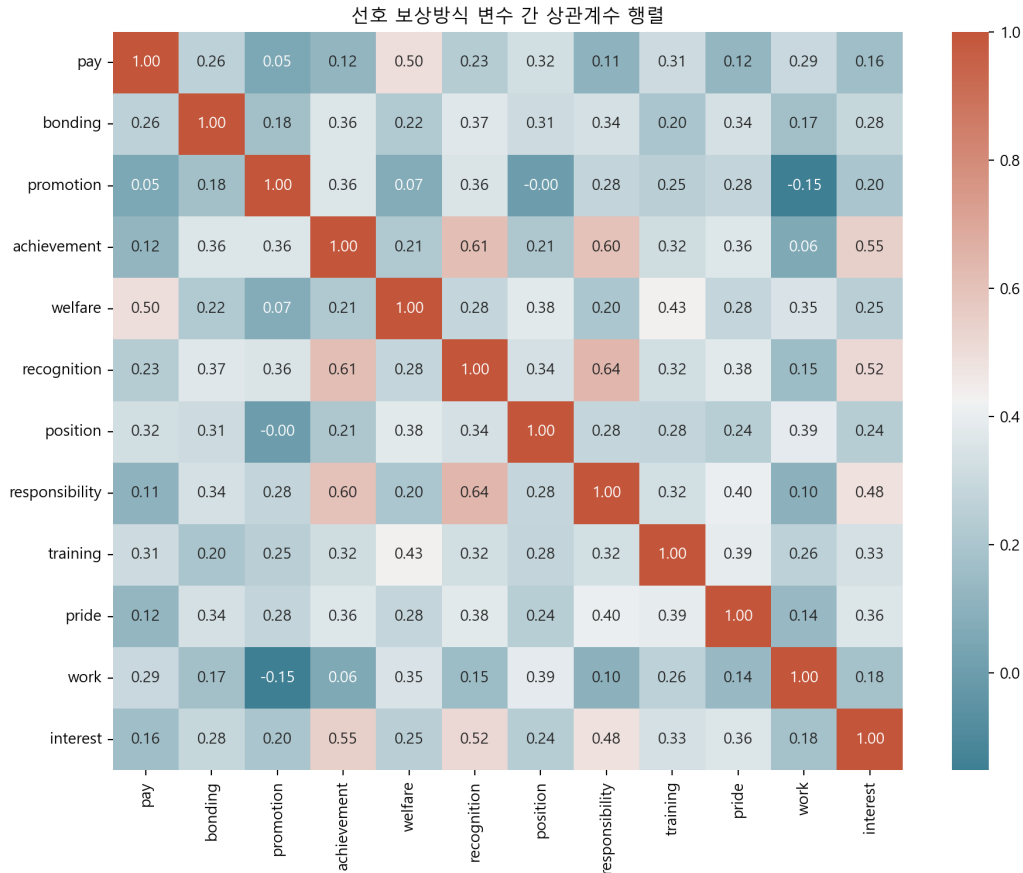
### 상관행렬 시각화

변수들 간에 충분히 높은 상관관계가 있어야 의미 있는 요인 구조가 존재한다. 분석 전에 상관행렬을 시각화하여 변수들의 구조를 파악한다.

### [파이썬 코드 5-34]

```
# 상관행렬 계산 및 히트맵 시각화
corr_matrix = survey_scaled.corr()

plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, fmt=".2f",
            cmap=sns.diverging_palette(220, 20, as_cmap=True))
plt.title("선호 보상방식 변수 간 상관관계수 행렬")
plt.tight_layout()
plt.show()
```



[그림 A-5-8] 선호 보상방식 변수 간 상관계수 행렬

[그림 A-5-8]의 히트맵을 살펴보면 **achievement**(성취감), **responsibility**(책임감), **recognition**(조직 내 인정) 등 내재적 보상 변수들끼리 높은 상관관계를 보이고, **pay**(보수), **welfare**(복리후생), **work**(근무여건) 등 외재적 보상 변수들끼리도 서로 높은 상관관계를 나타낸다. 이는 두 개의 뚜렷한 요인 구조가 존재할 가능성을 시사한다.

## 요인 분석

### 요인 분석 수행

요인의 개수를 2개로 설정하고, 요인 회전 방식으로 Varimax를 사용하여 요인 분석을 수행한다. Varimax 회전은 각 변수가 가능한 한 하나의 요인에만 높은 적재값을 갖도록 만들어 요인의 의미를 더 명확하게 해석할 수 있게 한다.

#### [파이썬 코드 5-35]

```
# 요인분석 수행 (요인 수 = 2, Varimax 회전)
fa = FactorAnalyzer(n_factors=2, rotation="varimax")
```

```
fa.fit(survey_scaled)

# 각 요인의 설명력 확인
ev, v = fa.get_eigenvalues()
variance = fa.get_factor_variance()

variance_df = pd.DataFrame(
    variance,
    index=["SS Loadings", "Proportion Var", "Cumulative Var"],
    columns=["F1", "F2"]
)
print(variance_df.round(4))
```

|                | F1     | F2     |
|----------------|--------|--------|
| SS Loadings    | 2.9430 | 1.9758 |
| Proportion Var | 0.2453 | 0.1646 |
| Cumulative Var | 0.2453 | 0.4099 |

출력 결과를 보면 F1은 전체 분산의 약 **24.5%**, F2는 약 **16.5%**를 설명하며, 두 요인을 합산한 누적 설명 분산 비율은 약 **41.0%**이다.

## 요인 적재값>Loading) 확인

요인 적재값은 각 변수가 특정 요인에 얼마나 강하게 연결되어 있는지를 나타낸다. 절댓값이 클수록 해당 요인에 대한 영향력이 크다.

### [파이썬 코드 5-36]

```
# 요인 적재값 행렬 추출
loading_df = pd.DataFrame(
    fa.loadings_,
    index=survey_vars.columns,
    columns=["F1", "F2"]
)

# F1 기준 내림차순 정렬
print("=== F1 적재값 ===")
print(loading_df.sort_values("F1", ascending=False).round(3))

print("\n=== F2 적재값 ===")
print(loading_df.sort_values("F2", ascending=False).round(3))
```

|                | F1    | F2    |
|----------------|-------|-------|
| achievement    | 0.791 | 0.096 |
| recognition    | 0.752 | 0.245 |
| responsibility | 0.745 | 0.146 |
| interest       | 0.603 | 0.231 |
| pride          | 0.495 | 0.259 |

```

promotion      0.463 -0.062
bonding         0.400  0.298
training        0.367  0.448
position        0.213  0.556
welfare         0.168  0.689
pay             0.076  0.600
work            0.002  0.572

```

```
=== F2 적재값 ===
```

```

              F1      F2
welfare       0.168  0.689
pay           0.076  0.600
work          0.002  0.572
position      0.213  0.556
training      0.367  0.448
bonding       0.400  0.298
pride         0.495  0.259
recognition   0.752  0.245
interest      0.603  0.231
responsibility 0.745  0.146
achievement   0.791  0.096
promotion     0.463 -0.062

```

F1의 적재값을 살펴보면 **achievement**(직무 성취감), **recognition**(조직 내 인정), **responsibility**(직무 책임감), **interest**(직무 흥미성), **pride**(조직 자부심) 순으로 높은 값을 보인다. 이는 직무 내면적 동기를 반영하는 요소들로, F1을 '내재적 보상 선호' 요인으로 해석할 수 있다. F2의 경우 **welfare**(복리후생), **pay**(보수), **work**(근무여건), **position**(보직 및 인사이동) 등 외재적 보상 항목들이 집중되어 나타난다. 따라서 F2를 '외재적 보상 선호' 요인으로 해석할 수 있다.

## 요인 적재값 산점도 시각화

Varimax 회전 후 각 변수의 요인 적재값을 산점도로 나타내면 변수들이 두 요인에 어떻게 배치되는지 한눈에 확인할 수 있다.

### [파이썬 코드 5-37]

```

# 회전 전후 요인 적재값 산점도 비교
fa_none = FactorAnalyzer(n_factors=2, rotation=None)
fa_none.fit(survey_scaled)

L_before = pd.DataFrame(fa_none.loadings_, index=survey_vars.columns,
                        columns=["F1", "F2"])
L_after = loading_df.copy()

fig, ax = plt.subplots(figsize=(8, 7))

# 회전 전 (연한 색)

```

```

ax.scatter(L_before["F1"], L_before["F2"], color="#6D9EC1", s=50, label="회전 전",
           zorder=3)
for var, row in L_before.iterrows():
    ax.text(row["F1"] + 0.02, row["F2"] + 0.01, var, fontsize=8, color="#6D9EC1")

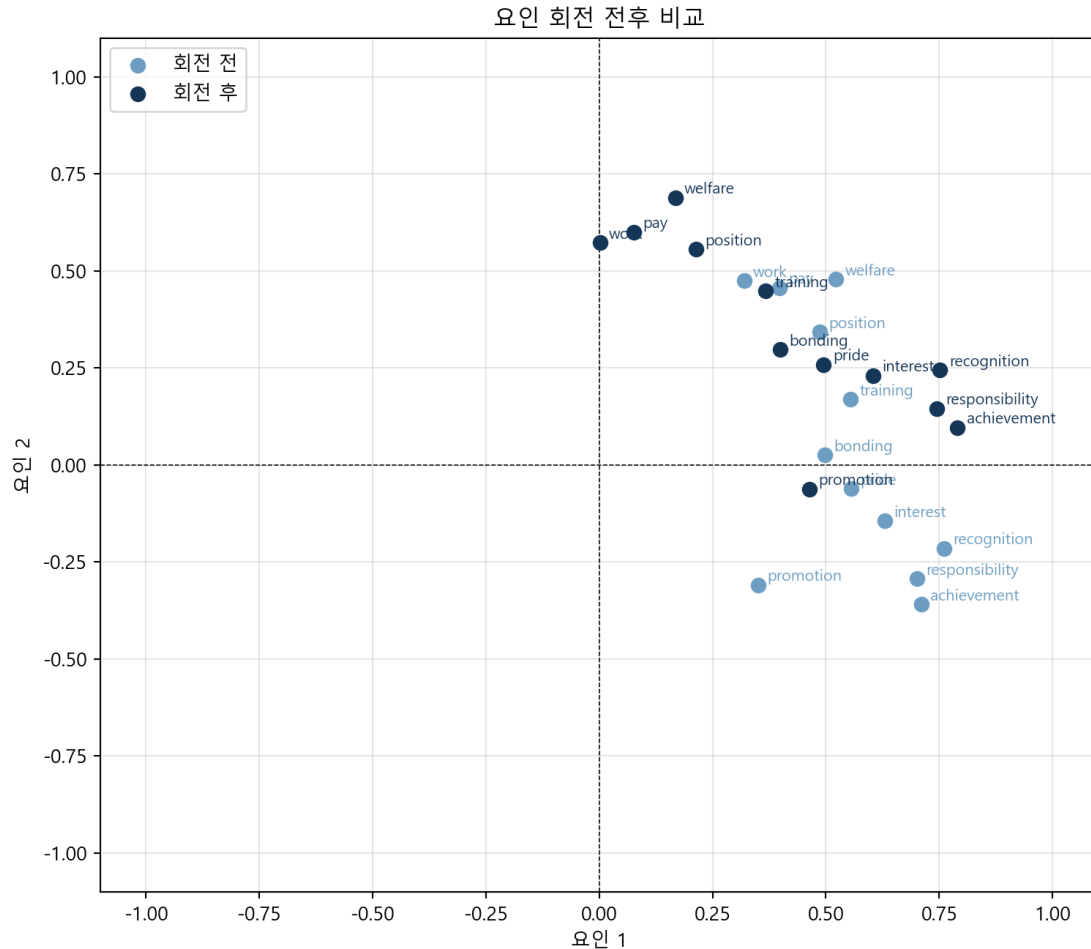
# 회전 후 (진한 색)
ax.scatter(L_after["F1"], L_after["F2"], color="#163657", s=50, label="회전 후",
           zorder=3)
for var, row in L_after.iterrows():
    ax.text(row["F1"] + 0.02, row["F2"] + 0.01, var, fontsize=8, color="#163657")

ax.axhline(0, color="black", linestyle="--", linewidth=0.6)
ax.axvline(0, color="black", linestyle="--", linewidth=0.6)
ax.set_xlim(-1.1, 1.1)
ax.set_ylim(-1.1, 1.1)
ax.set_xlabel("요인 1")
ax.set_ylabel("요인 2")
ax.legend(loc="upper left")
ax.grid(True, alpha=0.3)
plt.title("요인 회전 전후 비교")
plt.tight_layout()
plt.show()

```

```
(-1.1, 1.1)
```

```
(-1.1, 1.1)
```



[그림 A-5-9] 요인 회전 전후 비교

[그림 A-5-9]의 그래프에서 F1(가로축) 방향으로 위치한 변수는 **achievement**, **recognition**, **responsibility** 등 내재적 보상 변수들이며, F2(세로축) 방향으로 위치한 변수는 **welfare**, **pay**, **work** 등 외재적 보상 변수들이다. 두 요인이 각각의 보상 유형을 명확히 구분하고 있음을 확인할 수 있다. **promotion**(승진)이 F1(내재적 보상) 방향에 위치하는 것도 R 분석과 동일하게 확인된다.

### 요인 점수(Factor Score) 시각화

요인 점수는 각 응답자가 두 요인에서 어떤 위치에 있는지를 나타내는 값이다.

#### [파이썬 코드 5-38]

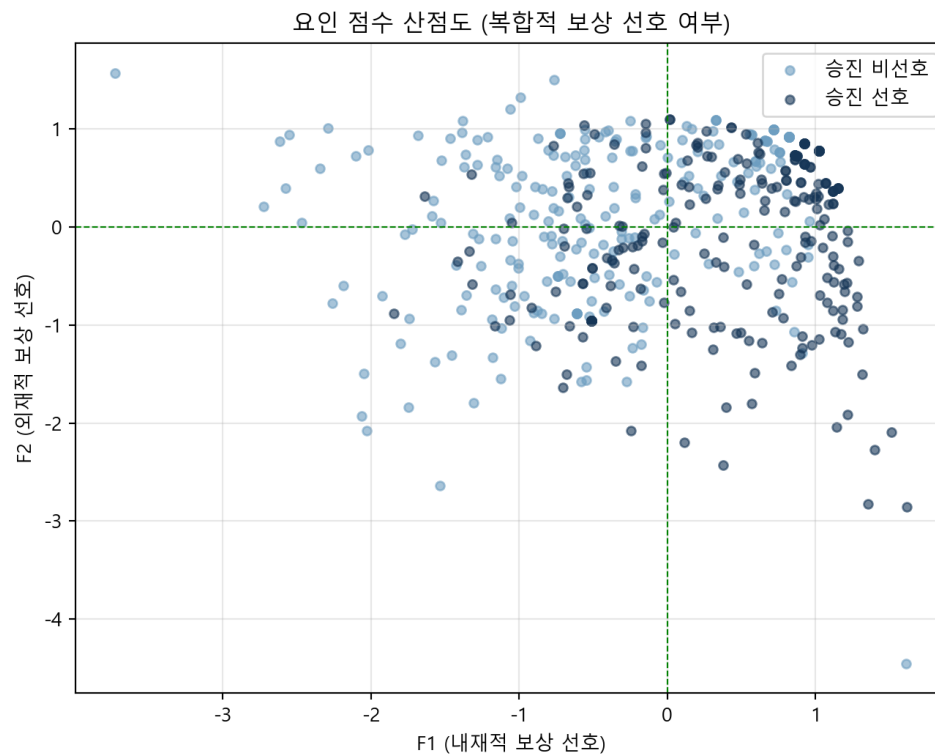
```
# 요인 점수 계산
factor_scores = pd.DataFrame(
    fa.transform(survey_scaled),
    columns=["F1", "F2"]
)
```

```
# 승진 선호 여부로 그룹 구분 (3 점 이상 = 선호, 미만 = 비선호)
factor_scores["reward_mix"] = (k_survey["promotion"] >= 3).map({True: "승진 선호",
    False: "승진 비선호"})

# 산점도 시각화
plt.figure(figsize=(8, 6))
colors = {"승진 선호": "#163657", "승진 비선호": "#6D9EC1"}

for group, sub in factor_scores.groupby("reward_mix"):
    plt.scatter(sub["F1"], sub["F2"], color=colors[group],
        alpha=0.6, label=group, s=20)

plt.axvline(0, color="green", linestyle="--", linewidth=0.8)
plt.axhline(0, color="green", linestyle="--", linewidth=0.8)
plt.xlabel("F1 (내재적 보상 선호)")
plt.ylabel("F2 (외재적 보상 선호)")
plt.title("요인 점수 산점도 (승진 선호 여부)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



[그림 A-5-10] 요인 점수 산점도

[그림 A-5-10]의 산점도를 살펴보면 F1 (내재적 보상 선호) 방향에서 두 집단의 차이가 뚜렷하게 나타난다. 승진을 선호하는 응답자들은 F1의 양의 방향에 더 많이 위치하고, 승진을 덜 중요하게 여기는 응답자들은 음의 방향에 더 많이 분포한다. 이는 **승진에 대한 선호가 외재적 보상보다는 내재적 동기 구조와 더 밀접하게 연관되어 있음**을 보여준다.

### 데이터 분석가의 한마디

이번 요인분석의 결과는 기존 이론적 구분(외재적/내재적/복합적 보상)과 대체로 일치하지만, 흥미로운 예외도 발견된다. '승진'(promotion)은 이론상 복합적 보상으로 분류되지만, 실제 응답 데이터에서는 내재적 보상 요인(F1)에 더 가깝게 위치했다. 이는 K공기업 임직원들이 승진을 단순한 외재적 보상이 아니라 조직 내 인정이나 성취감 같은 내면적 동기와 연관 지어 인식하고 있음을 시사한다. 요인분석은 이처럼 이론이 예상하지 못한 패턴을 발견하는 데에도 매우 유용한 탐색적 도구임을 확인할 수 있다.

## A.5.5 군집 분석

군집 분석(Clustering)은 정답(Label)이 주어지지 않은 자료에서 관측치들 간의 유사성을 바탕으로 비슷한 개체들을 그룹으로 묶는 대표적인 비지도 학습(Unsupervised Learning) 방법이다. 분석 목적에 따라 비슷한 특성을 가진 집단을 발견하거나 데이터 전체의 패턴을 탐색하는 데 널리 활용된다.

### 사전 준비

이번 장에서 추가로 필요한 라이브러리를 불러온다.

#### [파이썬 코드 5-39]

```
# 군집분석 관련 라이브러리 로딩
from sklearn.cluster import KMeans, AgglomerativeClustering
from scipy.cluster.hierarchy import linkage, dendrogram
```

**KMeans**는 K-평균 군집 분석, **AgglomerativeClustering**은 계층군집화를 수행하는 클래스다. **scipy**의 **linkage**와 **dendrogram**은 계층군집화의 병합 과정을 덴드로그램으로 시각화할 때 사용한다.

### K-평균 군집 분석

K-평균 알고리즘은 전체 관측치를 데이터 분석가가 미리 지정한  $k$ 개의 군집으로 나누는 방법이다. 각 군집은 중심점(Centroid)을 가지며, 모든 관측치는 가장 가까운 중심점에 속한 군집에 할당된다. 이후 할당된 데이터를 바탕으로 중심점을 다시 계산하고, 관측치를 재할당하는 과정을 반복하면서 **군집 내 거리 제곱합**을 최소화하는 방향으로 수렴한다.

## 데이터 준비

PCA 실습에서 사용한 서울시 지하철 승차 인원 데이터(subway\_boarding\_vars)를 그대로 사용한다. 군집화의 목적은 "각 역·날짜 관측치의 시간대별 승차 패턴 군집"을 찾아내는 것이다.

K-평균은 거리 기반 방법이므로 변수의 단위 차이가 결과에 직접적인 영향을 미친다. 출퇴근 시간대처럼 절대적인 승차량이 많은 시간대가 군집을 지배하는 왜곡을 막고자 반드시 **표준화**가 필요하다.

### [파이썬 코드 5-40]

```
# 표준화
scaler = StandardScaler()
subway_boarding_scaled = pd.DataFrame(
    scaler.fit_transform(subway_boarding_vars),
    columns=subway_boarding_vars.columns
)
```

## 군집모형 적합

### [파이썬 코드 5-41]

```
# K-평균 군집분석 (K=8)
k = 8

kmeans = KMeans(n_clusters=k, random_state=0, n_init=10)
cluster_labels = kmeans.fit_predict(subway_boarding_scaled)

print(cluster_labels[:10])

[1 5 6 6 6 5 5 5 6 5]
```

군집의 개수  $k$ 를 8로 설정한다. K-평균은 초기 중심점을 무작위로 배치하기 때문에 실행할 때마다 결과가 달라질 수 있다. 이를 방지하기 위해 `n_init=10`으로 서로 다른 초기값으로 10번 독립 실행한 뒤 가장 좋은 결과를 선택하도록 설정한다. 실행 결과 각 관측치에는 0부터 7까지 총 8개의 군집 라벨이 부여된다.

## 군집 결과 시각화

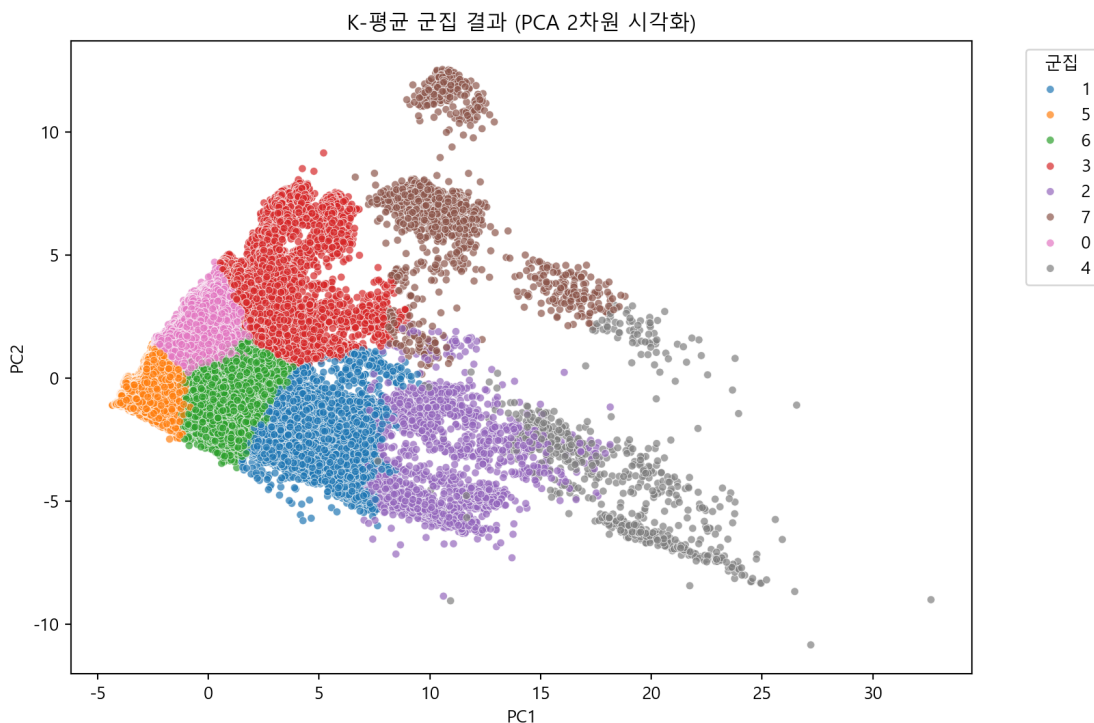
승차 데이터는 20차원이므로 2차원으로 직접 시각화할 수 없다. 이때 PCA를 활용하면 정보 손실을 최소화하며 2차원으로 축소한 뒤 군집의 분포를 확인할 수 있다.

### [파이썬 코드 5-42]

```
# PCA로 2차원 축소 후 군집 라벨 결합
pca = PCA(n_components=2)
subway_pca_2d = pca.fit_transform(subway_boarding_scaled)
```

```
subway_pc_df = pd.DataFrame(subway_pca_2d, columns=["PC1", "PC2"])
subway_pc_df["cluster"] = cluster_labels.astype(str)

# 산점도 시각화
plt.figure(figsize=(9, 6))
sns.scatterplot(data=subway_pc_df, x="PC1", y="PC2",
                hue="cluster", palette="tab10", alpha=0.7, s=20)
plt.title("K-평균 군집 결과 (PCA 2차원 시각화)")
plt.legend(title="군집", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.tight_layout()
plt.show()
```



[그림 A-5-11] K-평균 군집 결과 (PCA 2차원 시각화)

[그림 A-5-11]의 산점도에서 색상별로 구분된 군집이 서로 뚜렷하게 분리된 영역을 차지하고 있다면 군집화가 성공적으로 이루어진 것이다. PC1축(낮과 오후 시간대 승차량)과 PC2축(출퇴근 패턴)을 기준으로 비슷한 승차 패턴의 역들이 같은 군집으로 묶인 것을 확인할 수 있다.

### 데이터 분석가의 한마디

K-평균의 가장 큰 고민은  $K$ 를 **몇으로 설정할 것인가**이다. 실습에서는 8로 지정했지만, 실제 분석에서는  $K$ 를 1부터 늘려가며 군집 내 거리 제곱합이 급격히 감소하다가 완만해지는 지점(팔꿈치 지점)을 찾는 **엘보우 방법(Elbow Method)**을 자주 활용한다. 또한 K-평균은 초기 중심점에 따라

결과가 달라질 수 있고, 구형(Spherical) 군집이 아닌 복잡한 형태의 데이터에서는 성능이 떨어질 수 있다는 점도 함께 고려해야 한다.

## 계층군집화

계층군집화는 각 관측치를 개별 군집으로 시작하여 가장 가까운 군집끼리 단계적으로 병합해 나가는 **상향식(Bottom-up)** 방법이다. K-평균과 달리 k를 미리 지정할 필요가 없으며, 병합 과정을 **덴드로그램(Dendrogram)**으로 시각화하여 적절한 군집 수를 사후에 결정할 수 있다는 것이 가장 큰 장점이다.

## 데이터 준비

계층군집화 실습에는 자동차의 기술 제원을 담은 **mtcars** 데이터를 사용한다. 32개 자동차 모델에 대해 연비, 마력, 무게 등 11개의 수치형 변수가 기록되어 있다.

| 변수명  | 설명                 |
|------|--------------------|
| mpg  | 연비 (Miles/gallon)  |
| cyl  | 실린더 수              |
| disp | 배기량 (cu.in.)       |
| hp   | 마력                 |
| drat | 후륜 기어비             |
| wt   | 차량 무게 (1000 lbs)   |
| qsec | 1/4마일 주파 시간 (sec)  |
| vs   | 엔진 형태 (0=V형, 1=직렬) |
| am   | 변속기 (0=자동, 1=수동)   |
| gear | 전진 기어 수            |
| carb | 카뷰레터 수             |

### [파이썬 코드 5-43]

```
# mtcars 데이터 로드 및 표준화

mtcars = pd.read_csv("데이터/mtcars.csv")

X_mtcars_scaled = scaler.fit_transform(mtcars.iloc[:,1:12])
```

## 계층군집화 모형 적합

계층군집화에서는 군집 간 거리를 어떻게 정의할지 결정하는 **연결법(Linkage)**을 선택해야 한다. 여기서는 두 군집을 병합할 때 군집 내 오차제곱합의 증가를 최소화하는 **Ward 연결법**을 사용한다. Ward 연결법은 실무에서 가장 안정적이고 균형 잡힌 군집을 만들어 내어 자주 쓰인다.

### [파이썬 코드 5-44]

```
# Ward 연결법으로 계층군집화 (K=3)
hc_model = AgglomerativeClustering(n_clusters=3, linkage="ward")
hc_labels = hc_model.fit_predict(X_mtcars_scaled)
```

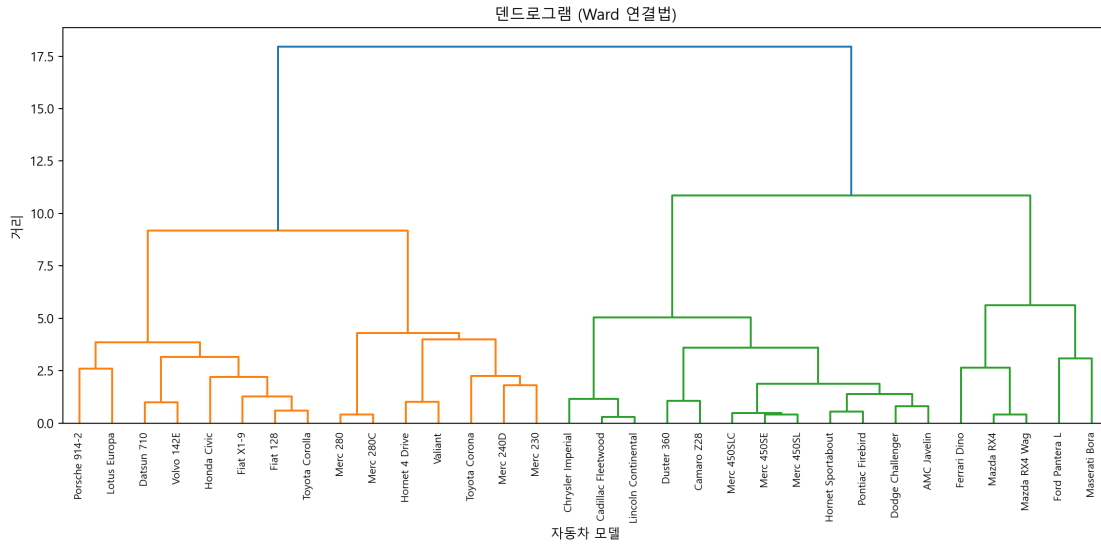
## 덴드로그램 시각화 및 해석

`scikit-learn`의 `AgglomerativeClustering`은 최종 군집 라벨만 반환하므로, 덴드로그램 시각화를 위해서는 `scipy`의 `linkage()` 함수로 병합 과정을 별도로 계산해야 한다.

### [파이썬 코드 5-45]

```
# scipy 로 병합 행렬 계산 후 덴드로그램 시각화
linked = linkage(X_mtcars_scaled, method="ward")

plt.figure(figsize=(12, 6))
dendrogram(
    linked,
    labels=mtcars.iloc[:, 0].tolist(),
    orientation="top",
    leaf_rotation=90,
    leaf_font_size=8
)
plt.title("덴드로그램 (Ward 연결법)")
plt.xlabel("자동차 모델")
plt.ylabel("거리")
plt.tight_layout()
plt.show()
```



[그림 A-5-12] 계층군집화 덴드로그램(Ward 연결법)

[그림 A-5-12]의 덴드로그램은 32개 자동차 모델이 서로 유사한 순서대로 단계적으로 병합되어 가는 전체 과정을 보여준다. 아래에서 위로 올라갈수록 더 이질적인 군집끼리 합쳐지며, y축의 거리는 병합 시 두 군집 간의 이질성 정도를 나타낸다.

가장 아래쪽에서는 "Mazda RX4"와 "Mazda RX4 Wag"처럼 제원이 거의 동일한 모델들이 먼저 합쳐진다. 거시적으로는 전체 데이터가 **소형·고연비 차량 군집**과 **대형·고성능 차량 군집**으로 크게 양분된다. [그림 A-5-12]의 덴드로그램을 적절한 높이에서 수평으로 자르면 원하는 개수의 군집을 얻을 수 있다. 예를 들어 중간 높이에서 자르면 3~4개의 군집을 얻을 수 있고, 더 낮게 자르면 더 세밀한 군집으로 나뉜다.

### 데이터 분석가의 한마디

K-평균과 계층군집화는 서로 다른 장단점을 가진다. K-평균은 대용량 데이터에서 빠르게 작동하지만  $k$ 를 미리 정해야 하고 초기값에 민감하다. 반면 계층군집화는 덴드로그램을 통해 데이터의 계층 구조를 한눈에 파악할 수 있고  $k$ 를 사후에 결정할 수 있다는 장점이 있지만, 관측치 수가 많아질수록 계산 시간이 급격히 증가한다. 실제 분석에서는 두 방법을 함께 사용하여 결과를 교차 확인하는 것이 좋다. 계층군집화 덴드로그램으로 적절한 군집 수를 파악한 뒤, 그  $k$  값으로 K-평균을 수행하는 방식이 대표적인 조합이다.